# 第五章 自底向上的语法分析

重点：自底向上分析的基本思想，算符优先分析法的基本思想，简单算符优先分析法。LR 分析器的基本构造思想，LR分析算法，规范句型活前缀及其识别器——DFA，LR(0)分析表的构造，SLR(1)分析表的构造, LR(1)分析表的构造。

难点：求FIRSTOP 和LASTOP，算符优先关系的确定，算符优先分析表的构造，素短语与最左素短语的概念。规范句型活前缀，LR(0)项目集闭包与项目集规范族，它们与句柄识别的关系，活前缀与句柄的关系, LR(1)项目集闭包与项目集规范族。

# 第5章 自底向上的语法分析


5.1 自底向上的语法分析概述

5.2 算符优先分析法

5.3 LR分析法

5.4 语法分析程序的自动生成工具Yacc

5.5 本章小结

# 5.1 自底向上的语法分析概述

**思想**

从输入串出发，反复利用产生式进行归约，如果最后能得到文法的开始符号，则输入串是句子，否则输入串有语法错误。

**核心**

寻找句型中的当前归约对象——“句柄”进行归约,用不同的方法寻找句柄，就可获得不同的分析方法

# 例5.1 一个简单的归约过程

- 设文法G为： S→aABe A→Abc|b B→d

句子分析：

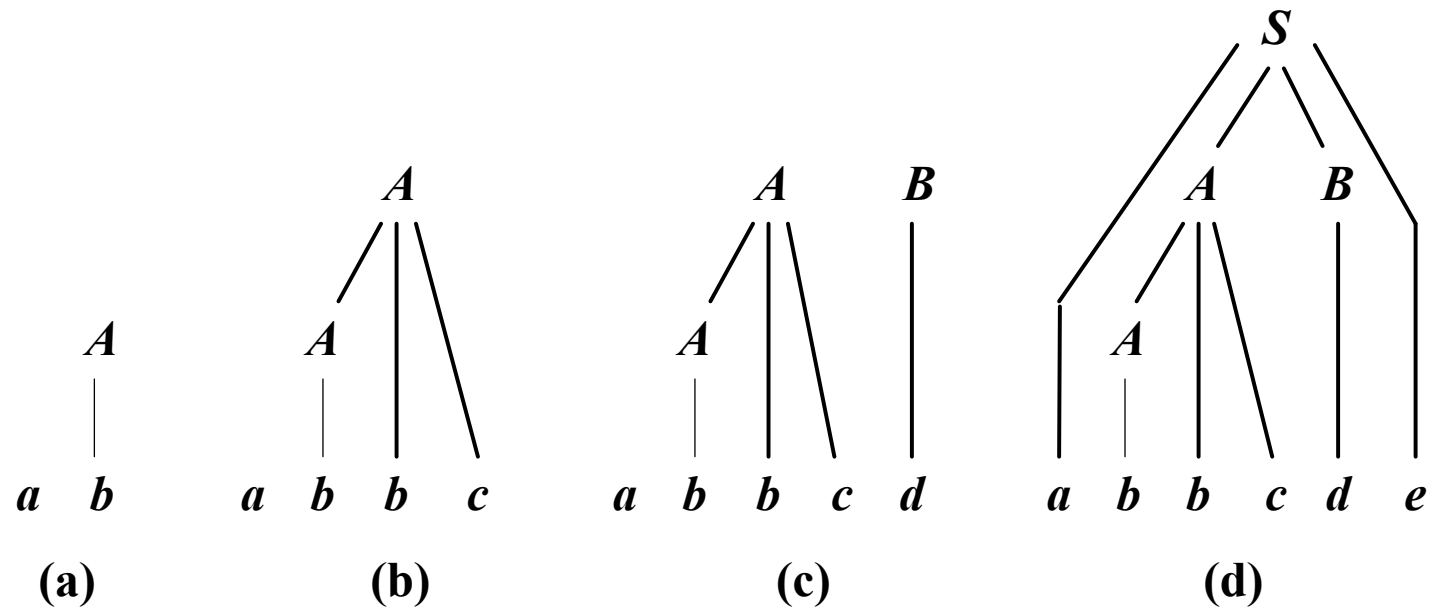
abbcde

⇐aAbcde

⇐aAde

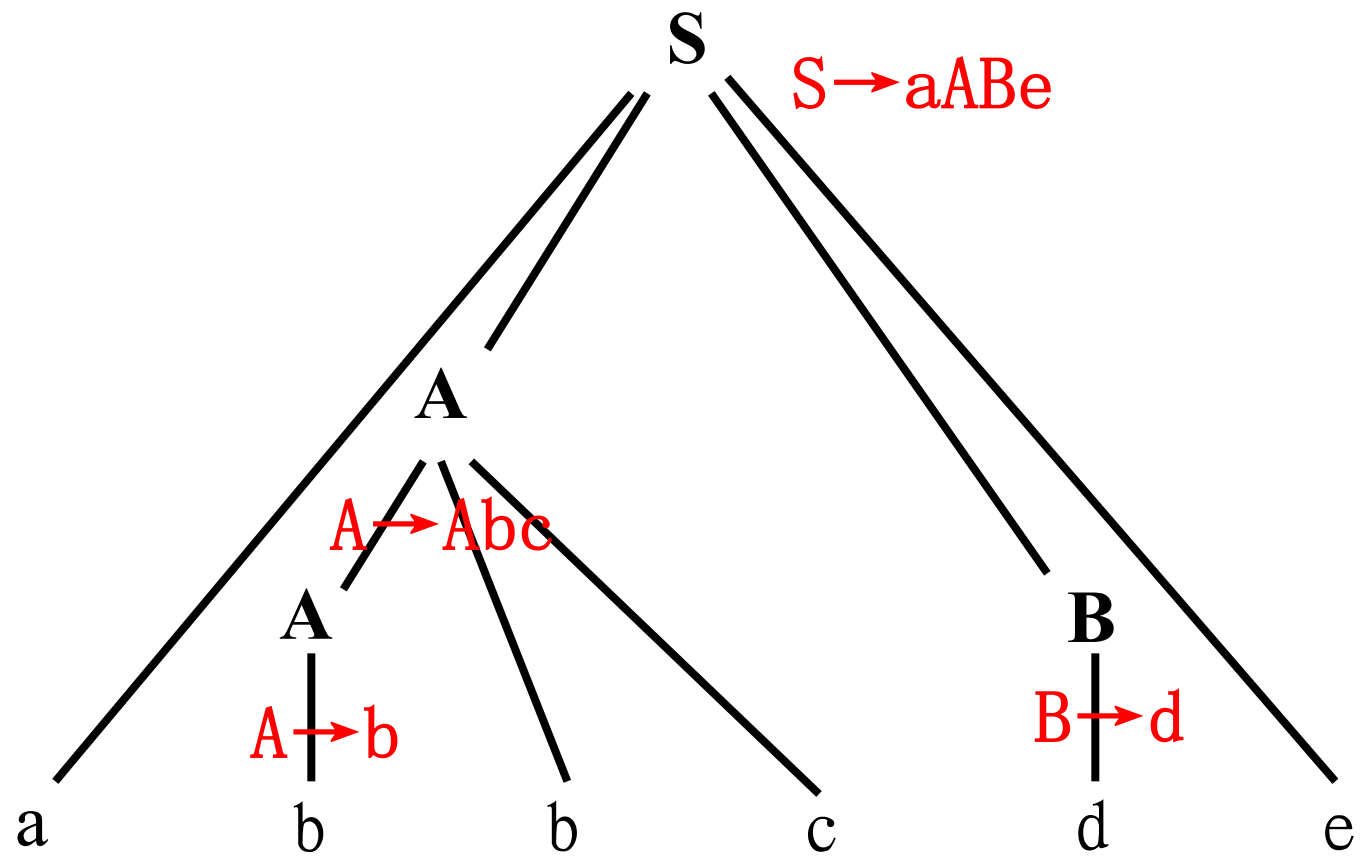
⇐aABe

⇐S




语法树的形成过程

# 语法分析树的生成演示



# 用子树解释短语，直接短语，句柄

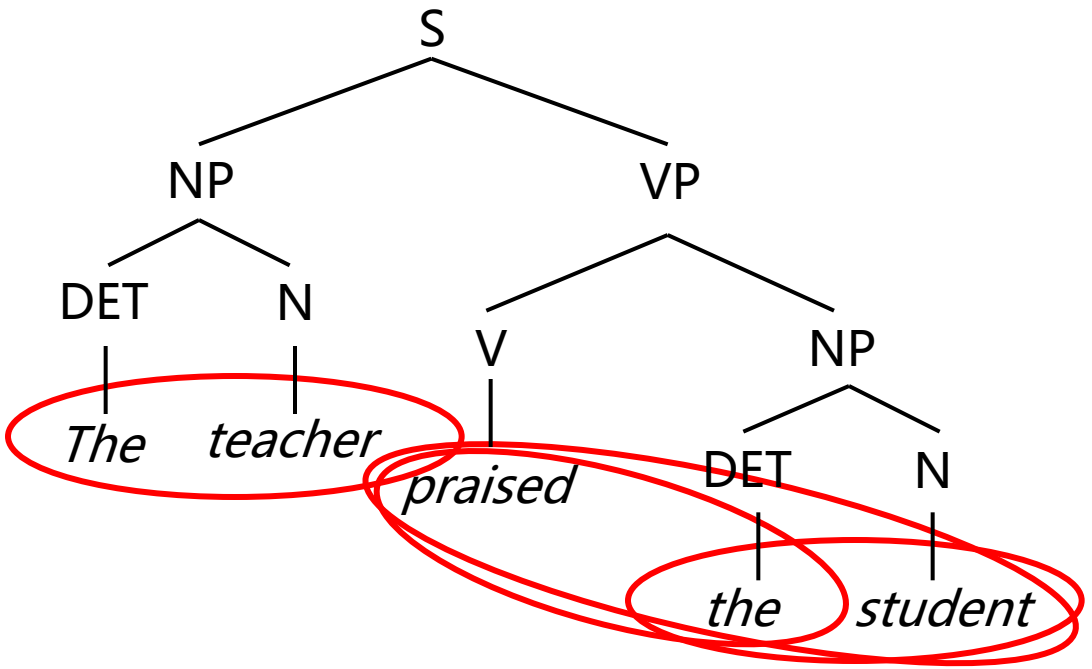
- 短语：一棵子树的所有叶子自左至右排列起来形成一个相对于子树根的短语。
- 直接短语：仅有父子两代的一棵子树，它的所有叶子自左至右排列起来所形成的符号串。
- 句柄：一个句型的分析树中最左那棵只有父子两代的子树的所有叶子的自左至右排列。
- 例如，对表达式文法$G[E]$和句子$a_1+a_2*a_3$，挑选出推导过程中产生的句型中的短语，直接短语，句柄

# 短语

- 定义：令G是一个文法，S是文法的开始符号，假定αβδ是文法G的一个句型，如果有

$$S \overset{*}{\Rightarrow} \alpha A \delta \quad 且 \quad A \overset{+}{\Rightarrow} \beta$$

则β称是句型αβδ相对于非终结符A的短语。



# 短语、直接短语和句柄

考虑文法G(E)：

$E \rightarrow T \mid E+T$

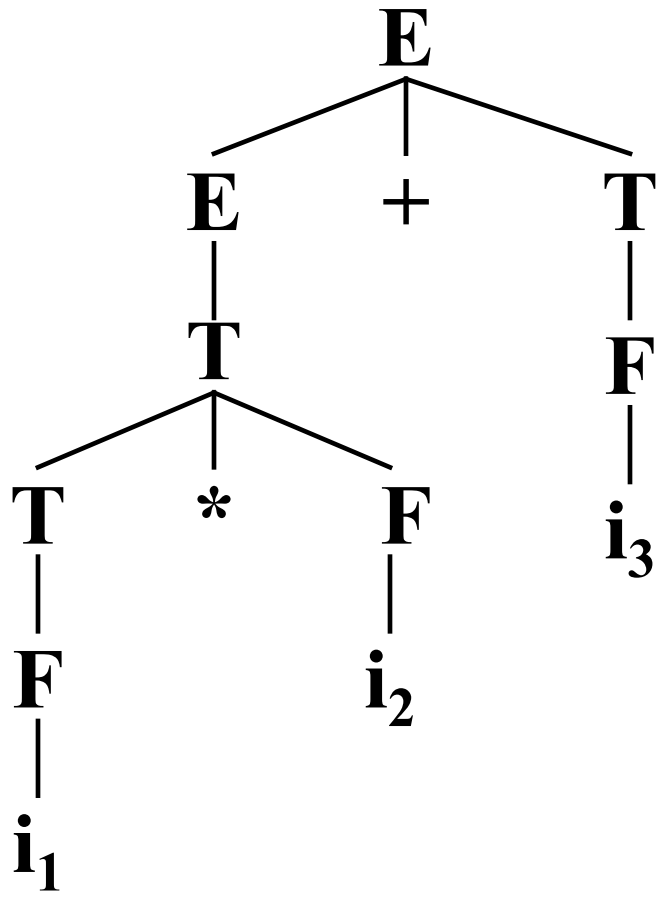
$T \rightarrow F \mid T*F$

$F \rightarrow (E) \mid i$

和句型$i_1*i_2+i_3$：

- 短语：　$i_1$，　$i_2$，　$i_3$，　$i_1*i_2$，　$i_1*i_2+i_3$
- 直接短语：　$i_1$，　$i_2$，　$i_3$
- 句柄：　$i_1$

$E \Rightarrow E+T$
$\Rightarrow E+F$
$\Rightarrow E+i_3$
$\Rightarrow T+i_3$
$\Rightarrow T*F+i_3$
$\Rightarrow T*i_2+i_3$
$\Rightarrow F*i_2+i_3$
$\Rightarrow i_1*i_2+i_3$



# 单选题 1分

设置

设有文法G[T]:

T→T*F|F

F→F个P|P

P→(T)|a

该文法句型T*P个(T*F)的句柄是下列符号串( )。

A （T*F）

B T*F

C P

D P个(T*F)

提交

# 单选题 1分

设置

设有文法G[T]:

T→T*F|F

F→F个P|P

P→(T)|a

该文法句型T*F个(T*F)的句柄是下列符号串( )。

A （T*F）

B T*F

C P

D P个(T*F)

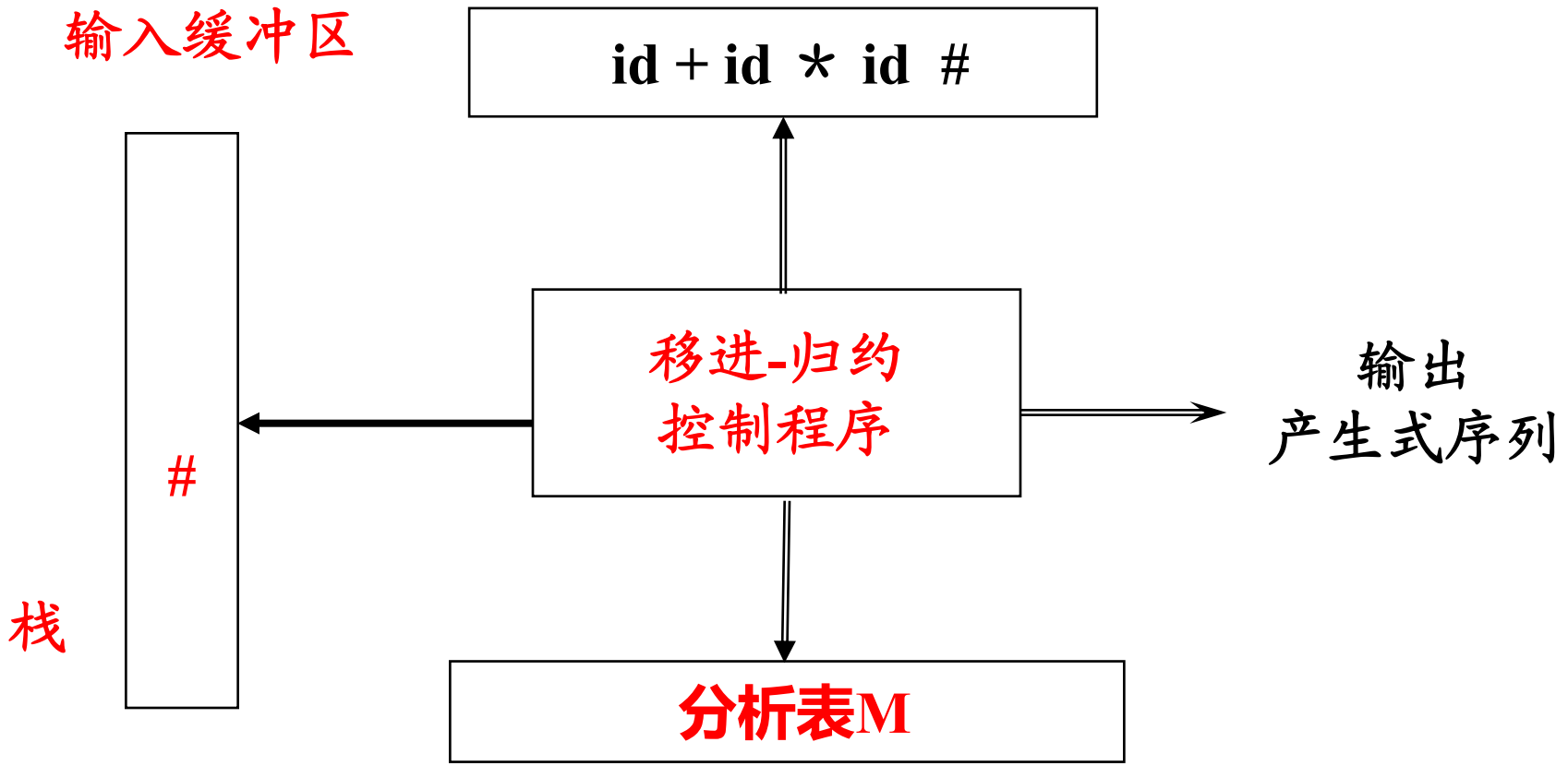
提交

# 5.1.1 移进-归约分析

## ▶系统框架

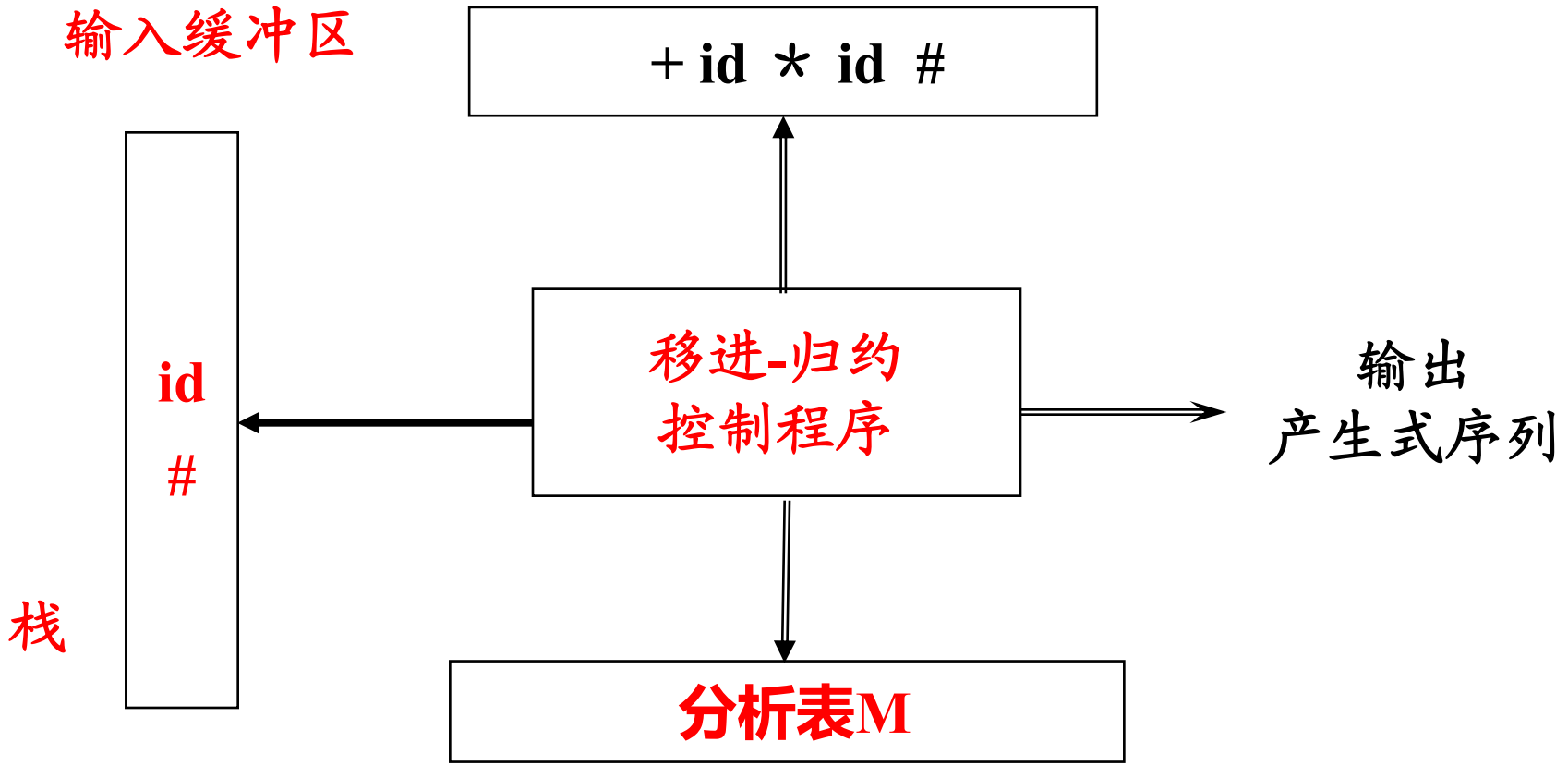
- 采用表驱动的方式实现
- 输入缓冲区：保存输入符号串
- 分析栈：保存语法符号—已经得到的那部分分析结果
- 控制程序：控制分析过程，输出分析结果——产生式序列
- 格局：栈+输入缓冲区剩余内容=“句型”

# 移进-归约语法分析器的总体结构



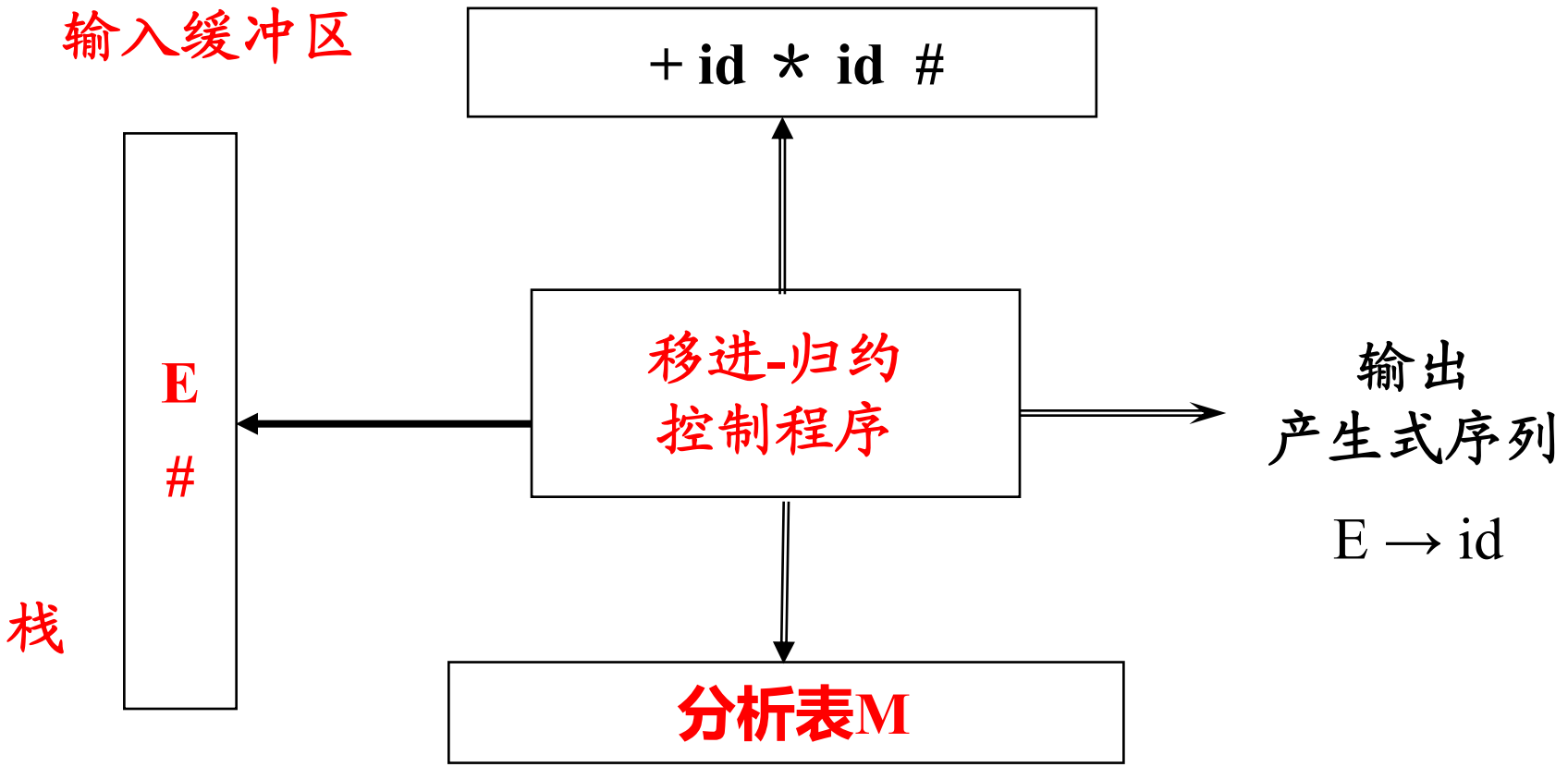

栈内容+输入缓冲区内容 = #“当前句型”#

# 移进-归约语法分析器的总体结构



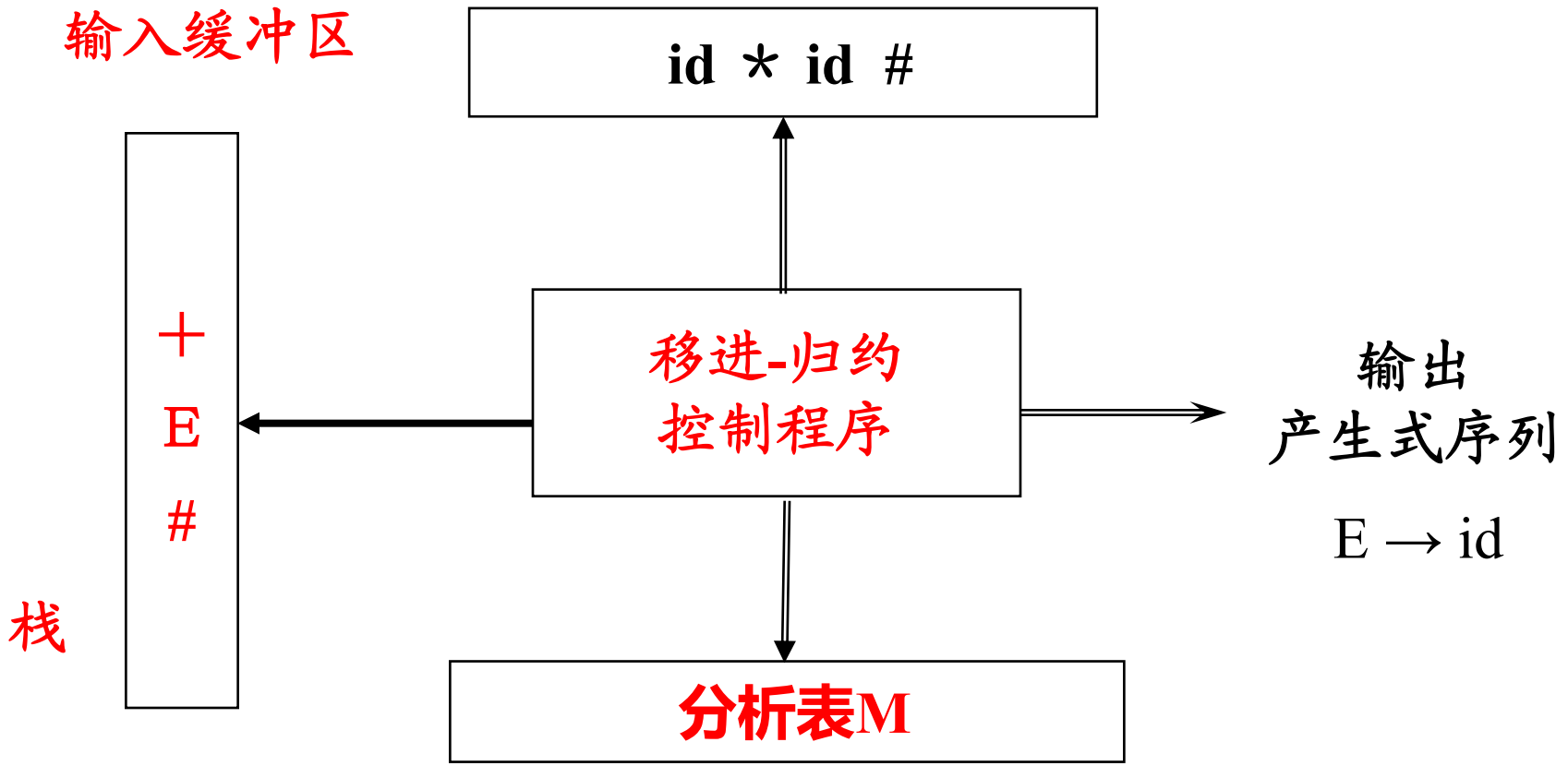

栈内容+输入缓冲区内容 = #“当前句型”#

# 移进-归约语法分析器的总体结构



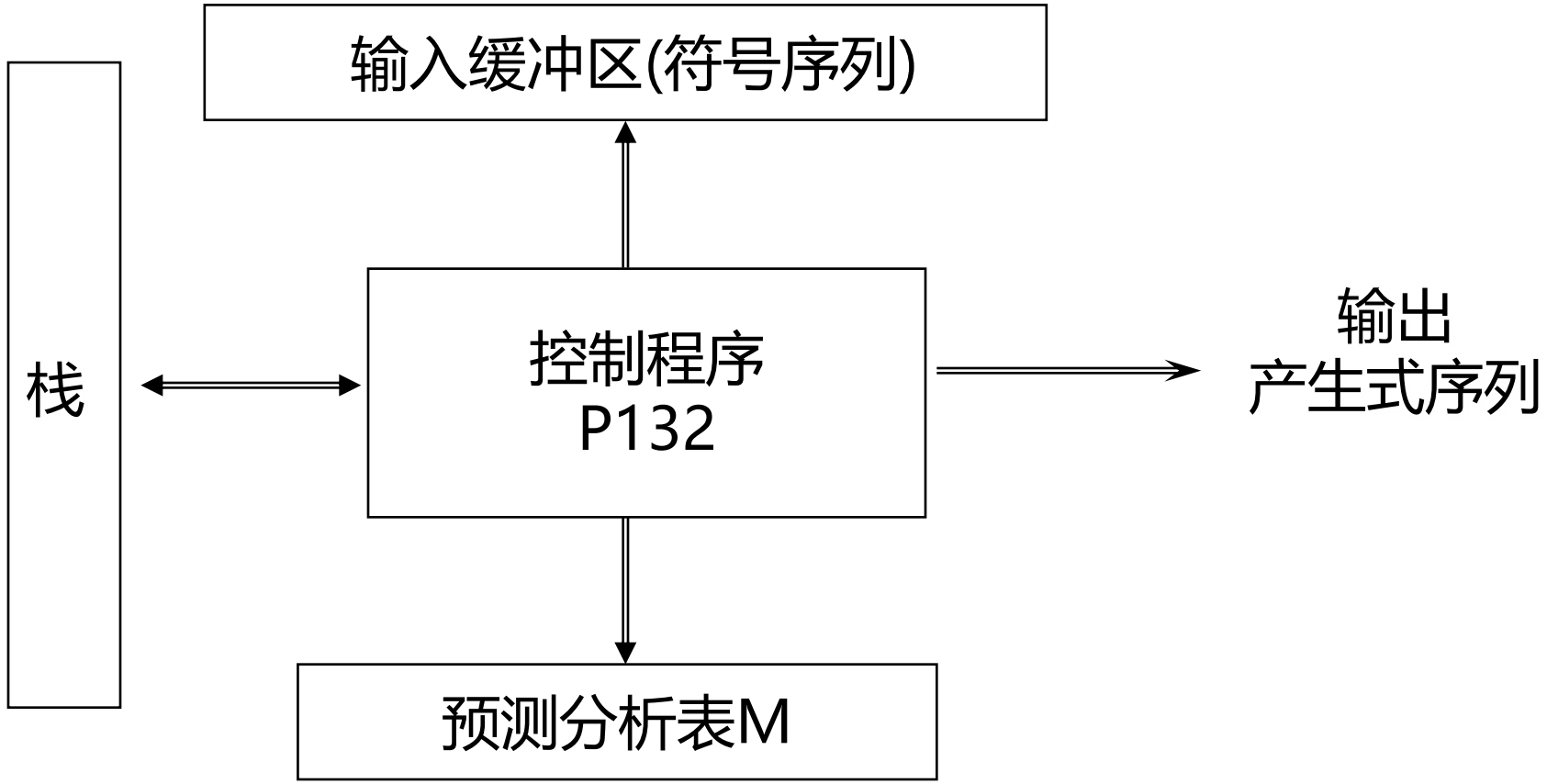

栈内容+输入缓冲区内容 = #“当前句型”#

# 移进-归约语法分析器的总体结构




栈内容+输入缓冲区内容 = #“当前句型”#

# 与LL(1)的体系结构比较



# 移进-归约分析的工作过程

- 系统运行
  - 开始格局
    - 栈：#；输入缓冲区：w#
  - 存放已经分析出来的结果,并将读入的符号送入栈，一旦句柄在栈顶形成，就将其弹出进行归约，并将结果压入栈
    - 问题：系统如何发现句柄在栈顶形成?
  - 正常结束：栈中为 #S，输入缓冲区只有 #

# 输出结果表示：用产生式序列表示语法分析树

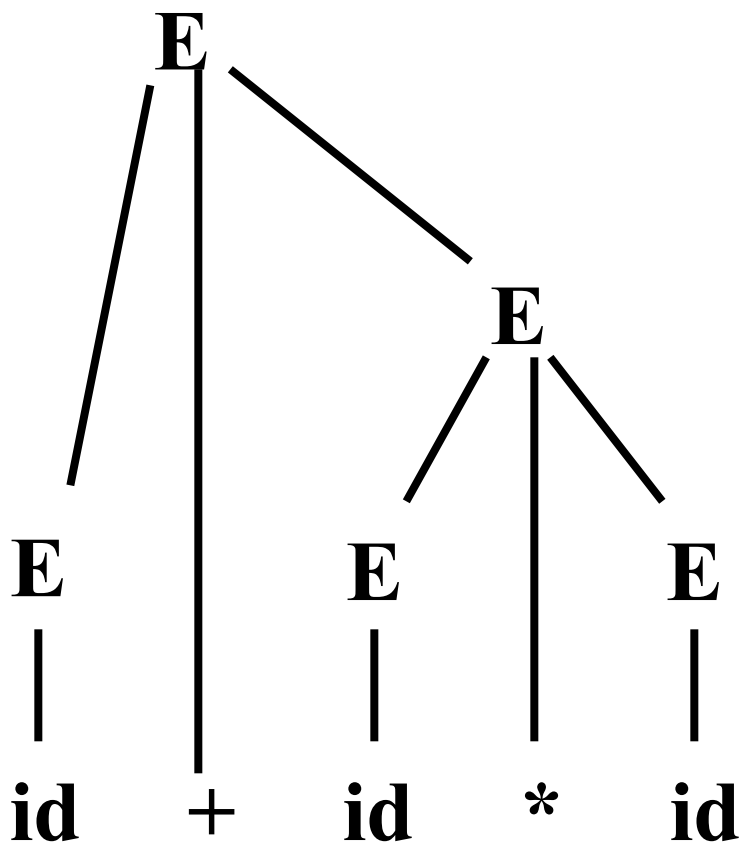
$E \rightarrow id$

$E \rightarrow id$

$E \rightarrow id$

$E \rightarrow E * E$

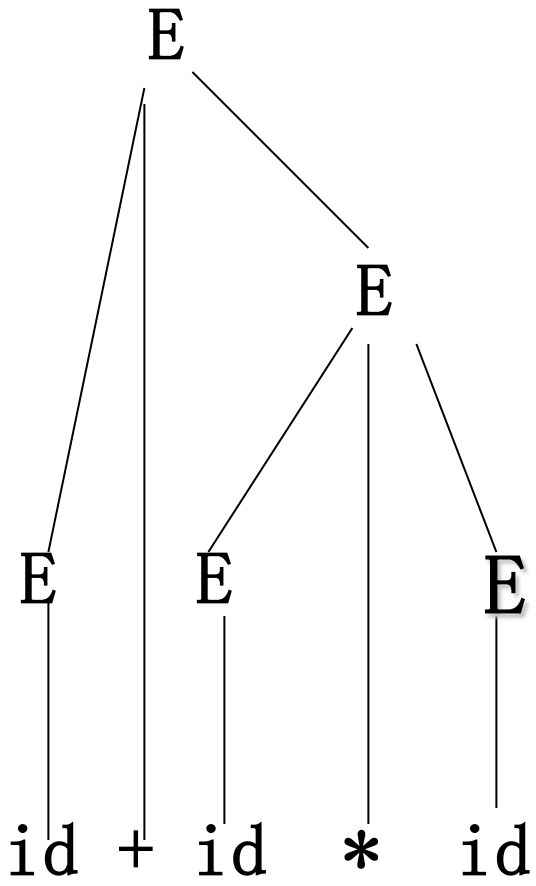
$E \rightarrow E + E$




**例5.2 E→E+E|E*E|(E)|id**

| | 动作 | 栈 | 缓冲区 |
|---|---|---|---|
| 1 | | # | id1+id2*id3# |
| 2 | 移进 | #id1 | +id2*id3# |
| 3 | 归约 E→id | #E | +id2*id3# |
| 4 | 移进 | #E+ | id2*id3# |
| 5 | 移进 | #E+id2 | *id3# |
| 6 | 归约 E→id | #E+E | *id3# |
| 7 | 移进 | #E+E* | id3# |
| 8 | 移进 | #E+E*id3 | # |
| 9 | 归约 E→id | #E+E*E | # |
| 10 | 归约 E→E*E | #E+E | # |
| 11 | 归约 E→E+E | #E | # |
| 12 | 接受 | | |

例5.2 分析过程




例5.2 E→E+E|E*E|(E)|id

# 分析器的四种动作

1) 移进：将下一输入符号移入栈

2) 归约：用产生式左侧的非终结符替换栈顶的句柄（某产生式右部）

3) 接受：分析成功

4) 出错：出错处理

？？决定移进和归约的依据是什么—回头看是否可以找到答案

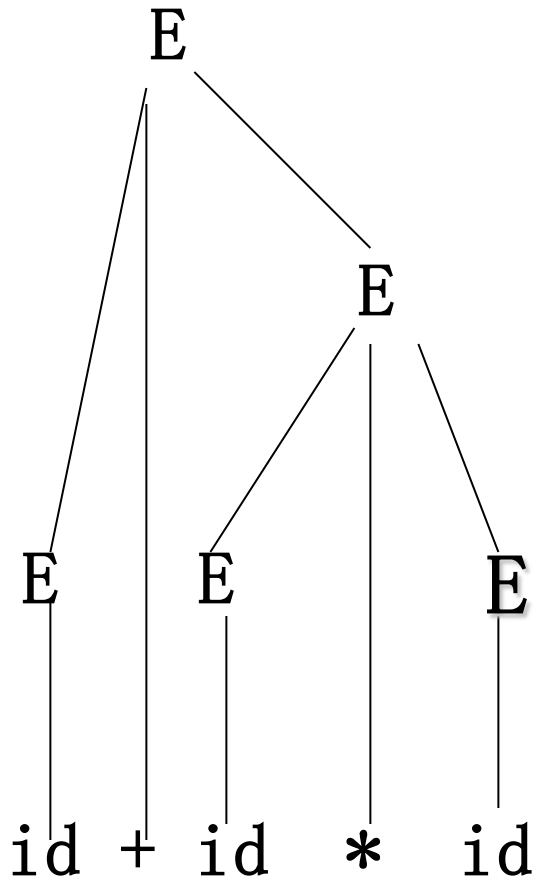
# 移进-归约分析中的问题

1) 移进归约冲突

- 例5.2中的 6)可以移进 * 或按产生式E→E+E归约

| | 动作 | 栈 | 缓冲区 |
|---|---|---|---|
| 1 | | # | id1+id2*id3# |
| 2 | 移进 | #id1 | +id2*id3# |
| 3 | 归约 E→id | #E | +id2*id3# |
| 4 | 移进 | #E+ | id2*id3# |
| 5 | 移进 | #E+id2 | *id3# |
| 6 | 归约 E→id | #E+E | *id3# |
| 7 | 移进 | #E+E* | id3# |
| 8 | 移进 | #E+E*id3 | # |
| 9 | 归约 E→id | #E+E*E | # |
| 10 | 归约 E→E*E | #E+E | # |
| 11 | 归约 E→E+E | #E | # |
| 12 | 接受 | | |

例5.2分析过程



# 移进-归约分析中的问题

1) 移进归约冲突

- 例5.2中的 6)可以移进 * 或按产生式E→E+E归约

2) 归约归约冲突

- 存在两个可用的产生式

- 各种分析方法处理冲突的方法不同
- 如何识别句柄?
  - 如何保证找到的直接短语是最左的? 利用栈
  - 如何确定句柄的开始处与结束处?

# 5.1.2 优先法

- 根据归约的先后次序为句型中相邻的文法符号规定优先关系
  - 句柄内相邻符号同时归约，是同优先的
  - 句柄两端符号的优先级要高于句柄外与之相邻的符号
- $a_1 \ldots a_{i-1} \lessdot a_i \equiv a_{i+1} \equiv \ldots \equiv a_{j-1} \equiv a_j \gtrdot a_{j+1} \ldots a_n$
- 定义了这种优先关系之后，语法分析程序就可以通过$a_{i-1} \lessdot a_i$和$a_j \gtrdot a_{j+1}$这两个关系来确定句柄的头和尾了

# 5.1.3 状态法

- 根据句柄的识别状态（句柄是逐步形成的）
  - 用状态来描述不同时刻下形成的那部分句柄
  - 因为句柄是产生式的右部，可用产生式来表示句柄的不同识别状态
- 例如： S→bBB可分解为如下识别状态
  - S→.bBB　移进b
  - S→bB.B　等待归约出B
  - S→b.BB　等待归约出B
  - S→bBB.　归约
- 采用这种方法，语法分析程序根据当前的分析状态就可以确定句柄的头和尾，并进行正确的归约。

# 5.2 算符优先分析法

- 算术表达式分析的启示
  - 先乘除后加减，同级从左到右
- 算符优先关系的直观意义
  - $+ \lessdot *$　　+ 的优先级低于 *
  - $( \equiv )$　　( 的优先级等于 )
  - $+ \gtrdot +$　　+ 的优先级高于 +
- 方法
  - 将句型中的终结符号当作“算符”，借助于算符之间的优先关系确定句柄
  - 归约即计算表达式的值。归约顺序不同，则计算的顺序也不同，结果也不一样
  - 如果规定算符的优先次序，并按这种规定进行归约，则归约过程是唯一的

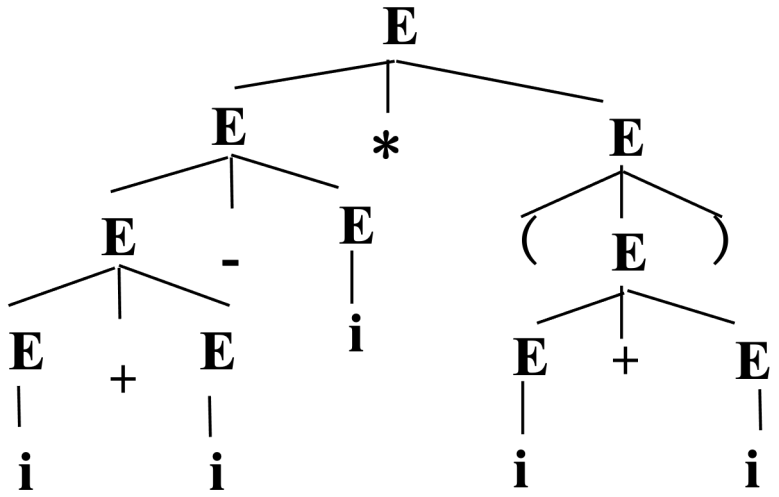
# 算术表达式文法的再分析

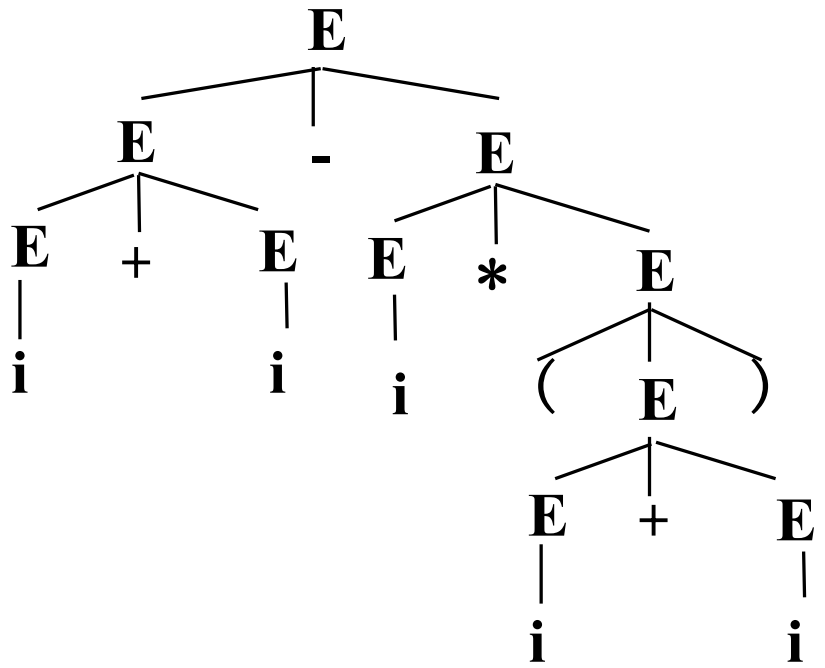
- 考虑二义文法文法G(E)：

$$G(E):\quad E \rightarrow i \mid E+E \mid E-E \mid E*E \mid E/E \mid (E)$$

- 它的句子可能有几种不同的规范归约。

句子i+i-i*(i+i)



# 句子i+i-i*(i+i)的归约过程

(1) i+i-i*(i+i)
(2) E+i-i*(i+i)
(3) E+E-i*(i+i)
(4) E-i*(i+i)
(5) E-E*(i+i)
(6) E-E*(E+i)
(7) E-E*(E+E)
(8) E-E*(E)
(9) E-E*E
(10) E-E
(11) E

**G'(E)：** $E \rightarrow T \mid E+T \mid E-T$
$T \rightarrow F \mid T*F \mid T/F$
$F \rightarrow (E) \mid i$

**G(E)：** $E \rightarrow i \mid E+E \mid E-E \mid E*E \mid E/E \mid (E)$

# 句子i+i-i*(i+i)的归约过程

(1) i+i-i*(i+i)
(2) F+i-i*(i+i)
(3) T+i-i*(i+i)
(4) E+i-i*(i+i)
(5) E+F-i*(i+i)
(6) E+T-i*(i+i)
(7) E-i*(i+i)
(8) E-F*(i+i)
(9) E-T*(i+i)
(10) E-T*(F+i)
(11) E-T*(T+i)
(12) E-T*(E+i)
(13) E-T*(E+F)
(14) E-T*(E+T)
(15) E-T*(E)
(16) E-T*F
(17) E-T
(18) E

- 为什么G’(E)可以保证归约的正确顺序、而G(E)却要额外的说明?

**G’(E)：E → T | E+T | E-T**
**T → F | T*F | T/F**
**F → (E) | i**

**G(E)： E → i| E+E|E-E|E*E|E/E|(E)**

# 算术表达式文法的再分析

- E→E+E
- E→E-E
- E→E*E
- E→E/E
- E→(E)
- E→id

**从如何去掉二义性,看对算符优先级的利用**

**句型的特征:**

**(E+E)*(E-E)/E/E+E*E*E**

- E→E+T| E-T| T

  T→T*F| T/F| F

  F→(E)|id

# 算符文法

- 如果文法G=（V，T，P，S）中不存在形如

$$A \rightarrow \alpha BC\beta$$

的产生式，则称之为算符文法(OG —Operator Grammar)

即：如果文法 G 中不存在具有相邻**非终结符**的产生式，则称为算符文法。

# 优先关系

- 定义任何两个可能相继出现的终结符a与b的三种优先关系
  - $a \lessdot b$　a的优先级低于b
  - $a \equiv b$　a的优先级等于b
  - $a \gtrdot b$　a的优先级高于b
- 注意:与数学上的< > =不同
  - + $\lessdot$ +
  - $a \lessdot b$ 并不意味着$b \gtrdot a$，如( $\lessdot$ +和 + $\lessdot$ (

# 5.2.1 算符优先文法

定义5.1 假设$G$是一个不含$\varepsilon$-产生式的文法，$A$、$B$和$C$均是$G$的语法变量，$G$的任何一对终结符$a$和$b$之间的优先关系定义为：

(1) $a \equiv b$，当且仅当文法$G$中含有形如$A \to \ldots ab \ldots$或$A \to \ldots aBb \ldots$的产生式；

(2) $a \lessdot b$，当且仅当文法$G$中含有形如$A \to \ldots aB \ldots$的产生式，而且$B \Rightarrow^{+} b \ldots$或$B \Rightarrow^{+} Cb \ldots$；

(3) $a \gtrdot b$，当且仅当文法$G$中含有形如$A \to \ldots Bb \ldots$的产生式，而且$B \Rightarrow^{+} \ldots a$或$B \Rightarrow^{+} \ldots aC$；

(4) $a$与$b$无关系，当且仅当$a$与$b$在$G$的任何句型中都不相邻。

# 5.2.1 算符优先文法

G(E)： E → i| E+E|E-E|E*E|E/E|(E)
不是一个算符优先文法

- 定义5.1 假设$G$是一个不含$\varepsilon$-产生式的文法，$A$、$B$和$C$均是$G$的语法变量，$G$的任何一对终结符$a$和$b$之间的优先关系定义为：

(1) $a \doteq b$，当且仅当文法$G$中含有形如$A \to \ldots ab \ldots$或$A \to \ldots aBb \ldots$的产生式；

(2) $a \lessdot b$，当且仅当文法$G$中含有形如$A \to \ldots aB \ldots$的产生式，而且$B \Rightarrow^{+} b \ldots$或$B \Rightarrow^{+} Cb \ldots$；

(3) $a \gtrdot b$，当且仅当文法$G$中含有形如$A \to \ldots Bb \ldots$的产生式，而且$B \Rightarrow^{+} \ldots a$或$B \Rightarrow^{+} \ldots aC$；

- 如果一个算符文法G中的任何终结符对(a，b)至多只满足下述三关系之一：

a≡b，a⋖b， a⋗b

则称G是一个算符优先文法。

G(E)： E → T | E+T | E-T
T → F | T*F | T/F
F → (E) | i
是一个算符优先文法

# 例子

考虑下面的文法G(E)：

(1) E→E+T | T
(2) T→T*F | F
(3) F→P↑F | P
(4) P→(E) | i

- 由规则P→(E)，有 ( ≡ )
- 由规则E→E＋T和T⇒T*F， 有 ＋ ⋖ *
- 由T→T*F 和F→P↑F，可得 * ⋖ ↑
- 由E→E＋T和E ⇒ E+T，可得 +(left) ⋗ +(right)
- 由F→P↑F和F ⇒ P↑F，可得 ↑(left) ⋖ ↑(right)
- 由P→(E)和 E⇒E+T⇒T+T⇒T*F+T⇒F*F+T
  ⇒P↑F*F+T⇒i↑F*F+T
  有 ( ⋖ + ( ⋖ * ( ⋖ ↑ ( ⋖ i

# 5.2.2 算符优先矩阵的构造

## 优先关系的确定

- 根据优先关系的定义
  - $a \lessdot b \Leftrightarrow A \rightarrow \ldots aB \ldots \in P$且$(B \Rightarrow^{+} b \ldots$或者$B \Rightarrow^{+} Cb \ldots)$
  - 需要求出非终结符B派生出的第一个终结符集
  - $a \gtrdot b \Leftrightarrow A \rightarrow \ldots Bb \ldots \in P$且$(B \Rightarrow^{+} \ldots a$或者$B \Rightarrow^{+} \ldots aC)$
  - 需要求出非终结符B派生出的最后一个终结符集

## 设G=（V，T，P，S）为OG，则定义

- $FIRSTOP(A)=\{b|A \Rightarrow^{+} b \ldots$或者$A \Rightarrow^{+} Bb \ldots, b \in T, B \in V\}$
- $LASTOP(A)=\{b|A \Rightarrow^{+} \ldots b$或者$A \Rightarrow^{+} \ldots bB, b \in T, B \in V\}$

# 算符优先关系矩阵的构造

- A→…ab… ; A→…aBb…, 则a≡b
- A→…aB…，则对∀b∈FIRSTOP(B)，a≺b
- A→…Bb…，则对∀a∈LASTOP(B)，a≻b
- if A→B…∈P，then FIRSTOP(B)⊆ FIRSTOP(A)
- if A→…B∈P，then LASTOP(B)⊆ LASTOP(A)
- 问题：编程求FIRSTOP、LASTOP

# 例题

1. 若有产生式P→... a或P→ ... aQ，则a∈ LASTOP(P)；

2. 若a∈ LASTOP(Q)，且有产生式P→... Q ，则a∈ LASTOP(P)。

考虑下面的文法G(E)：

(1) E→E+T | T

(2) T→T*F | F

(3) F→P↑F | P

(4) P→(E) | I

计算文法G的FIRSTOP和LASTOP

| | + | * | ↑ | ( | ) | i |
|---|---|---|---|---|---|---|
| E | ∽ | ∽ | ∽ | ∽ | | ∽ |
| T | | ∽ | ∽ | ∽ | | ∽ |
| F | | | ∽ | ∽ | | ∽ |
| P | | | | ∽ | | ∽ |

| | + | * | ↑ | ( | ) | i |
|---|---|---|---|---|---|---|
| E | ∽ | ∽ | ∽ | | ∽ | ∽ |
| T | | ∽ | ∽ | | ∽ | ∽ |
| F | | | ∽ | | ∽ | ∽ |
| P | | | | | ∽ | ∽ |

$$FIRSTOP(T)=\{*,\uparrow,(,i\}$$

$$FIRSTOP(F)=\{\uparrow,(,i\}$$

$$FIRSTOP(E)=\{+,*,\uparrow,(,i\}$$

$$FIRSTOP(P)=\{(,i\}$$

$$LASTOP(T)=\{*,\uparrow,),i\}$$

$$LASTOP(F)=\{\uparrow,),i\}$$

$$LASTOP(E)=\{+,*,\uparrow,),i\}$$

$$LASTOP(P)=\{),i\}$$

# 算符优先关系矩阵的构造

- $A \rightarrow X_1X_2 \ldots X_n$

① 如果$X_iX_{i+1} \in TT$则：$X_i \equiv X_{i+1}$

② 如果$X_iX_{i+1}X_{i+2} \in TVT$则：$X_i \equiv X_{i+2}$

③ 如果$X_iX_{i+1} \in TV$则：$\forall a \in FIRSTOP(X_{i+1})$，$X_i \lessdot a$

④ 如果$X_iX_{i+1} \in VT$则：$\forall a \in LASTOP(X_i)$，$a \gtrdot X_{i+1}$

# 例题

考虑下面的文法G(E)：

(1) E→E+T | T

(2) T→T*F | F

(3) F→P↑F | P

(4) P→(E) | I

计算文法G的FIRSTOP和LASTOP

$$FIRSTOP(T)=\{*,\uparrow,(,i\}$$
$$FIRSTOP(F)=\{\uparrow,(,i\}$$
$$FIRSTOP(E)=\{+,*,\uparrow,(,i\}$$
$$FIRSTOP(P)=\{(,i\}$$

$$LASTOP(T)=\{*,\uparrow,),i\}$$
$$LASTOP(F)=\{\uparrow,),i\}$$
$$LASTOP(E)=\{+,*,\uparrow,),i\}$$
$$LASTOP(P)=\{),i\}$$

| | + | * | ↑ | i | ( | ) |
|---|---|---|---|---|---|---|
| + | ⋗ | ⋖ | ⋖ | ⋖ | ⋖ | ⋗ |
| * | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| ↑ | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| i | ⋗ | ⋗ | ⋗ | | | ⋗ |
| ( | ⋖ | ⋖ | ⋖ | ⋖ | ⋖ | ≡ |
| ) | ⋗ | ⋗ | ⋗ | | | ⋗ |

# 例题

考虑下面的文法G(E)：

(1) E→E+T | T

(2) T→T*F | F

(3) F→P↑F | P

(4) P→(E) | I

计算文法G的FIRSTOP和LASTOP

$$FIRSTOP(T)=\{*,\uparrow,(,i\}$$
$$FIRSTOP(F)=\{\uparrow,(,i\}$$
$$FIRSTOP(E)=\{+,*,\uparrow,(,i\}$$
$$FIRSTOP(P)=\{(,i\}$$

$$LASTOP(T)=\{*,\uparrow,),i\}$$
$$LASTOP(F)=\{\uparrow,),i\}$$
$$LASTOP(E)=\{+,*,\uparrow,),i\}$$
$$LASTOP(P)=\{),i\}$$

| | + | * | ↑ | i | ( | ) |
|---|---|---|---|---|---|---|
| + | ⋗ | ⋖ | ⋖ | ⋖ | ⋖ | ⋗ |
| * | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| ↑ | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| i | ⋗ | ⋗ | ⋗ | | | ⋗ |
| ( | ⋖ | ⋖ | ⋖ | ⋖ | ⋖ | ≡ |
| ) | ⋗ | ⋗ | ⋗ | | | ⋗ |

# 例题

考虑下面的文法G(E)：

(1) E→E+T | T

(2) T→T*F | F

(3) F→P↑F | P

(4) P→(E) | I

计算文法G的FIRSTOP和LASTOP

$$FIRSTOP(T)=\{*,\uparrow,(,i\}$$

$$FIRSTOP(F)=\{\uparrow,(,i\}$$

$$FIRSTOP(E)=\{+,*,\uparrow,(,i\}$$

$$FIRSTOP(P)=\{(,i\}$$

$$LASTOP(T)=\{*,\uparrow,),i\}$$

$$LASTOP(F)=\{\uparrow,),i\}$$

$$LASTOP(E)=\{+,*,\uparrow,),i\}$$

$$LASTOP(P)=\{),i\}$$

| | + | * | ↑ | i | ( | ) |
|---|---|---|---|---|---|---|
| + | ⋗ | ⋖ | ⋖ | ⋖ | ⋖ | ⋗ |
| * | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| ↑ | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| i | ⋗ | ⋗ | ⋗ | | | ⋗ |
| ( | ⋖ | ⋖ | ⋖ | ⋖ | ⋖ | ≡ |
| ) | ⋗ | ⋗ | ⋗ | | | ⋗ |

# 例题

考虑下面的文法G(E)：

(1) E→E+T | T

(2) T→T*F | F

(3) F→P↑F | P

(4) P→(E) | I

计算文法G的FIRSTOP和LASTOP

$$FIRSTOP(T)=\{*,\uparrow,(,i\}$$

$$FIRSTOP(F)=\{\uparrow,(,i\}$$

$$FIRSTOP(E)=\{+,*,\uparrow,(,i\}$$

$$FIRSTOP(P)=\{(,i\}$$

$$LASTOP(T)=\{*,\uparrow,),i\}$$

$$LASTOP(F)=\{\uparrow,),i\}$$

$$LASTOP(E)=\{+,*,\uparrow,),i\}$$

$$LASTOP(P)=\{),i\}$$

| | + | * | ↑ | i | ( | ) |
|---|---|---|---|---|---|---|
| + | ⋗ | ⋖ | ⋖ | ⋖ | ⋖ | ⋗ |
| * | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| ↑ | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| i | ⋗ | ⋗ | ⋗ | | | ⋗ |
| ( | ⋖ | ⋖ | ⋖ | ⋖ | ⋖ | ≡ |
| ) | ⋗ | ⋗ | ⋗ | | | ⋗ |

# 例题

考虑下面的文法G(E)：

(1) E→E+T | T

(2) T→T*F | F

(3) F→P↑F | P

(4) P→(E) | I

计算文法G的FIRSTOP和LASTOP

$$FIRSTOP(T)=\{*,\uparrow,(,i\}$$

$$FIRSTOP(F)=\{\uparrow,(,i\}$$

$$FIRSTOP(E)=\{+,*,\uparrow,(,i\}$$

$$FIRSTOP(P)=\{(,i\}$$

$$LASTOP(T)=\{*,\uparrow,),i\}$$

$$LASTOP(F)=\{\uparrow,),i\}$$

$$LASTOP(E)=\{+,*,\uparrow,),i\}$$

$$LASTOP(P)=\{),i\}$$

| | + | * | ↑ | i | ( | ) |
|---|---|---|---|---|---|---|
| + | ⋗ | ⋖ | ⋖ | ⋖ | ⋖ | ⋗ |
| * | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| ↑ | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ |
| i | ⋗ | ⋗ | ⋗ | | | ⋗ |
| ( | ⋖ | ⋖ | ⋖ | ⋖ | ⋖ | ≡ |
| ) | ⋗ | ⋗ | ⋗ | | | ⋗ |

# 例 5.6 表达式文法的算符优先关系

**E → T | E+T | E-T　　T → F | T*F | T/F　　F → (E) | id**

| | + | − | * | / | ( | ) | id | # |
|---|---|---|---|---|---|---|---|---|
| + | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ | ⋖ | ⋗ |
| − | ⋗ | ⋗ | ⋖ | ⋖ | ⋖ | ⋗ | ⋖ | ⋗ |
| * | ⋗ | ⋗ | ⋗ | ⋗ | ⋖ | ⋗ | ⋖ | ⋗ |
| / | ⋗ | ⋗ | ⋗ | ⋗ | ⋖ | ⋗ | ⋖ | ⋗ |
| ( | ⋖ | ⋖ | ⋖ | ⋖ | ⋖ | ≡ | ⋖ | |
| ) | ⋗ | ⋗ | ⋗ | ⋗ | | ⋗ | | ⋗ |
| id | ⋗ | ⋗ | ⋗ | ⋗ | | ⋗ | | ⋗ |
| # | ⋖ | ⋖ | ⋖ | ⋖ | ⋖ | | ⋖ | acc |

# 例5.7

- E→E+T|E-T|T　T→T*F|T/F|F　F→(E)|id，试利用算符优先分析法对id+id进行分析

| 步骤 | 栈 | 输入串 | 优先关系 | 动作 |
|---|---|---|---|---|
| 1 | # | $id_1+id_2$# | | |
| 2 | # $id_1$ | $+id_2$# | #$\lessdot id_1$ | 移进$id_1$ |
| 3 | #F | $+id_2$# | #$\lessdot id_1 \gtrdot$+ | 用$F\rightarrow$id归约 |
| 4 | #F+ | $id_2$# | #$\lessdot$+ | 移进+ |
| 5 | #F+ $id_2$ | # | +$\lessdot id_2$ | 移进$id_2$ |
| 6 | #F+F | # | +$\lessdot id_2 \gtrdot$# | 用$F\rightarrow$id归约 |
| 7 | #E | # | #$\lessdot$+$\gtrdot$# | 用$E\rightarrow E+T$归约 |

# 问题

- 有时未归约真正的句柄
- 不是严格的最左归约
- 归约的符号串有时与产生式右部不同

E→E+T|E-T|T　T→T*F|T/F|F　F→(E)|id，
试利用算符优先分析法对id+id进行分析

| 步骤 | 栈 | 输入串 | 优先关系 | 动作 |
|---|---|---|---|---|
| 1 | # | $id_1+id_2\#$ | | |
| 2 | # $id_1$ | $+id_2\#$ | $\#\lessdot id_1$ | 移进$id_1$ |
| 3 | #F | $+id_2\#$ | $\#\lessdot id_1\gtrdot +$ | 用$F\rightarrow id$归约 |
| 4 | #F+ | $id_2\#$ | $\#\lessdot +$ | 移进+ |
| 5 | #F+ $id_2$ | # | $+\lessdot id_2$ | 移进$id_2$ |
| 6 | #F+F | # | $+\lessdot id_2\gtrdot \#$ | 用$F\rightarrow id$归约 |
| 7 | #E | # | $\#\lessdot +\gtrdot \#$ | 用$E\rightarrow E+T$归约 |

# 问题

- 有时未归约真正的句柄
- 不是严格的最左归约
- 归约的符号串有时与产生式右部不同

E→E+T|E-T|T　T→T*F|T/F|F　F→(E)|id，
试利用算符优先分析法对id+id进行分析

| 步骤 | 栈 | 输入串 | 优先关系 | 动作 |
|---|---|---|---|---|
| 1 | # | $id_1+id_2\#$ | | |
| 2 | # $id_1$ | $+id_2\#$ | $\#\lessdot id_1$ | 移进$id_1$ |
| 3 | #F | $+id_2\#$ | $\#\lessdot id_1\gtrdot +$ | 用$F\rightarrow id$归约 |
| 4 | #F+ | $id_2\#$ | $\#\lessdot +$ | 移进+ |
| 5 | #F+ $id_2$ | # | $+\lessdot id_2$ | 移进$id_2$ |
| 6 | #F+F | # | $+\lessdot id_2\gtrdot \#$ | 用$F\rightarrow id$归约 |
| 7 | #E | # | $\#\lessdot +\gtrdot \#$ | 用$E\rightarrow E+T$归约 |

# 问题

- 有时未归约真正的句柄
- 不是严格的最左归约
- 归约的符号串有时与产生式右部不同

E→E+T|E-T|T　T→T*F|T/F|F　F→(E)|id，
试利用算符优先分析法对id+id进行分析

| 步骤 | 栈 | 输入串 | 优先关系 | 动作 |
|---|---|---|---|---|
| 1 | # | $id_1+id_2\#$ | | |
| 2 | # $id_1$ | $+id_2\#$ | $\#\lessdot id_1$ | 移进$id_1$ |
| 3 | #F | $+id_2\#$ | $\#\lessdot id_1\gtrdot +$ | 用$F\rightarrow id$归约 |
| 4 | #F+ | $id_2\#$ | $\#\lessdot +$ | 移进+ |
| 5 | #F+ $id_2$ | # | $+\lessdot id_2$ | 移进$id_2$ |
| 6 | #F+F | # | $+\lessdot id_2\gtrdot \#$ | 用$F\rightarrow id$归约 |
| 7 | #E | # | $\#\lessdot +\gtrdot \#$ | 用$E\rightarrow E+T$归约 |

# 问题

- 有时未归约真正的句柄（F）
- 不是严格的最左归约
- 归约的符号串有时与产生式右部不同
- 仍能正确识别句子的原因
  - OPG未定义非终结符之间的优先关系，不能识别由单非终结符组成的句柄
  - 定义算符优先分析过程识别的“句柄”为最左素短语LPP（Leftmost Prime Phase）

# 素短语与最左素短语

- 什么是短语？当前我们要找什么样的短语？——至少有一个算符

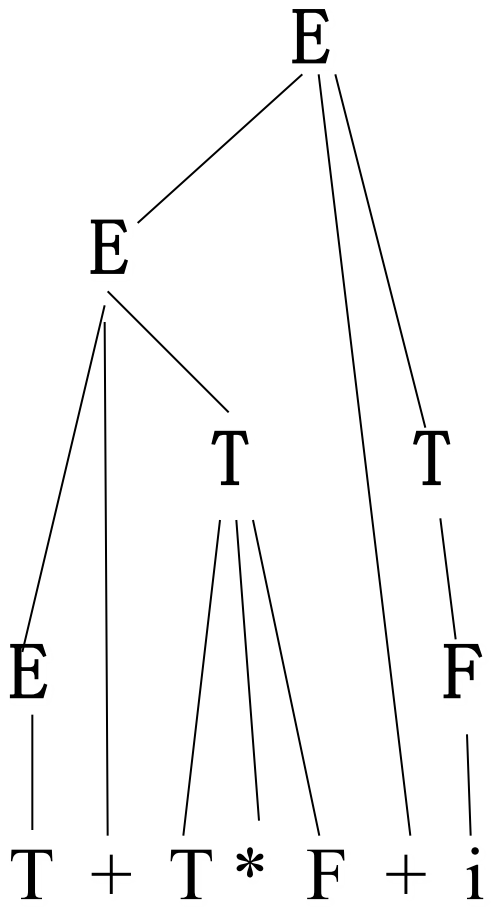
- $S \Rightarrow^{*} \alpha A \beta$ and $A \Rightarrow^{+} \gamma$，$\gamma$至少含一个终结符，且不含更小的含终结符的短语，则称$\gamma$是句型$\alpha\gamma\beta$的相对于变量A的素短语(Prime Phrase)

- 句型的至少含一个终结符且不含其它素短语的短语

# 例

- E→E+T|T  T→T*F|F  F→(E)|id

句型 T+T*F+i 的短语有:



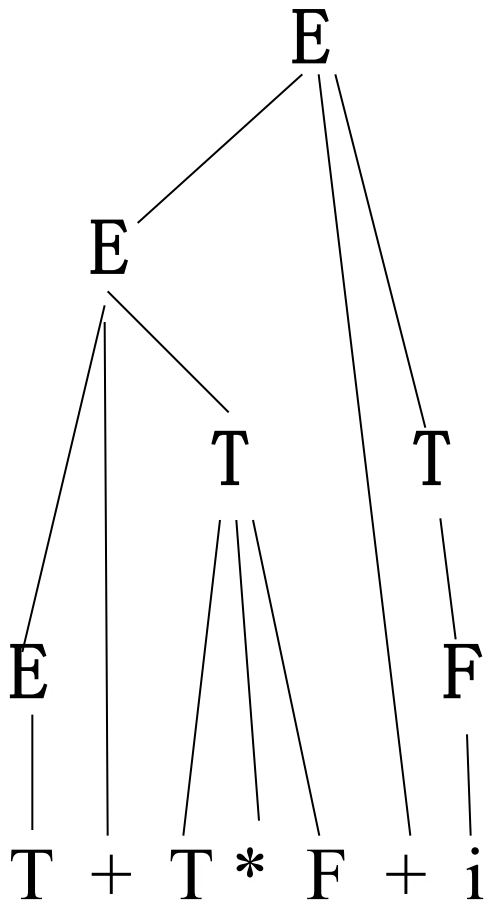
# 例

- E→E+T|T  T→T*F|F  F→(E)|id

句型 T+T*F+i 的短语有:

T  T*F  i  T+T*F  T+T*F+i

其中的素短语为:



# 例

- E→E+T|T  T→T*F|F  F→(E)|id

句型 T+T*F+i 的短语有:

T　T*F　i　T+T*F　T+T*F+i

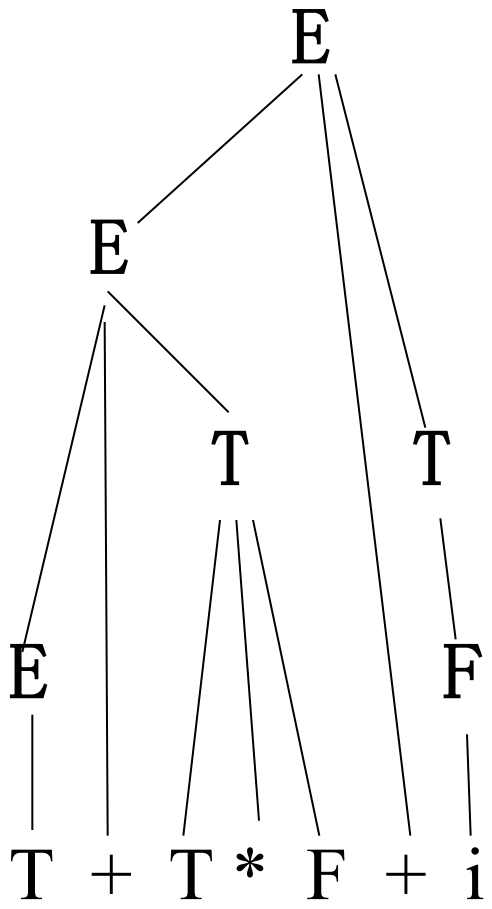
其中的素短语为:

T*F　i

T*F为最左素短语，是被归约的对象

问题：按照文法E→E+E|E*E|(E)|id，

求i+E*i+i的短语和素短语

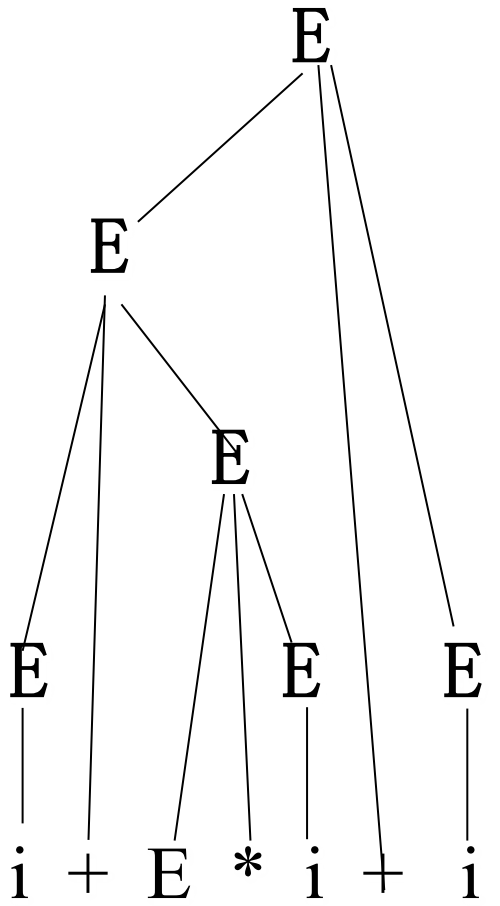


# 例

文法：E→E+E|E*E

E→(E)|id

句型i+E*i+i的短语



# 例

文法：E→E+E|E*E

E→(E)|id

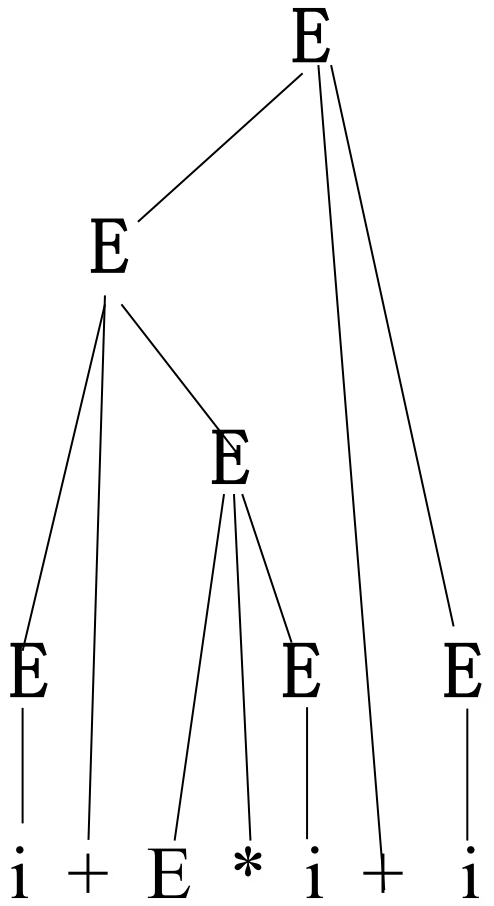
句型i+E*i+i的短语

i　i　E*i　i

i+E*i　i+E*i+i

其中的素短语为

问题：归约过程中如何发现“中间句型”　的最左素短语?



# 例

文法：E→E+E|E*E

E→(E)|id

句型i+E*i+i的短语

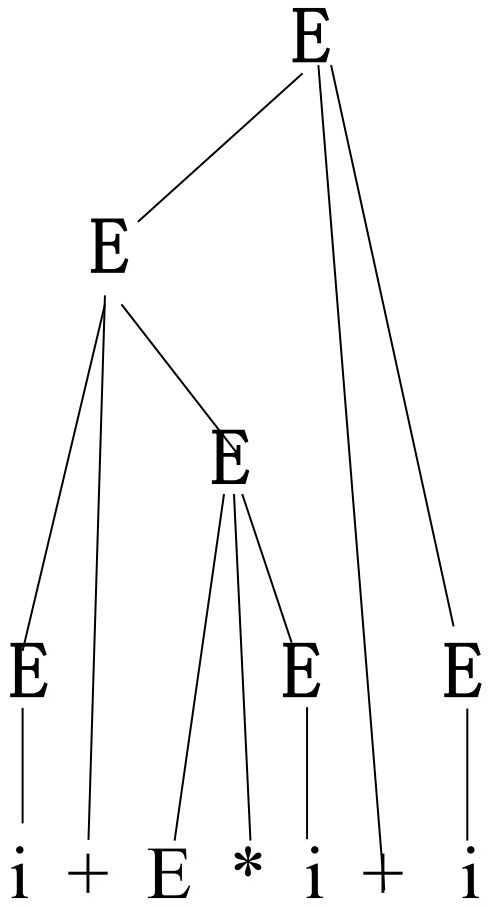
i　i　E*i　i

i+E*i　i+E*i+i

其中的素短语为

i　i　i

问题：归约过程中如何发现“中间句型”　的最左素短语?



# 素短语与最左素短语

设句型的一般形式为

$\#N_1a_1\,N_2a_2\ldots N_na_n\,\#\ (N_i\in V\cup\{\varepsilon\},a_i\in V_T)$

它的最左素短语是满足下列条件的最左子串

$$N_ia_i\,N_{i+1}a_{i+1}\ldots N_ja_j\,N_{j+1}$$

其中：　$a_{i-1}\lessdot a_i$,,

$a_i\equiv a_{i+1}\equiv\ldots\equiv a_{j-1}\equiv a_j$ ，

$a_j\gtrdot a_{j+1}$

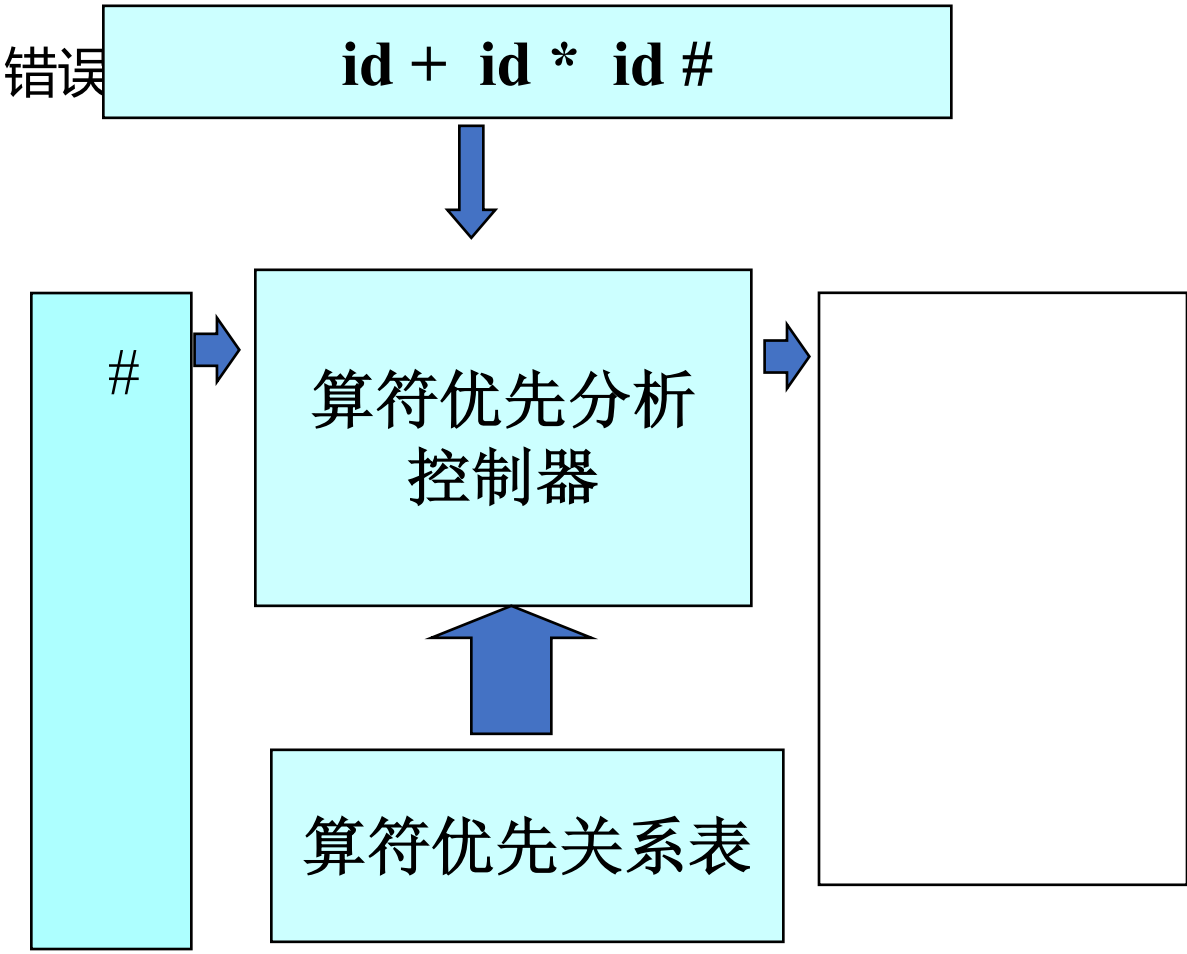
# 算符优先分析的实现

- 系统组成
  - 移进归约分析器 + 优先关系表
- 分析算法
  - 参照输入串、优先关系表，完成一系列归约，生成语法分析树——输出产生式

# 算法5.3 算符优先分析算法（id+id*id 的分析过程）

输入：文法$G=(V, T, P, S)$，输入字符串$w$和优先关系表;
输出：如果$w$是一个句子则输出一个分析树架子，否则指出错误
步骤：

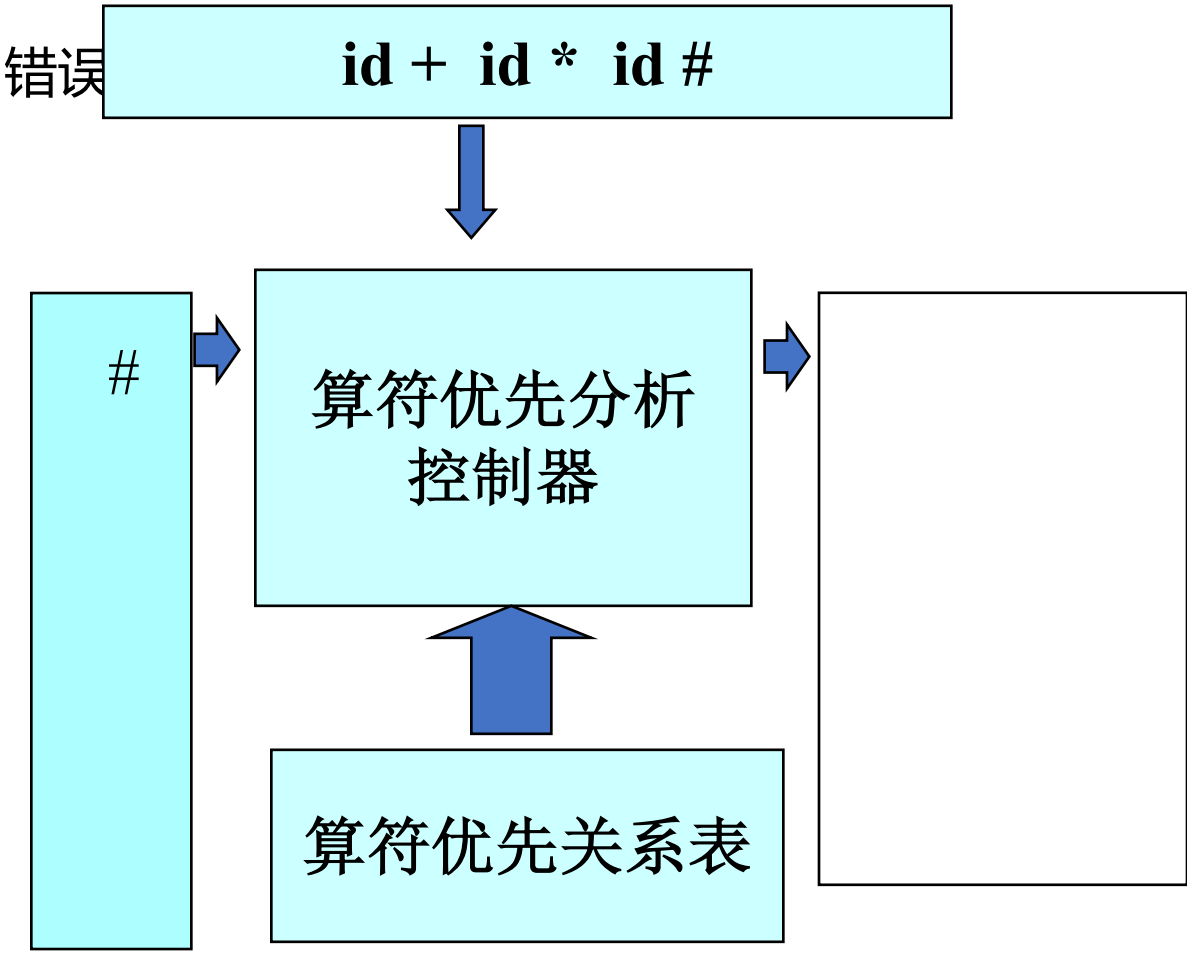
```
begin   S[1]:='#';   i:=1;
        repeat   将下一输入符号读入R;
          if S[i]∈T then j:=i else j:=i-1; (指向栈顶的终结符)
          while S[j]≻R do  begin
                repeat
                      Q:=S[j];
                      if S[j-1]∈T then j:=j-1 else j:=j-2
                until S[j]≺Q;
                将S[j+1]…S[i]归约为N; i:=j+1;S[i]:=N
          end;
          if S[j]≺R or S[j]≡R then  begin i:=i+1; S[i]:=R end
          else error
until i=2 and R='#'  end;
```



# 算法5.3 算符优先分析算法（id+id*id 的分析过程）

输入：文法$G=(V, T, P, S)$，输入字符串$w$和优先关系表;
输出：如果$w$是一个句子则输出一个分析树架子，否则指出错误
步骤：

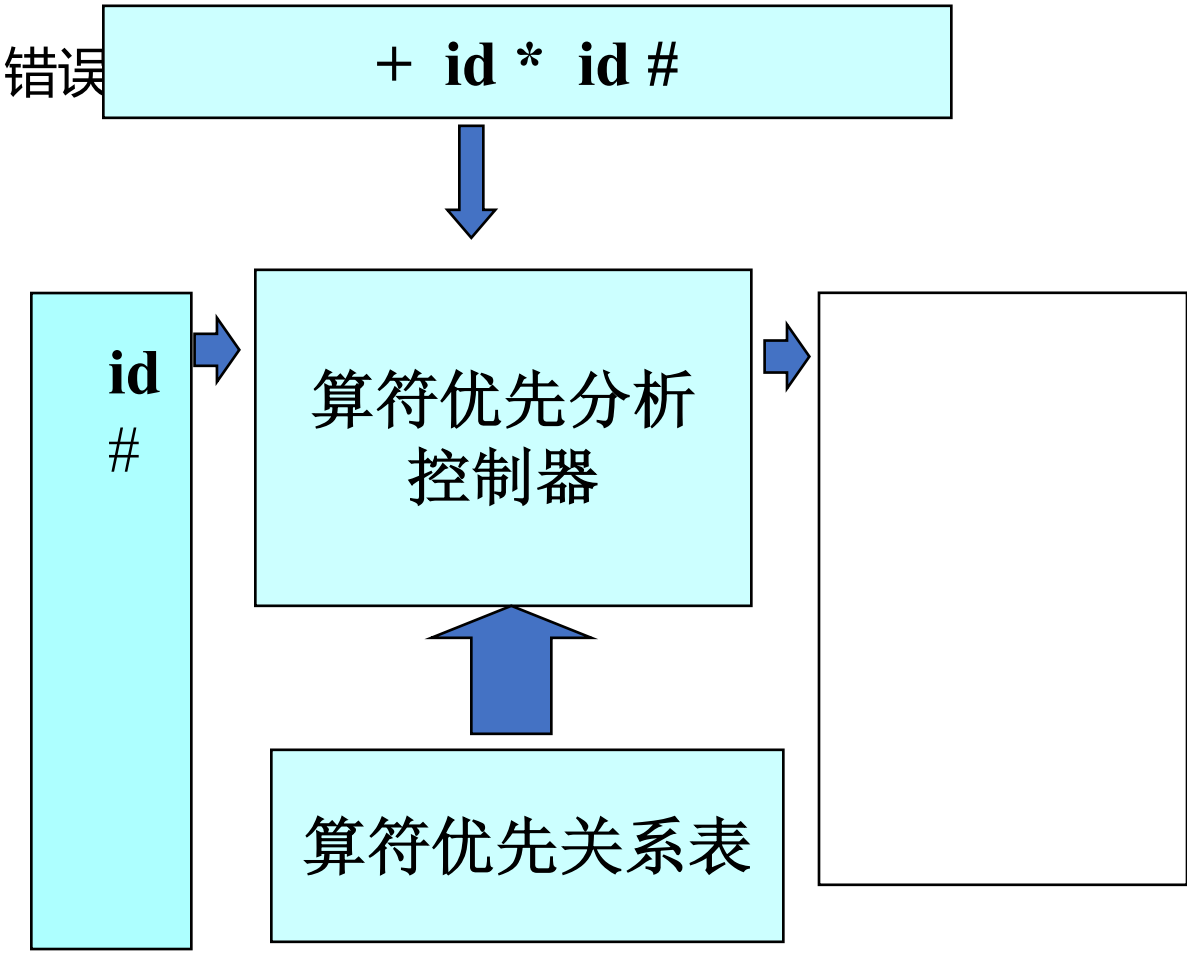
```
begin    S[1]:='#';   i:=1;
         repeat    将下一输入符号读入R;
            if S[i]∈T then j:=i else j:=i-1; (指向栈顶的终结符)
            while S[j]≻R do  begin
                  repeat
                        Q:=S[j];
                        if S[j-1]∈T then j:=j-1 else j:=j-2
                  until S[j]≺Q;
                  将S[j+1]…S[i]归约为N; i:=j+1;S[i]:=N
            end;
            if S[j]≺R or S[j]≡R then  begin i:=i+1; S[i]:=R end
            else error
until i=2 and R='#'  end;
```



# 算法5.3 算符优先分析算法（id+id*id 的分析过程）

输入：文法$G=(V, T, P, S)$，输入字符串$w$和优先关系表;
输出：如果$w$是一个句子则输出一个分析树架子，否则指出错误
步骤：

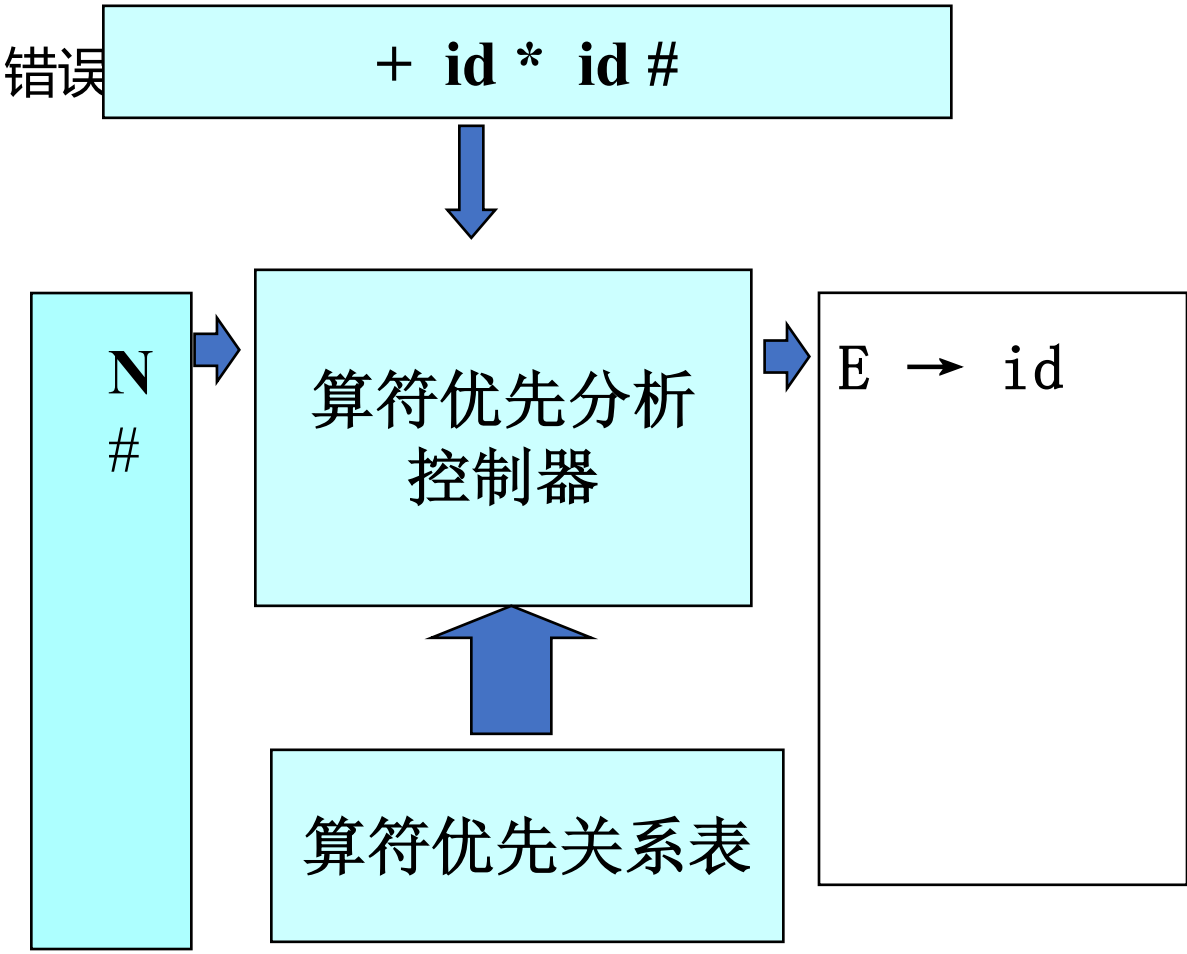
```
begin   S[1]:='#';   i:=1;
        repeat   将下一输入符号读入R;
          if S[i]∈T then j:=i else j:=i-1; (指向栈顶的终结符)
          while S[j]≻R do  begin
                repeat
                      Q:=S[j];
                      if S[j-1]∈T then j:=j-1 else j:=j-2
                until S[j]≺Q;
                将S[j+1]…S[i]归约为N; i:=j+1;S[i]:=N
          end;
          if S[j]≺R or S[j]≡R then  begin i:=i+1; S[i]:=R end
          else error
until i=2 and R='#'  end;
```



# 算法5.3 算符优先分析算法（id+id*id 的分析过程）

输入：文法$G=(V, T, P, S)$，输入字符串$w$和优先关系表;
输出：如果$w$是一个句子则输出一个分析树架子，否则指出错误
步骤：

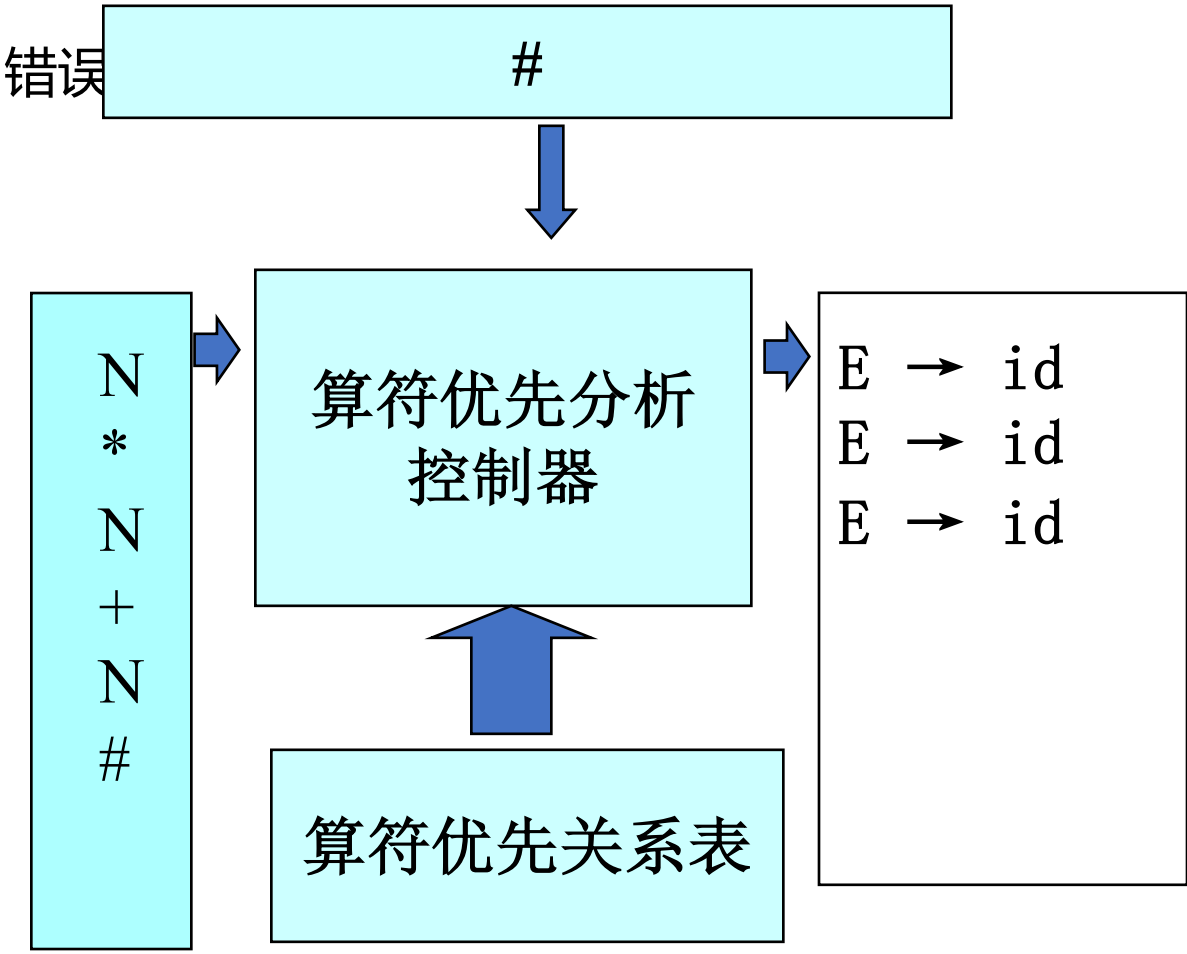
```
begin    S[1]:='#';   i:=1;
         repeat    将下一输入符号读入R;
            if S[i]∈T then j:=i else j:=i-1; (指向栈顶的终结符)
            while S[j]⋗R do  begin
                  repeat
                        Q:=S[j];
                        if S[j-1]∈T then j:=j-1 else j:=j-2
                  until S[j]⋖Q;
                  将S[j+1]…S[i]归约为N; i:=j+1;S[i]:=N
            end;
            if S[j]⋖R or S[j]≡R then  begin i:=i+1; S[i]:=R end
            else error
until i=2 and R='#'  end;
```



# 算法5.3 算符优先分析算法（id+id*id 的分析过程）

输入：文法$G=(V, T, P, S)$，输入字符串$w$和优先关系表;
输出：如果$w$是一个句子则输出一个分析树架子，否则指出错误
步骤：

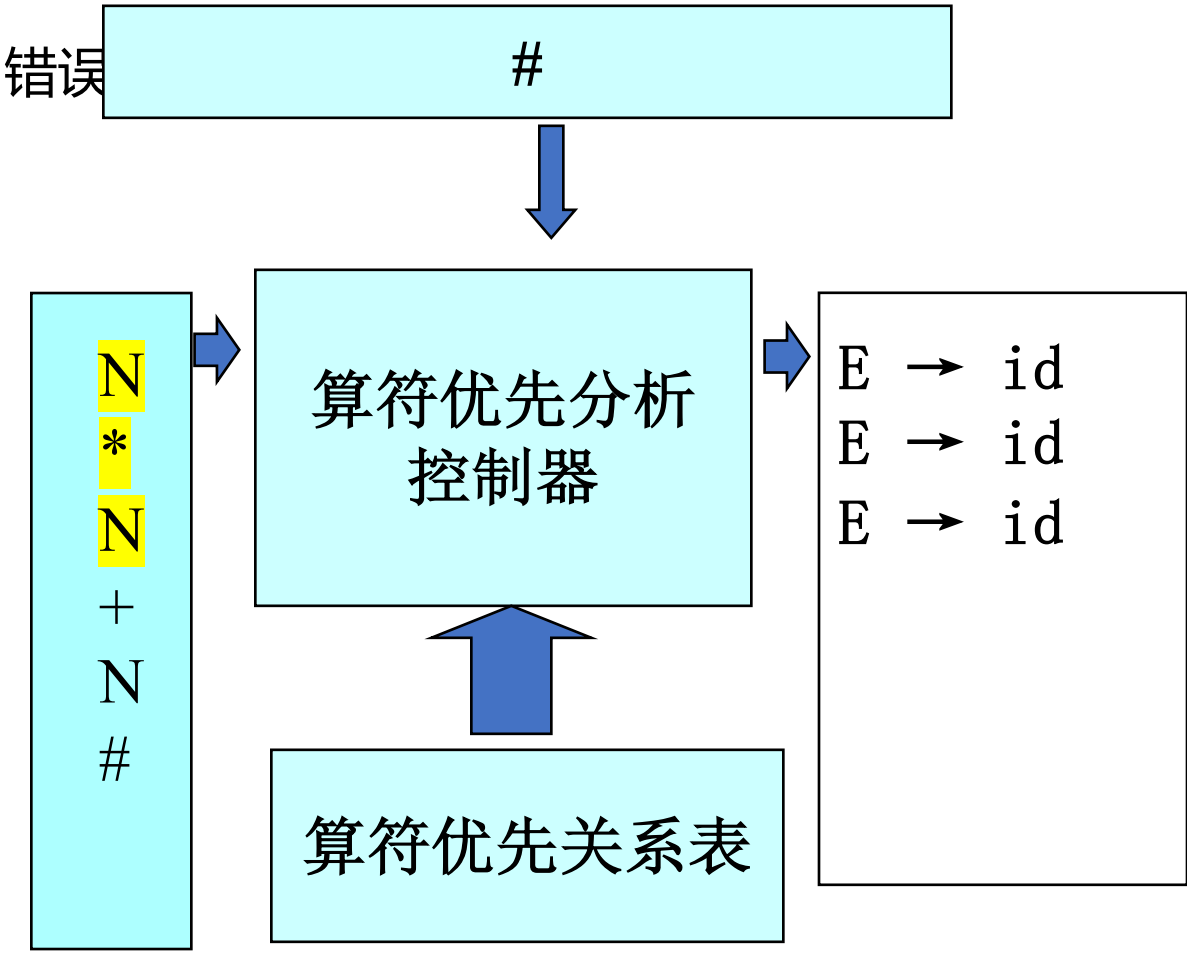
```
begin    S[1]:='#';   i:=1;
         repeat    将下一输入符号读入R;
            if S[i]∈T then j:=i else j:=i-1; (指向栈顶的终结符)
            while S[j]⋗R do  begin
                  repeat
                        Q:=S[j];
                        if S[j-1]∈T then j:=j-1 else j:=j-2
                  until S[j]⋖Q;
                  将S[j+1]…S[i]归约为N; i:=j+1;S[i]:=N
            end;
            if S[j]⋖R or S[j]≡R then  begin i:=i+1; S[i]:=R end
            else error
until i=2 and R='#'  end;
```



# 算法5.3 算符优先分析算法（id+id*id 的分析过程）

输入：文法$G=(V, T, P, S)$，输入字符串$w$和优先关系表;
输出：如果$w$是一个句子则输出一个分析树架子，否则指出错误
步骤：

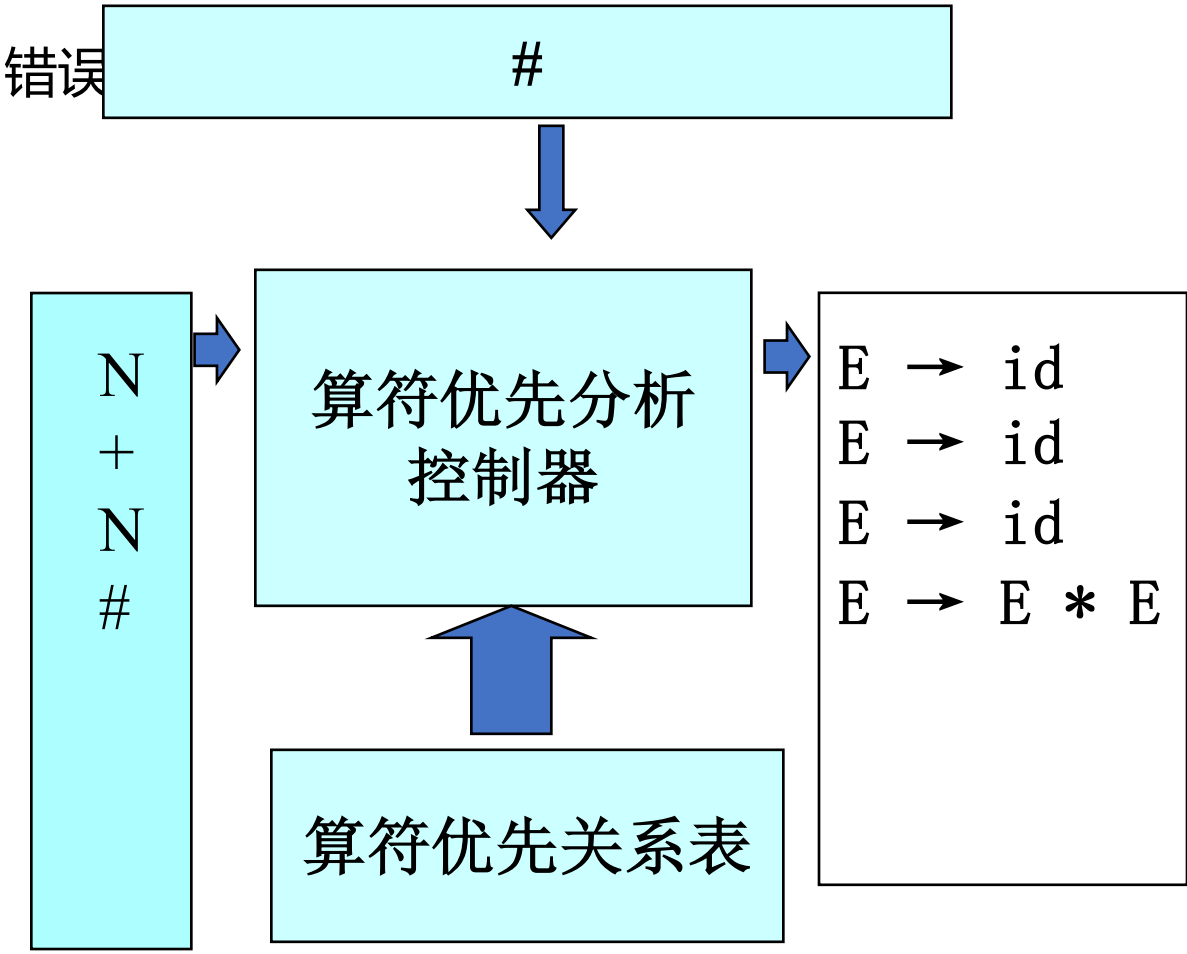
```
begin    S[1]:='#';   i:=1;
         repeat    将下一输入符号读入R;
            if S[i]∈T then j:=i else j:=i-1; (指向栈顶的终结符)
            while S[j]≻R do  begin
                  repeat
                        Q:=S[j];
                        if S[j-1]∈T then j:=j-1 else j:=j-2
                  until S[j]≺Q;
                  将S[j+1]…S[i]归约为N; i:=j+1;S[i]:=N
            end;
            if S[j]≺R or S[j]≡R then  begin i:=i+1; S[i]:=R end
            else error
until i=2 and R='#'  end;
```



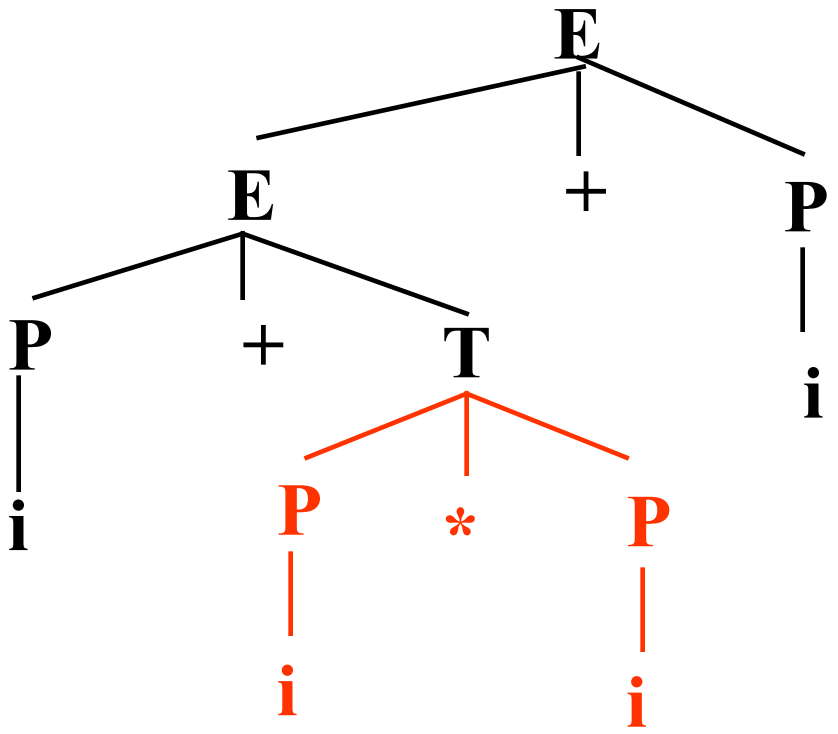
# 算法5.3 算符优先分析算法（id+id*id 的分析过程）

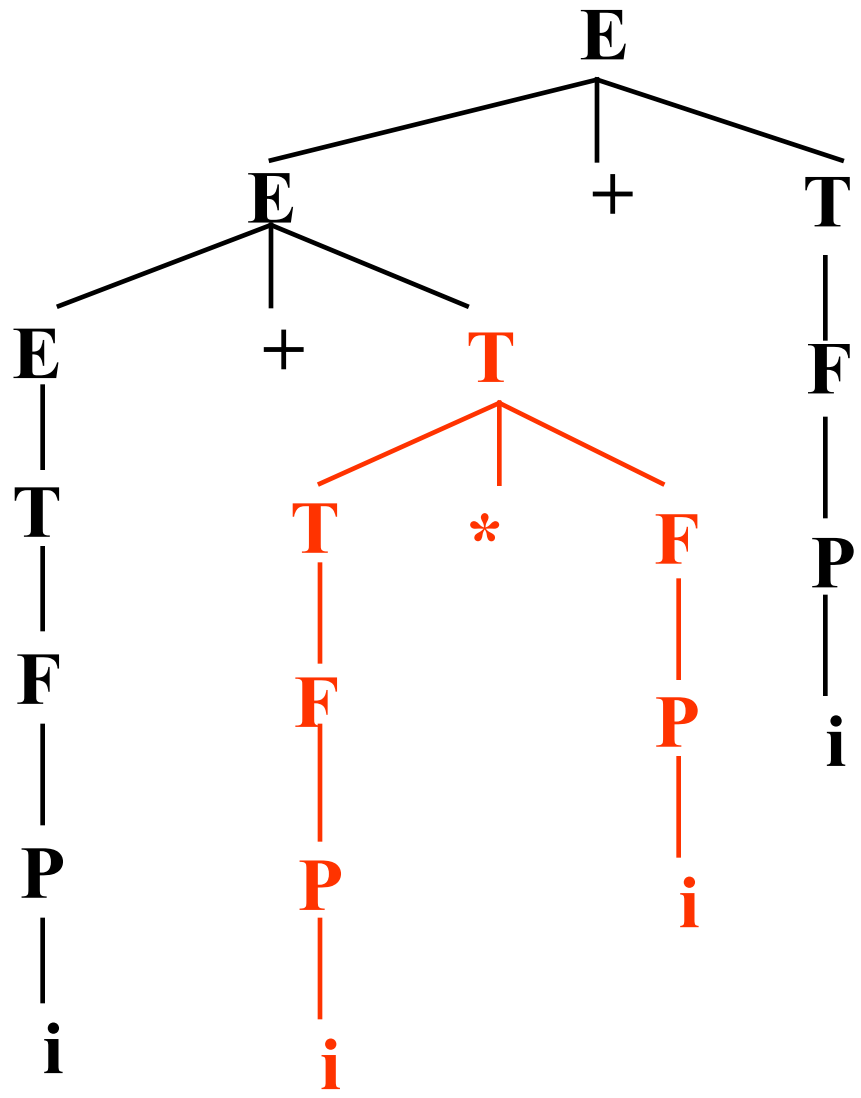
输入：文法$G=(V, T, P, S)$，输入字符串$w$和优先关系表;
输出：如果$w$是一个句子则输出一个分析树架子，否则指出错误
步骤：

```
begin    S[1]:='#';   i:=1;
         repeat    将下一输入符号读入R;
            if S[i]∈T then j:=i else j:=i-1; (指向栈顶的终结符)
            while S[j]⋗R do  begin
                  repeat
                        Q:=S[j];
                        if S[j-1]∈T then j:=j-1 else j:=j-2
                  until S[j]⋖Q;
                  将S[j+1]…S[i]归约为N; i:=j+1;S[i]:=N
            end;
            if S[j]⋖R or S[j]≡R then  begin i:=i+1; S[i]:=R end
            else error
until i=2 and R='#'  end;
```



# 分析树架子vs.分析树




- 考虑下面的文法G(E):
  (1) E→E+T | T
  (2) T→T*F | F
  (3) F→P ↑ F | P
  (4) P→(E) | i
  的句子i+i*i+i

# 5.2.4 优先函数

- 为了节省存储空间（$n^2 \rightarrow 2n$）和便于执行比较运算，用两个优先函数f和g，它们是从终结符号到整数的映射。对于终结符号a和b选择f和g，使之满足：
  - $f(a)<g(b)$，如果$a \lessdot b$
  - $f(a)=g(b)$，如果$a \doteq b$
  - $f(a)>g(b)$，如果$a \gtrdot b$。
- 损失
  - 错误检测能力降低
    - 如：id $\gtrdot$id不存在，但f(id)>g(id)可比较

# 表5.2 对应的优先函数：

| | + | - | * | / | ( | ) | id | # |
|---|---|---|---|---|---|---|---|---|
| ***f*** | **2** | **2** | **4** | **4** | **0** | **4** | **4** | **0** |
| ***g*** | **1** | **1** | **3** | **3** | **5** | **0** | **5** | **0** |

1) 构造优先函数的算法不是唯一的。

2) 存在一组优先函数，那就存在无穷组优先函数。

# 优先函数的构造

- 算法5.4 优先函数的构造。

  输入：算符优先矩阵;

  输出：表示输入矩阵的优先函数，或指出其不存在;

  步骤：

  1. 对$\forall a \in T \cup \{\#\}$，建立以$fa$和$ga$为标记的顶点；
  2. 对$\forall a, b \in T \cup \{\#\}$，若$a \succ b$或者$a \equiv b$，则从$fa$至$gb$画一条有向弧；若$a \prec b$或者$a \equiv b$，则从$gb$至$fa$画一条有向弧；
  3. 如果构造的有向图中有环路，则说明不存在优先函数；如果没有环路，则对$\forall a \in T \cup \{\#\}$，将$f(a)$设为从$fa$开始的最长路经的长度，将$g(a)$设为从$ga$开始的最长路经的长度。

# 例 5.10

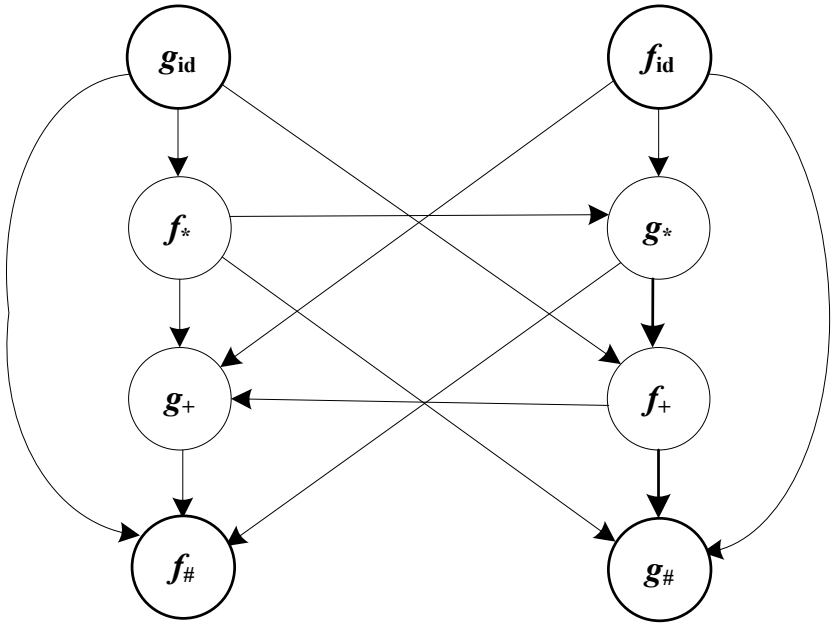
$G_{es}: E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow \text{id}$

| | + | * | id | # |
|---|---|---|---|---|
| + | ⋗ | ⋖ | ⋖ | ⋗ |
| * | ⋗ | ⋗ | ⋖ | ⋗ |
| id | ⋗ | ⋗ | | ⋗ |
| # | ⋖ | ⋖ | ⋖ | |

$G_{es}$的优先矩阵




| | + | * | id | # |
|---|---|---|---|---|
| $f$ | 2 | 4 | 4 | 0 |
| $g$ | 1 | 3 | 5 | 0 |

根据$G_{es}$的优先矩阵建立的有向图和优先函数

# 5.2.5 算符优先分析的出错处理

(1) 栈顶的终结符号和当前输入符号之间不存在任何优先关系；

(2) 发现被“归约对象”，但该“归约对象”不能满足归约要求。

- 对于第(1)种情况，为了进行错误恢复，必须修改栈、输入或两者都修改。
- 对于优先矩阵中的每个空白项，必须指定一个出错处理程序，而且同一程序可用在多个地方。
- 对于第(2)种情况，由于找不到与“归约对象”匹配的产生式右部，分析器可以继续将这些符号弹出栈，而不执行任何语义动作。

# 算符优先分析法小结

- 优点
  - 简单、效率高
  - 能够处理部分二义性文法
- 缺点
  - 文法书写限制大——强调算符之间的优先关系的唯一性
  - 占用内存空间大
  - 不规范、存在查不到的语法错误
  - 算法在发现最左素短语的尾时，需要回头寻找对应的头

# 单选题 1分

设置

算符优先文法句型（括在两个#之间）的一般形式写成#N1a1N2a2...NnanNn+1#，其中，每个ai都是_____，Ni是可有可无的_____。

A 终结符 非终结符

B 非终结符 空字符

C 终结符 空字符

D 空字符 非终结符

提交

单选题 1分

设置

算符优先分析过程是自下而上的归约过程，但未必是严格的最左归约。也就是说，算符优先分析法_____一种规范归约法。

A 是

B 不是

提交

# 5.1.3 状态法识别句柄

- 根据句柄的识别状态（句柄是逐步形成的）
  - 用状态来描述不同时刻下形成的那部分句柄
  - 因为句柄是产生式的右部，可用产生式来表示句柄的不同识别状态
- 例如： S→bBB可分解为如下识别状态
  - S→.bBB　移进b
  - S→bB.B　等待归约出B
  - S→b.BB　等待归约出B
  - S→bBB.　归约
- 采用这种方法，语法分析程序根据当前的分析状态就可以确定句柄的头和尾，并进行正确的归约。

# 5.3 LR分析法

- LR(k)分析法可分析LR(k)文法产生的语言
  - L： 从左到右扫描输入符号
  - R： 最右推导对应的最左归约
  - k： 超前读入k个符号，以便确定归约用的产生式
  - 使用语言的文法描述内涵解决句柄的识别问题，从语言的形式描述入手，为语法分析器的自动生成提供了前提和基础
  - 分析器根据当前的状态，并至多向前查看k个输入符号，就可以确定是否找到了句柄
    - 如果找到了句柄，则按相应的产生式归约，并进入相应的状态
    - 如果未找到句柄，则移进输入符号，并进入相应的状态

# 5.3 LR分析法

- 优点：
  - 能识别所有CFG描述的程序设计语言（**强**）
  - 实现高效，无回溯移入规约（**快**）
  - 包含预测分析法所能分析的文法类（**全**）
  - 自左向右扫描，能及时发现语法错误（**准**）
- 缺点：
  - 手工构建分析器工作量大
    - 解决方案：LR分析器的生成器-YACC
    - 输入文法，自动生成该文法的语法分析器（**自动**）

# 5.3.1 LR分析算法

- 产生分析表

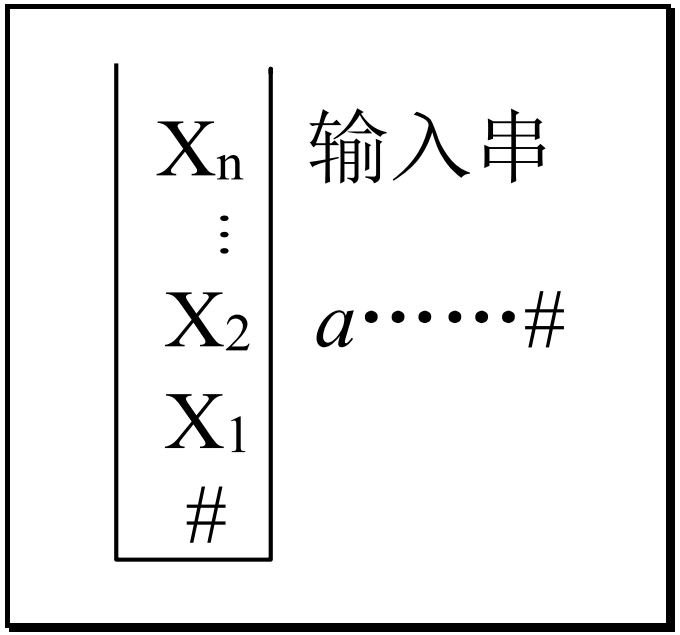



- LR分析器工作



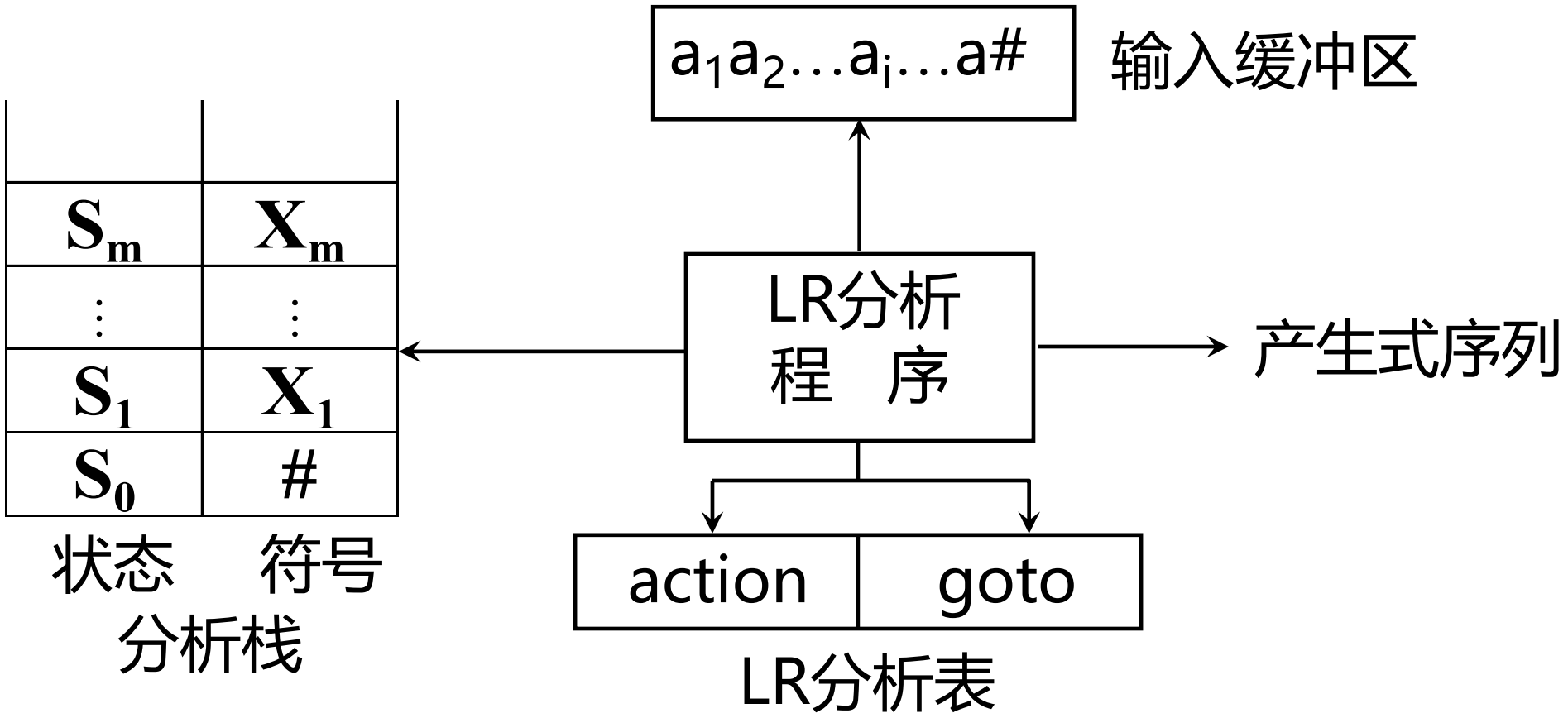
# LR分析器

- 规范归约的关键问题是寻找句柄.
  - 历史：已移入符号栈的内容
  - 展望：根据产生式推测未来可能遇到的输入符号
  - 现实：当前的输入符号



# LR分析器

- LR分析方法：把"历史"及"展望"综合抽象成状态；由栈顶的状态和现行的输入符号唯一确定每一步工作



# LR分析器

- LR分析器的核心是一张分析表

**ACTION[s，a]：当状态s面临输入符号a时，应采取什么动作**

**GOTO[s，X]：状态s面对文法符号X时，下一状态是什么**

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# LR分析器

移进(shift) 把(*s*，*a*)的下一状态*s*′ 和输入符号*a*推进栈，下一输入符号变成现行输入符号

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# LR分析器

归约(reduce)：指用某产生式A→β进行归约. 假若β的长度为r， 归约动作是， 去除栈顶r个项，使状态$s_{m-r}$变成栈顶状态，然后把($s_{m-r}$, A)的下一状态s'=GOTO[$s_{m-r}$, *A*]和文法符号A推进栈

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | | # | E | T | F |
| 0 | s5 | | | | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# LR分析器

接受 宣布分析成功，停止分析器工作

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# LR分析器

报错

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# LR分析器

- 分析开始时:

  状态　　已归约串　　输入串

  ($s_0$，　　#，　　$a_1a_2 \ldots a_n$ #)

- 以后每步的结果可以表示为:

  ($s_0\ s_1 \ldots s_m$ ，　# $X_1 \ldots X_m$ ，　$a_ia_{i+1} \ldots a_n$ #)

# LR分析器

$(s_0\ s_1\ \dots\ s_m\ ,\ \#\ X_1\ \dots\ X_m\ ,\ a_i a_{i+1}\ \dots\ a_n\ \#)$

$(s_0\ s_1 \dots s_{m-r}\ s_{m-r+1}\ \cdots\ s_m\ ,\ \#\ X_1 \dots X_{m-r}\ X_{m-r+1} \dots X_m\ ,\ a_i a_{i+1}\ \dots\ a_n\ \#)$

$(s_0\ s_1 \dots s_{m-r}\ ,\ \#\ X_1 \dots X_{m-r},\ a_i a_{i+1}\ \dots\ a_n\ \#)$

$(s_0\ s_1 \dots s_{m-r},\ \#\ X_1 \dots X_{m-r} A\ ,\ a_i a_{i+1}\ \dots\ a_n\ \#)$

$(s_0\ s_1 \dots s_{m-r} s\ ,\ \#\ X_1 \dots X_{m-r} A\ ,\ a_i a_{i+1}\ \dots\ a_n\ \#)$

此处, $s$=GOTO($s_{m-r}$, $A$), r为β的长度, β= $X_{m-r+1} \dots X_m$

3. 若ACTION($s_m$ , $a_i$)为"接受"，则三元式不再变化，变化过程终止，宣布分析成功.

4. 若ACTION($s_m$ , $a_i$)为"报错"，则三元式变化过程终止，报告错误.

# LR分析器示例

**文法G(E)：**

**(1) E→E + T**
**(2) E→T**
**(3) T→T*F**
**(4) T→F**
**(5) F→(E)**
**(6) F→i**

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# LR分析器示例

文法G(E)：

(1) E→E + T
(2) E→T
(3) T→T*F
(4) T→F
(5) F→(E)
(6) F→i

| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| (1) | 0 | # | i*i+i# |
| (2) | 05 | #i | *i+i# |
| (3) | 03 | #F | *i+i# |
| (4) | 02 | #T | *i+i# |
| (5) | 027 | #T* | i+i# |

输入串为i*i+i

T *
|
F
|
i

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# LR分析器示例

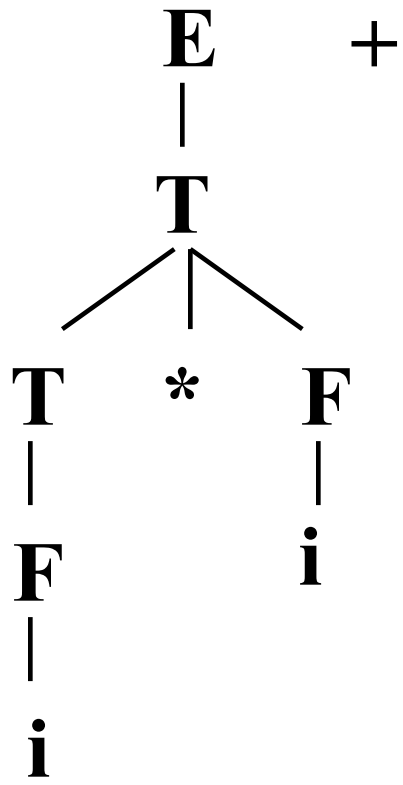
文法G(E):

(1) E→E + T
(2) E→T
(3) T→T*F
(4) T→F
(5) F→(E)
(6) F→i

| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| (5) | 027 | #T* | i+i# |
| (6) | 0275 | #T*i | +i# |
| (7) | 02710 | #T*F | +i# |
| (8) | 02 | #T | +i# |
| (9) | 01 | #E | +i# |
| (10) | 016 | #E+ | i# |

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

输入串为i*i+i



# LR分析器示例

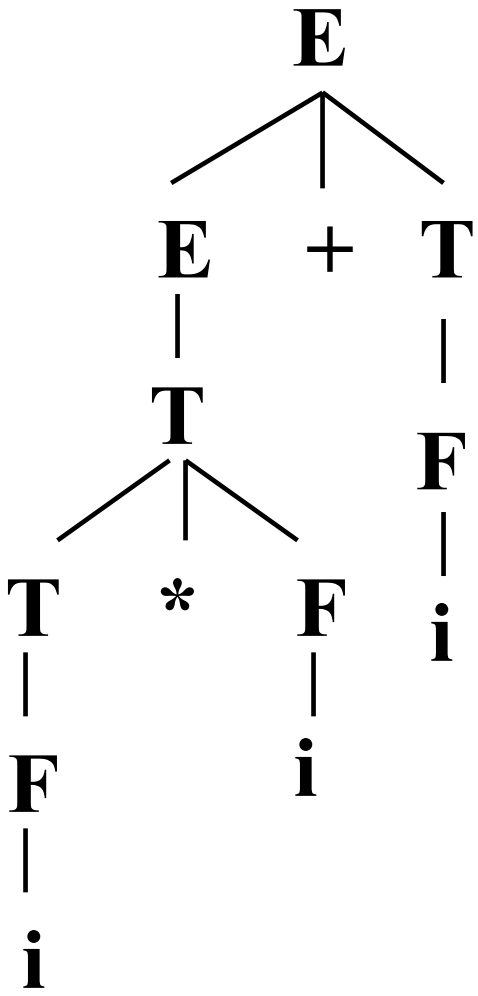
文法G(E):

(1) E→E + T
(2) E→T
(3) T→T*F
(4) T→F
(5) F→(E)
(6) F→i

| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| (10) | 016 | #E+ | i# |
| (11) | 0165 | #E+i | # |
| (12) | 0163 | #E+F | # |
| (13) | 0169 | #E+T | # |
| (14) | 01 | #E | # |
| (15) | 接受 | | |

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

输入串为i*i+i



# LR分析算法

▶算法5.5 *LR*分析算法。

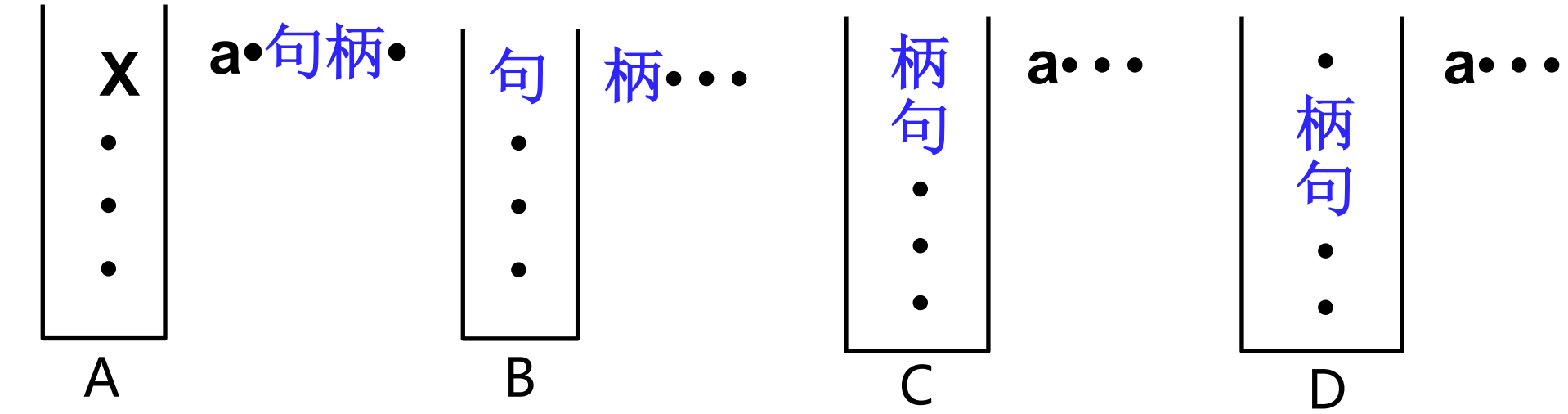
输入：文法*G*的*LR*分析表和输入串*w*;
输出：如果*w*∈*L*(*G*)，则输出*w*的自底向上分析，否则报错;
步骤:
1．将#和初始状态*S*0压入栈，将*w*#放入输入缓冲区；
2．令输入指针ip指向*w*#的第一个符号；
3．令*S*是栈顶状态，*a*是ip所指向的符号;
4．repeat
5．if *action*[*S*,*a*]=*Si* then　　/* *Si*表示移进*a*并转入状态*i**/
6．　begin
7．　　把符号*a*和状态*i*先后压入栈；
8．　　令ip指向下一输入符号
9．　end

# LR分析算法（续）

```
10. elseif action[S,a]=rk then  /* ri表示按第k个产生式A→β归约 */
11.     begin
12.                 从栈顶弹出2*|β|个符号;
13.                 令S'是现在的栈顶状态;
14.                 把A和goto[S',A]先后压入栈中;
15.                 输出产生式 A→β
16.     end
17. elseif action[S,a]= acc then
18.     return
19. else
20.     error();
```

# 讨论：规范归约过程中栈内符号串

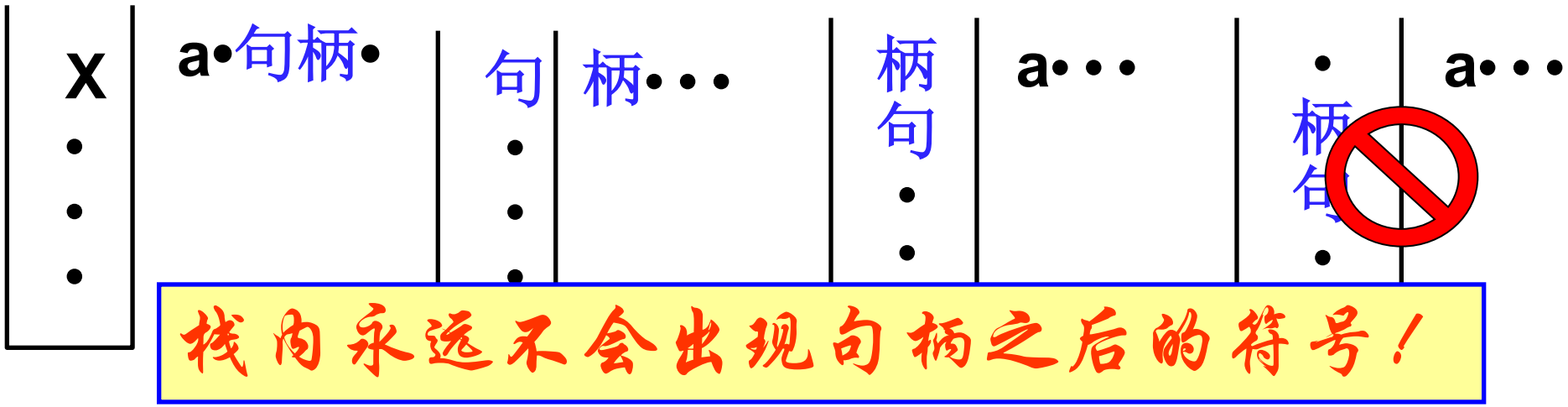
- 对于句子，在规范归约过程中，栈内的符号串和扫描剩下的输入符号串构成了一个规范句型，下面哪种格局不会出现：




iclicker

# 规范归约过程中栈内符号串

- 规范归约过程中
  - 栈内的符号串和扫描剩下的输入符号串构成了一个规范句型
  - 栈内的如果出现句柄，句柄一定在栈的顶部



# 字的前缀、活前缀

- 字的前缀：是指字的任意首部，如字*abc*的前缀有ε，*a*，*ab*，*abc*
- 活前缀：是指规范句型的一个前缀，这种前缀不含句柄之后的任何符号。即，对于规范句型αβδ，β为句柄，如果$\alpha\beta=u_1u_2...u_r$，则符号串$u_1u_2...u_i(1\le i\le r)$是αβδ的活前缀。(δ必为终结符串)

# 指导思想——目标驱动



- 踢足球

  “如果你不知道怎样踢球，就往球门方向踢”

  ——施拉普纳

- LR分析

  “如果你不知道怎样分析，就保证栈中总是活前缀”

- 哪些字符串是活前缀?
- 能不能构造一个DFA来识别活前缀?

回答：对于一个文法G, 可以构造一个DFA,它能识别G的所有活前缀.

# 字的前缀、活前缀

- 字的前缀：是指字的任意首部，如字*abc*的前缀有ε，*a*，*ab*，*abc*
- 活前缀：是指规范句型的一个前缀，这种前缀不含句柄之后的任何符号。即，对于规范句型$\alpha\beta\delta$，$\beta$为句柄，如果$\alpha\beta=u_1u_2...u_r$，则符号串$u_1u_2...u_i(1\le i\le r)$是$\alpha\beta\delta$的活前缀。($\delta$必为终结符串)
- 规范归约过程中，保证分析栈中总是活前缀，就说明分析采取的移进/归约动作是正确的

# 规范句型活前缀

- 分析栈中内容+剩余输入符号=规范句型
  - 分析栈中内容为某一句型的前缀
- 来自分析栈的活前缀(Active Prefix)
  - 不含句柄右侧任意符号的规范句型的前缀
- 例：id + id * id 的分析中
  - 句型 E + id . * id 和 E + E * . id

活前缀　　活前缀

$$S \Rightarrow^*_{rm} \alpha Aw \Rightarrow_{rm} \alpha\beta_1\beta_2 w$$

# 规范句型活前缀

- 规范归约所得到的规范句型(Canonical Sentential Form)的活前缀是出现在分析栈中的符号串，所以，不会出现句柄之后的任何字符，而且相应的后缀正是输入串中还未处理的终结符号串。
- 活前缀与句柄的关系
  - 包含句柄 $A\rightarrow\beta.$
  - 包含句柄的部分符号$A\rightarrow\beta_1.\beta_2$
  - 不含句柄的任何符号$A\rightarrow.\beta$
- 于是，对句柄的识别就转变成了对规范句型活前缀的识别。

# 规范句型活前缀的识别

- 一个文法的规范句型的所有活前缀构成一个语言，而且该语言是正则的，可以用一个DFA识别。

- *LR*分析器的工作过程实际上就是逐步产生规范句型的活前缀的过程，并利用一个栈存放识别出来的句型的活前缀部分和相应的状态。

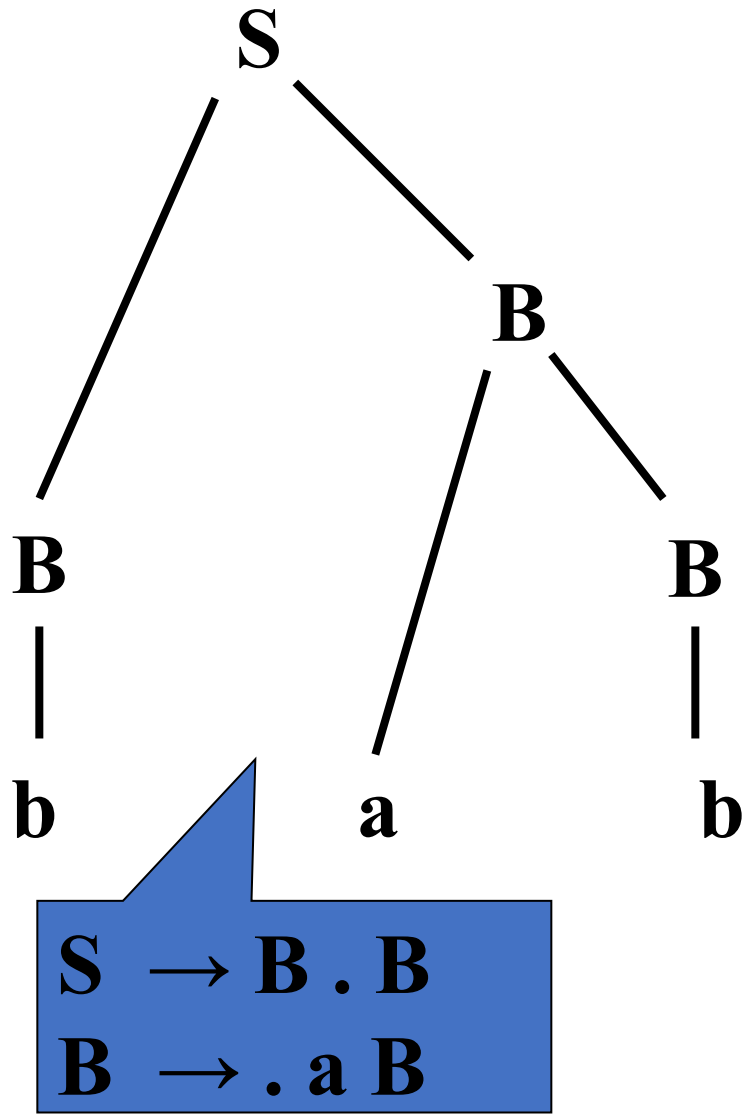
- 如果能构造一个识别文法所有规范句型活前缀的有穷自动机(DFA)，就能很方便地构造出*LR*分析表。

# 5.3.2 LR(0)分析表的构造

- LR(0)项目——从产生式寻找归约方法
  - 右部某个位置标有圆点的产生式称为相应文法的LR(0)项目（Item）
  - 例 S→.bBB　S→bB.B　S→b.BB S→bBB.
  - 归约（Reduce）项目: S→aBB.
  - 移进（Shift）项目：S→.bBB
  - 待约项目：S→b.BB　S→bB.B

# 项目的意义

- 用项目表示分析的进程(句柄的识别状态)
- 方法：在产生式右部加一圆点以分割已获取的内容和待获取的内容：构成句柄



# 项目的意义

- 文法G(S′)

  S′→E
  E→aA|bB
  A→cA|d
  B→cB|d

- 该文法的项目有：

| | | |
|---|---|---|
| 1. S′→·E | 2. S′→E· | 3. E→·aA |
| 4. E→a·A | 5. E→aA· | 6. A→·cA |
| 7. A→c·A | 8. A→cA· | 9. A→·d |
| 10. A→d· | 11. E→·bB | 12. E→b·B |
| 13. E→bB· | 14. B→·cB | 15. B→c·B |
| 16. B→cB· | 17. B→·d | 18. B→d· |

# 拓广(Augmented)文法

- 需要一个对“归约成S”的表示（只有一个接受状态）
- 文法 G= (V, T, P, S)的拓广文法 G'：
  - G'=（V ∪ {S'}，T，P ∪{S'→S}，S'）
  - S'∉V
  - 对应S'→.S（分析开始）和S'→S.（分析成功）
- 例5.13

0) S'→S
1) S→BB
2) B→aB
3) B→b

# 构造DFA

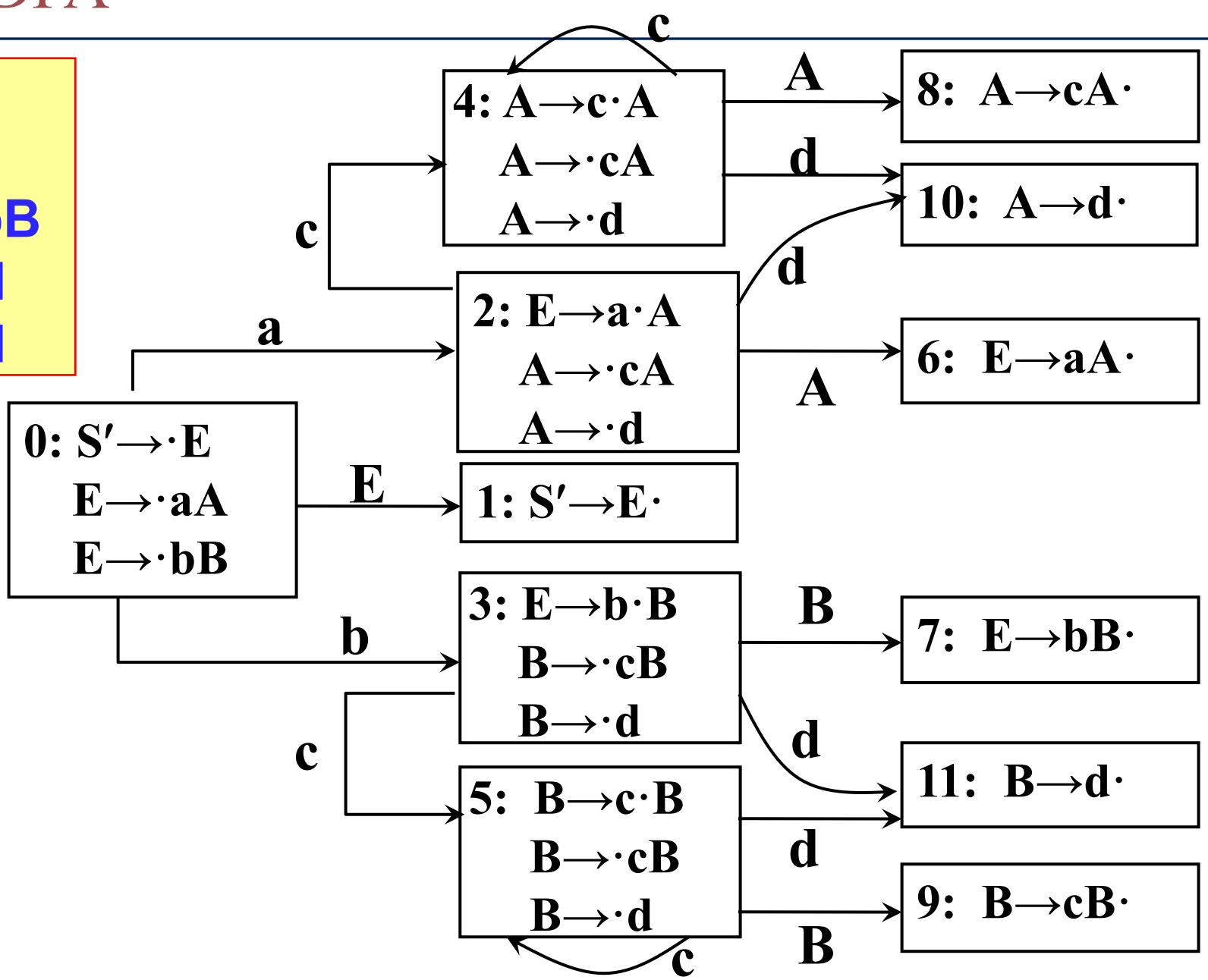
**G'(S'):**

$S' \rightarrow E$

$E \rightarrow aA|bB$

$A \rightarrow cA|d$

$B \rightarrow cB|d$



单选题 1分

设置

LR分析器包括_____、_____。

A 一个总控程序 一张分析表

B 一个子程序 多张分析表

C 一个总控程序 多张分析表

D 多个子程序 多张分析表

提交

单选题 1分

设置

LR分析器核心部分是一张分析表，该表包括_____、_____等子表。

A ACTION 空子表

B 空子表 GOTO

C GOTO ACTION

D GOTO 符号表

提交

# 多选题 1分

设置

对LR分析表的构造，有可能存在_____、_____动作冲突。

A 移进/归约

B 移进/移进

C 归约/归约

D 接受/移进

提交

单选题 1分

设置

LR分析表中的动作表（action）是以（ ）作为列标题的。

A 终结符

B 非终结符

C 终结符或非终结符

D 终结符和结束符

提交

单选题 1分

设置

LR分析表中的转移表（goto）是以（）作为列标题的。

A 终结符

B 非终结符

C 终结符或非终结符

D 表示状态的整型数

提交

# 构造识别G的所有规范句型活前缀的DFA

- 问题：如何设计能够指导分析器运行，并且能够根据当前状态（栈顶）确定句柄——归约对象的头——的装置
  - 构成识别一个文法活前缀的DFA的项目集(状态)的全体称为文法的LR(0)项目集规范族。

# 项目集闭包的计算

项目集 I 的闭包（Closure）

CLOSURE(I)=I ∪{B→.γ| A→α .Bβ∈I, B→γ∈P}

算法

J:=I;

repeat

J=J∪{B→.η|A→α.Bβ∈J, B→η∈P}

until J不再扩大

**G'(S'):**

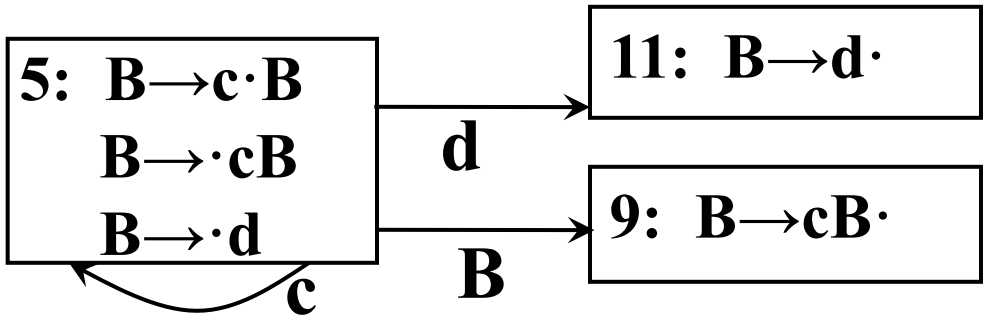
**S′→E**
**E→aA|bB**
**A→cA|d**
**B→cB|d**

**0: S′→·E**
**E→·aA**
**E→·bB**

# 闭包之间的转移

- 后继项目（Successive Item）
  - A→α.Xβ的后继项目是A→αX.β
- 闭包之间的转移
  - GO(I,X)=CLOSURE({A→αX.β| A→α.Xβ∈I})

**G’(S’):**
**S′→E**
**E→aA|bB**
**A→cA|d**
**B→cB|d**



# 状态转移的计算

- 确定在某状态遇到一个文法符号后的状态转移目标

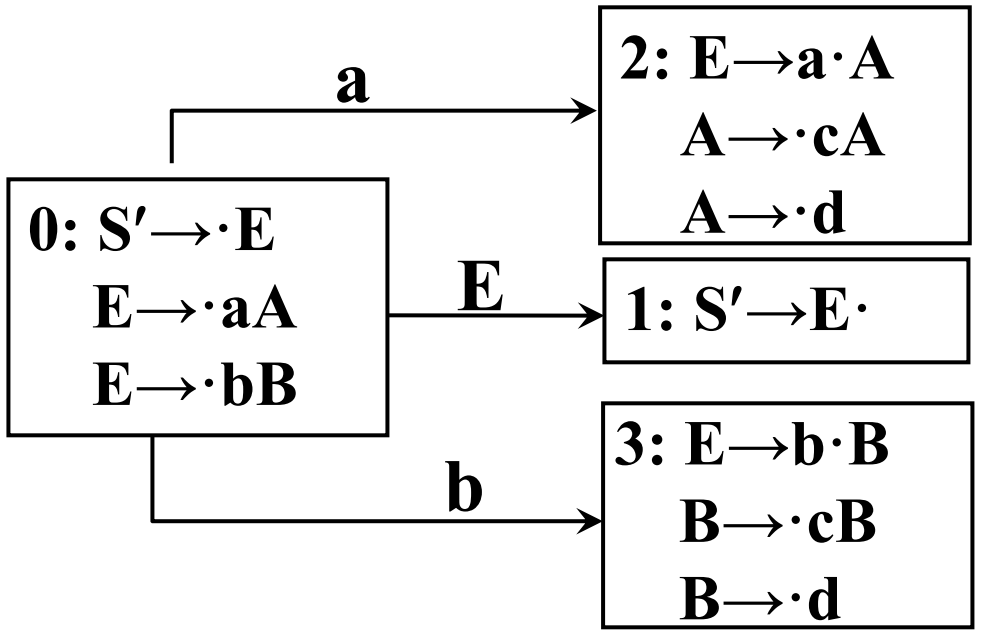
function GO(I, $X$);
begin
  J:=∅;
for I中每个形如$A\rightarrow\alpha.X\beta$的项目do
  begin  J:=J∪{$A\rightarrow\alpha X.\beta$}  end;
  return CLOSURE(J)
end;

# 状态转移的计算

- 文法G(S′)

  **S′→E**
  **E→aA|bB**
  **A→cA|d**
  **B→cB|d**

- $I_0$={S′→·E, E→·aA, E→·bB}




GO($I_0$, E)= closure(J)=closure({S′ →E·})
= {S′ →E·} = $I_1$

GO($I_0$, a)= closure(J)=closure({E→a·A})
={ E→a·A, A→·cA, A→·d} )=$I_2$

GO($I_0$, b)= closure(J)=closure ({E →b·B})
={E →b·B, B→·cB, B→·d}= $I_3$

# 识别拓广文法所有规范句型活前缀的DFA

识别文法G=（V，T，P，S）的拓广文法G‘的所有规范句型活前缀的DFA：

$$M=(C, V\cup T, go, I_0, C)$$

- $I_0$=CLOSURE({S‘ →.S})　　$C=\{I_0\}\cup\{I|\exists J\in C, X\in V\cup T, I=go(J,X)\}$

称为G'的LR(0)项目集规范族(Canonical Collection)

```
begin
C := {closure({ S'→.S})};
 repeat
   for ∀I∈C，∀ X ∈ V∪T
     if go(I,X)≠Φ & go(I,X)∉C then
                C=C∪{go(I,X)}
 until C不变化
end.
```

# 构造DFA

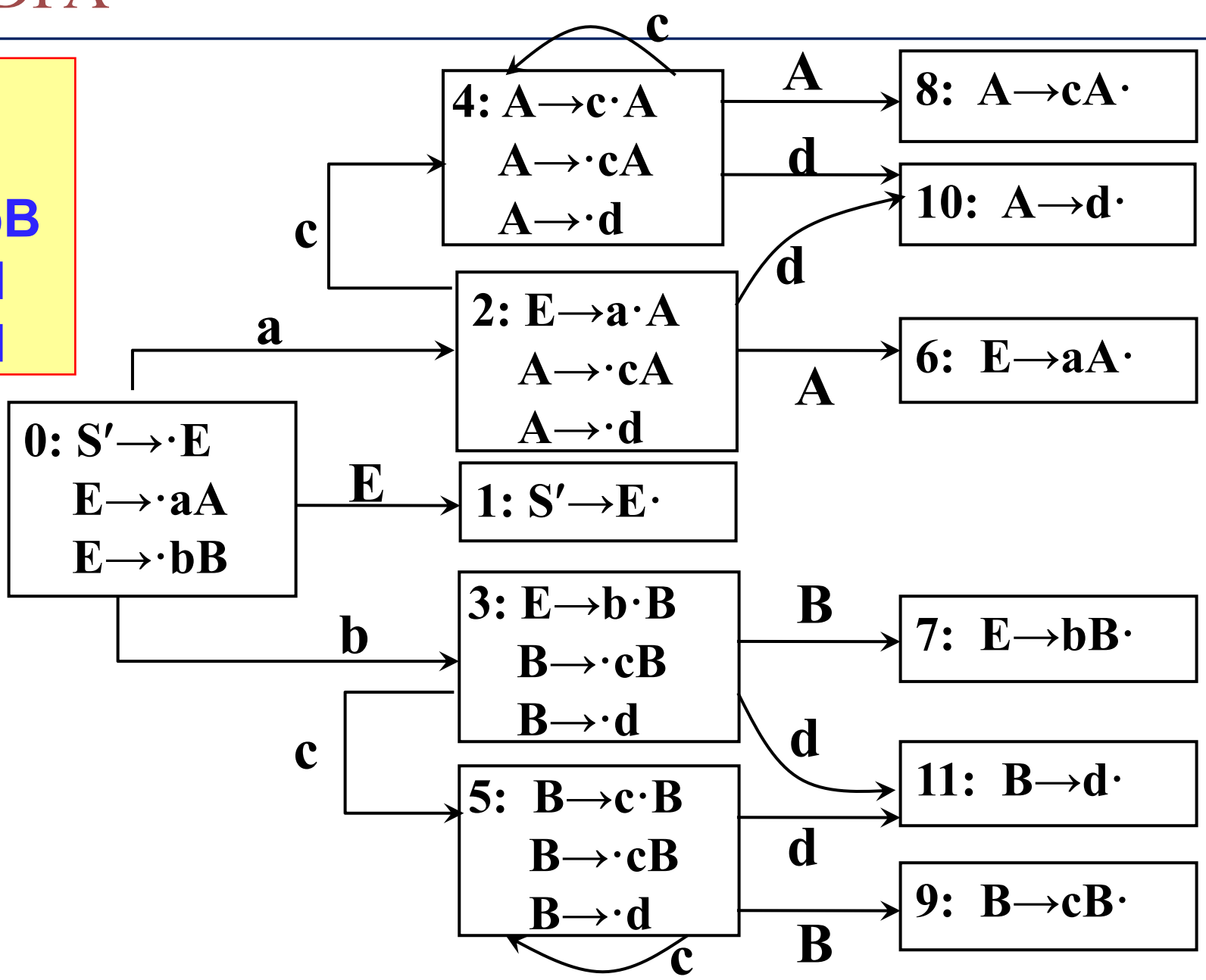
**G'(S'):**

$S' \rightarrow E$

$E \rightarrow aA|bB$

$A \rightarrow cA|d$

$B \rightarrow cB|d$



# LR(0)分析表的构造

- 假若一个文法G的拓广文法G′的活前缀识别自动机中的每个状态(项目集)不存在下述情况：

1) 既含移进项目又含归约项目，

2) 含有多个归约项目

则称G是一个LR(0)文法。

# LR(0)分析表的ACTION和GOTO子表构造

1. 若项目A→α·aβ属于$I_k$且GO($I_k$, a) = $I_j$，a为终结符，则置ACTION[k,a] 为“sj”。
2. 若项目A→α·属于$I_k$，那么，对任何终结符a(或结束符#)，置ACTION[k,a]为“rj”(假定产生式A→α是文法G′的第j个产生式)。
3. 若项目S′→S·属于$I_k$，则置ACTION[k,#]为 “acc”。
4. 若GO($I_k$,A) = $I_j$，A为非终结符，则置GOTO[k,A]=j。
5. 分析表中凡不能用规则1至4填入信息的空白格均置上“报错标志”。

# LR(0)分析表的构造算法

- 算法5.6 *LR*(0)分析表的构造。

输入：文法$G=(V, T, P, S)$的拓广文法$G'$;

输出：$G'$的*LR*(0)分析表，即*action*表和*goto*表;

步骤:

1.令$I_0$= CLOSURE($\{S' \rightarrow .S\}$)，构造$G'$的*LR*(0)项目集规范族$C=\{ I_0,I_1,\ldots,I_n\}$

2. 让$I_i$对应状态$i$，$I_0$对应状态0，0为初始状态。

3. for $k$=0 to $n$ do begin

(1) if $A\rightarrow\alpha.a\beta\in I_k$ & $a\in T$ & GO($I_k$, $a$)=$I_j$ then *action*[$k$,$a$]:=$Sj$;

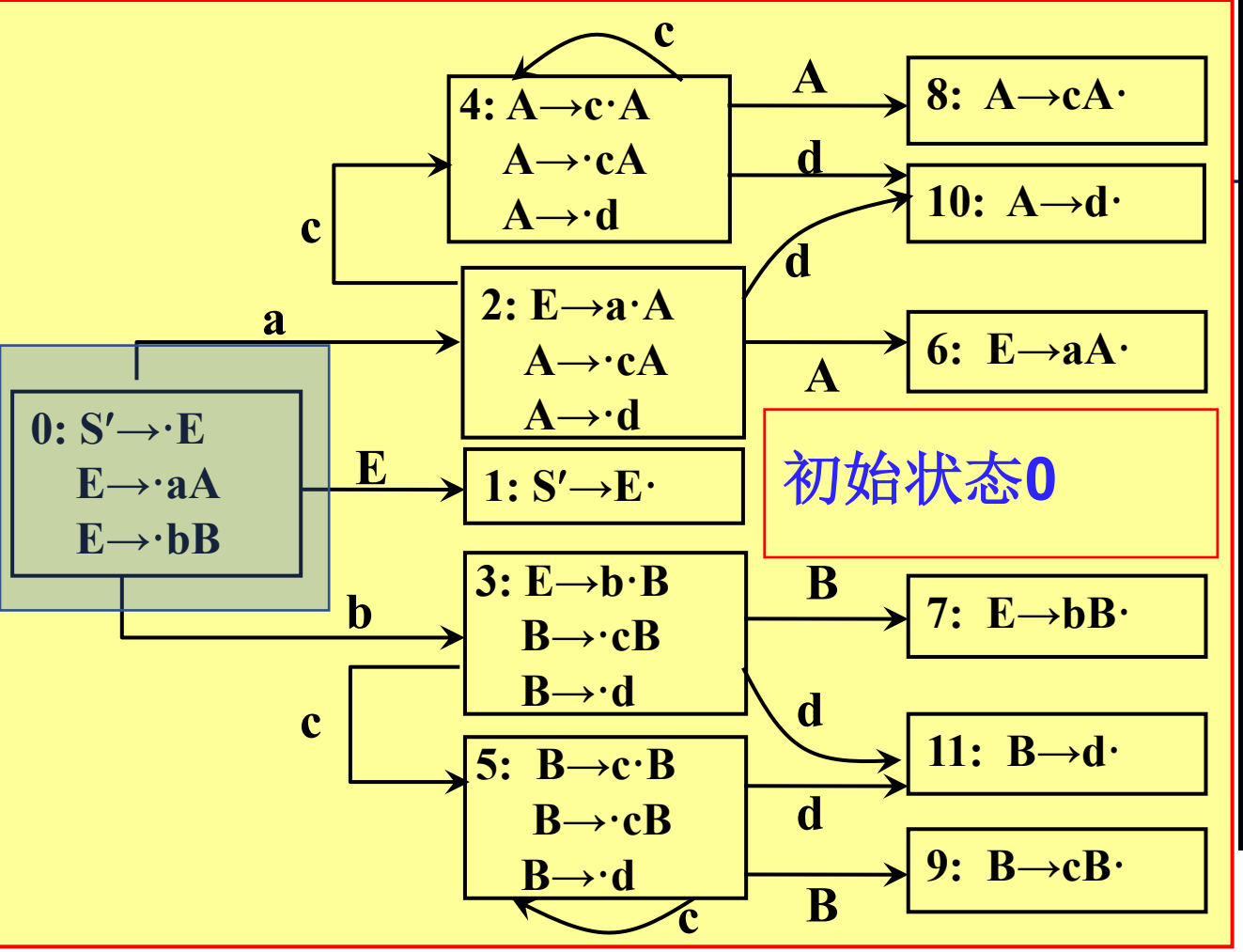
(2) if $A\rightarrow\alpha.\in I_k$ & $A\rightarrow\alpha$为$G$的第$j$个产生式then

for $\forall a\in T\cup\{\#\}$ do *action*[$k$,$a$]:=$rj$;

(3) if $S'\rightarrow S.\in I_k$ then *action*[$k$,#]:=acc end;

(4) if $A\rightarrow\alpha.B\beta\in I_k$ & $B\in V$ & GO($I_k$, $B$)=$I_j$ then *goto*[$k$,$B$]:=$j$;

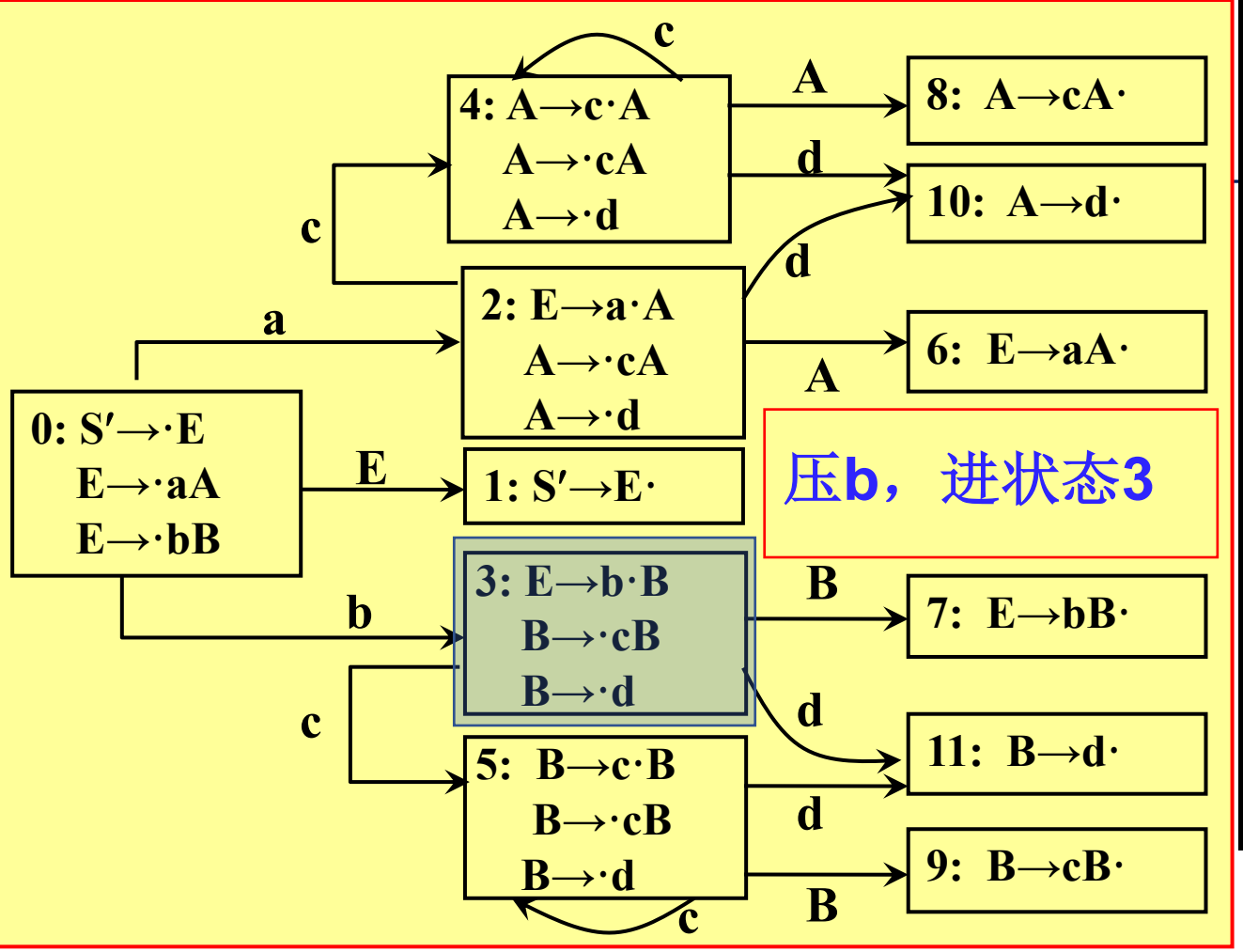
4. 上述(1)到(4)步未填入信息的表项均置为error。




| 状态 | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| 馕 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

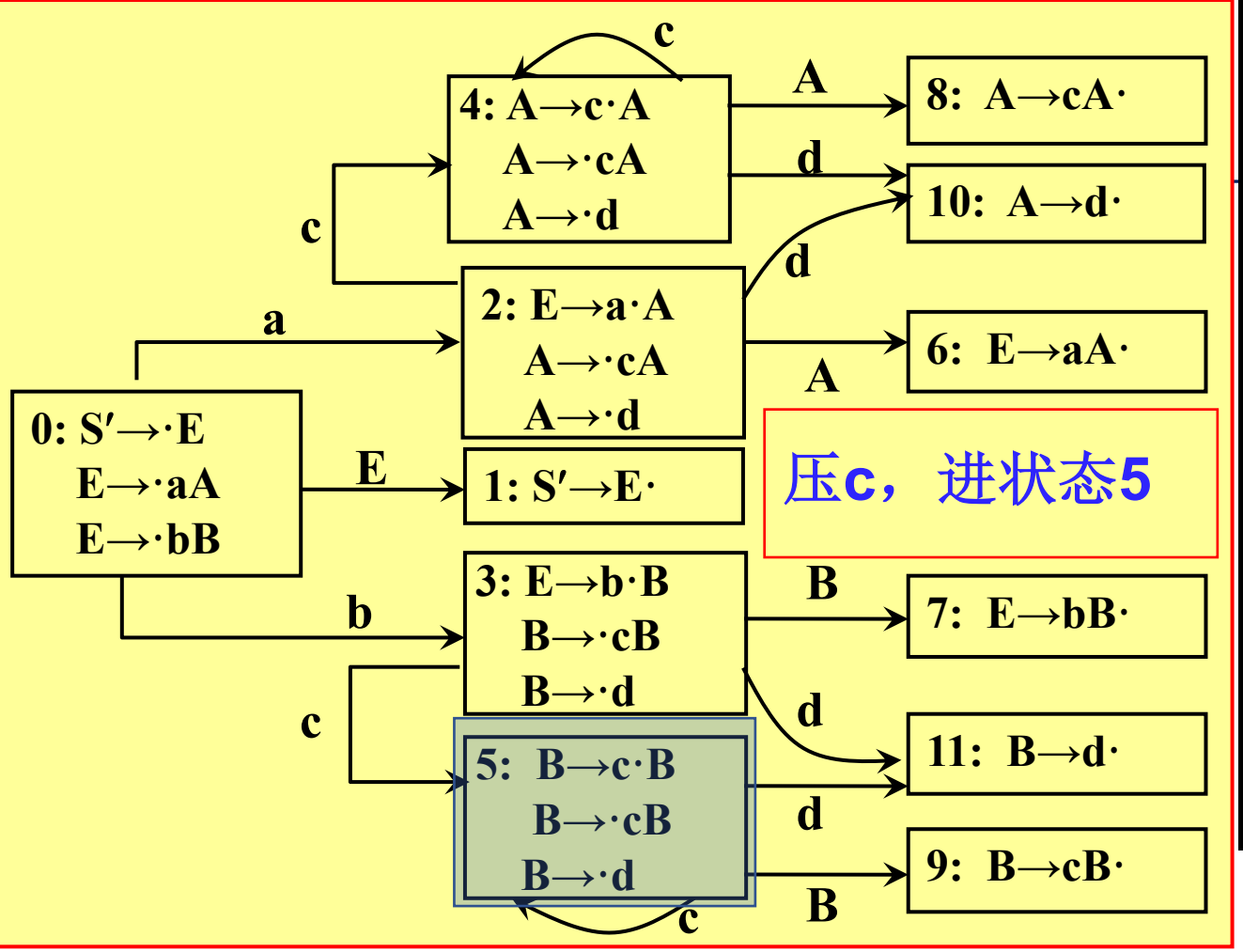
| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |




| 状态 | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| 馊 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

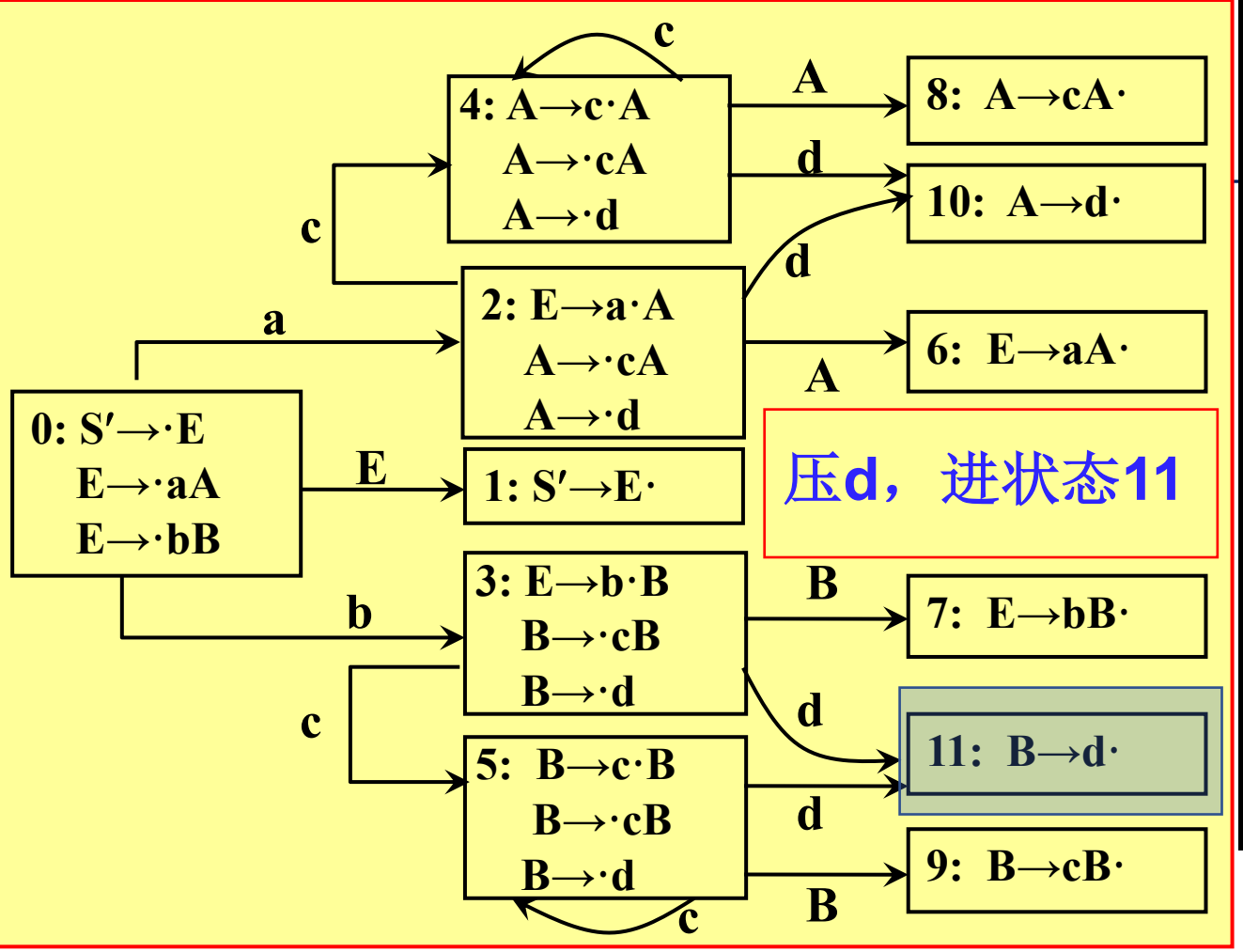
| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |
| 2 | 03 | #b | cd# |




| 状态 | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| 馈 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |
| 2 | 03 | #b | cd# |
| 3 | 035 | #bc | d# |

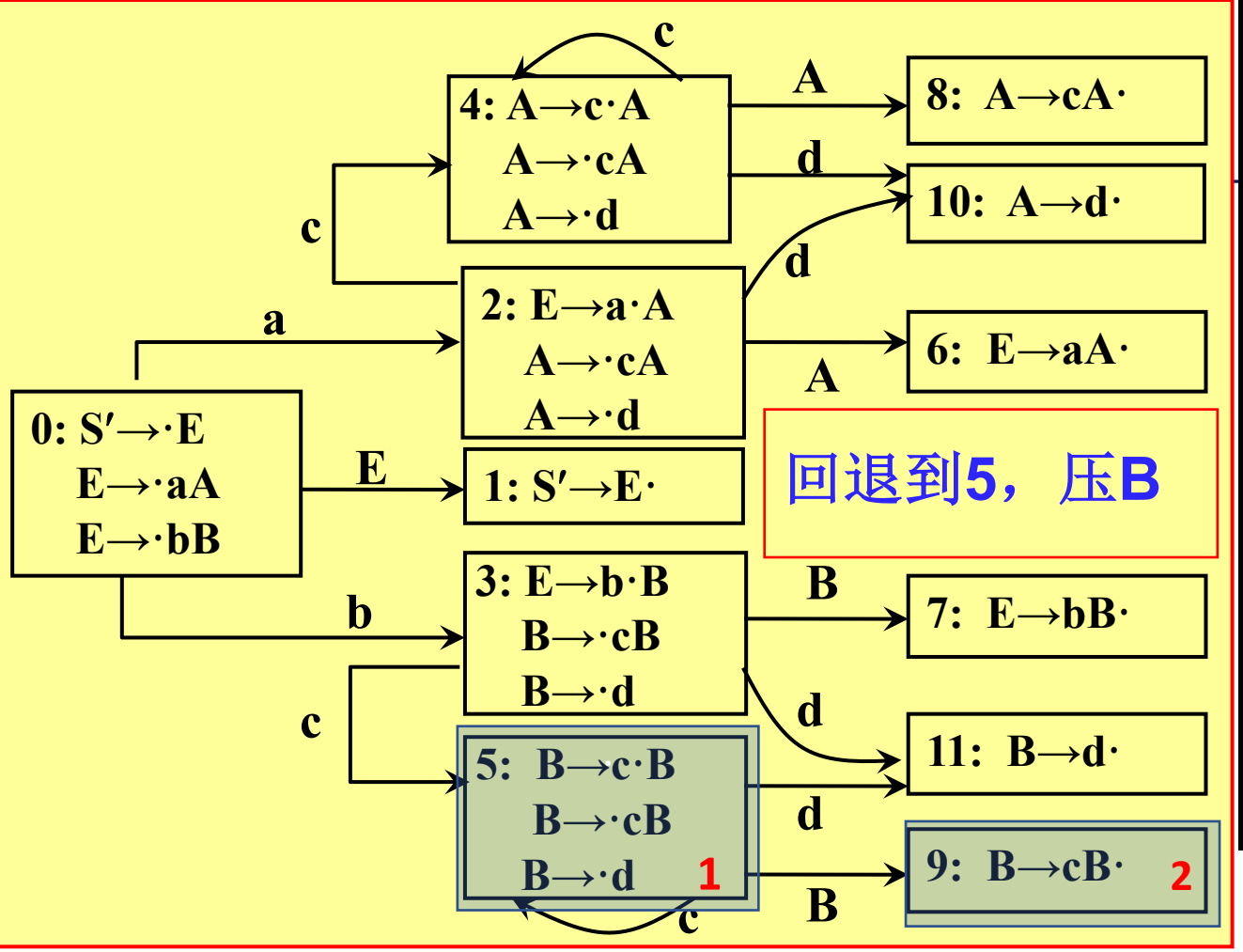



| 状态 | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| 馕 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

4 03511 #bcd #

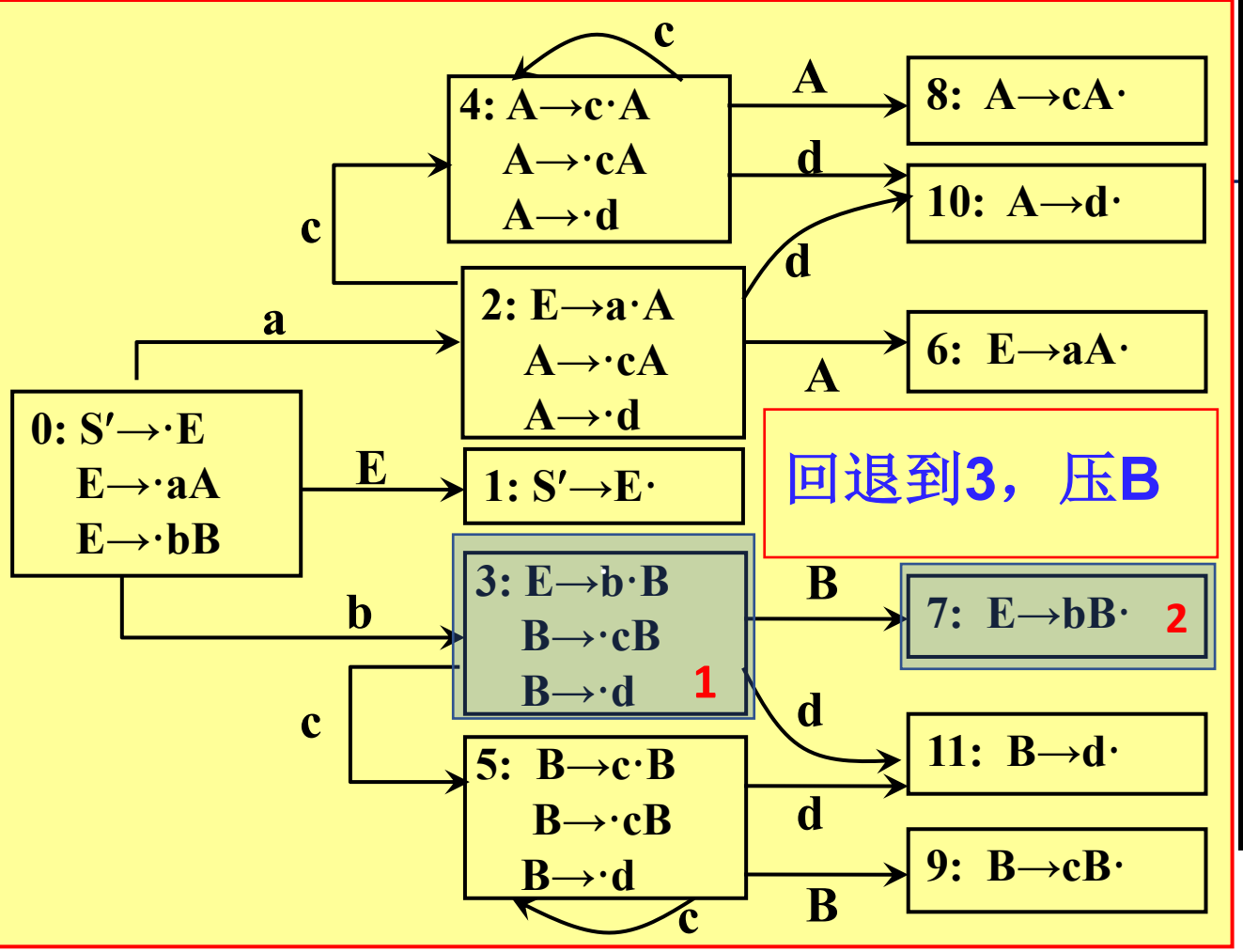
| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |
| 2 | 03 | #b | cd# |
| 3 | 035 | #bc | d# |




| | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| □馊 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

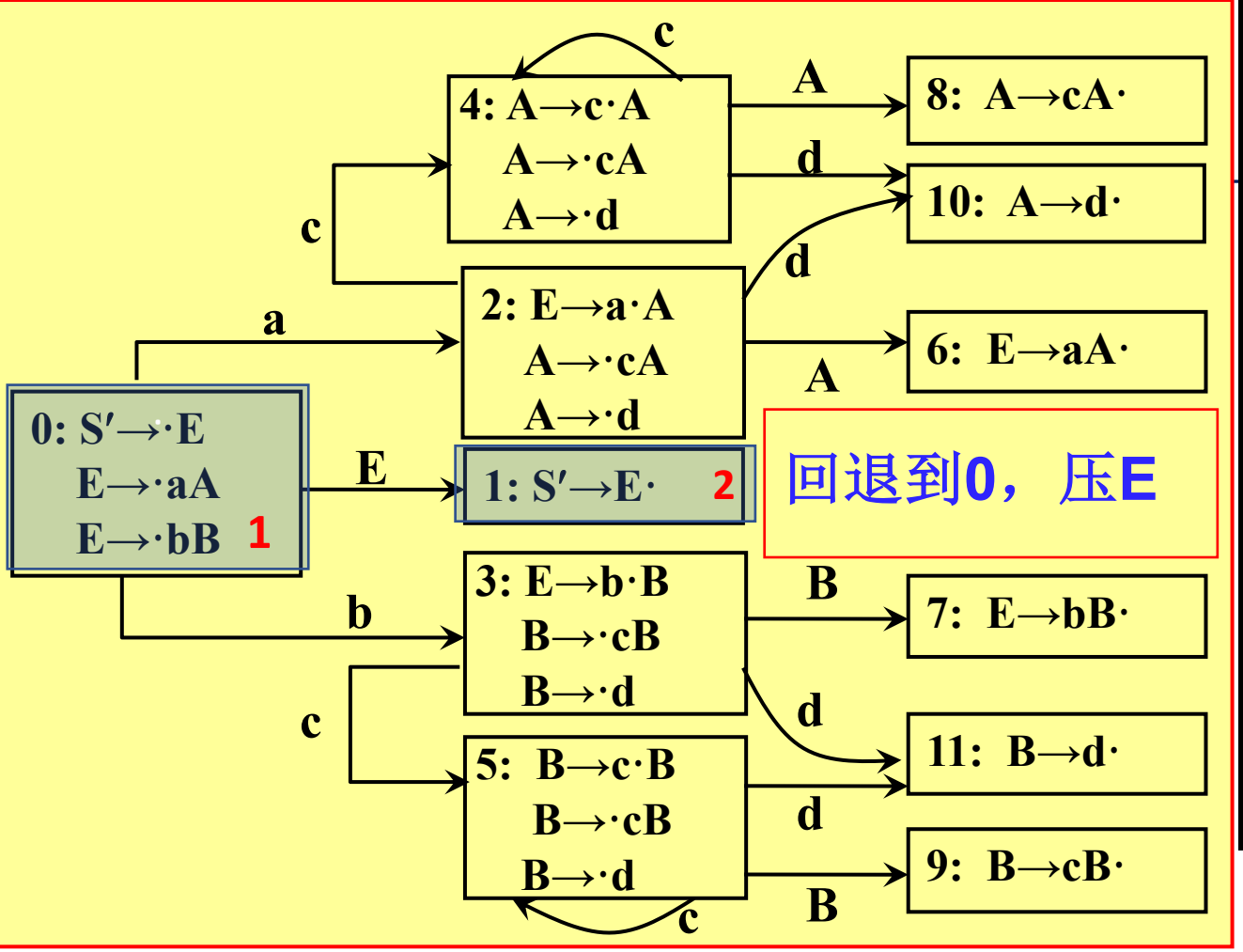
| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |
| 2 | 03 | #b | cd# |
| 3 | 035 | #bc | d# |
| 4 | 03511 | #bcd | # |
| 5 | 0359 | #bcB | # |




| | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| □馊 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

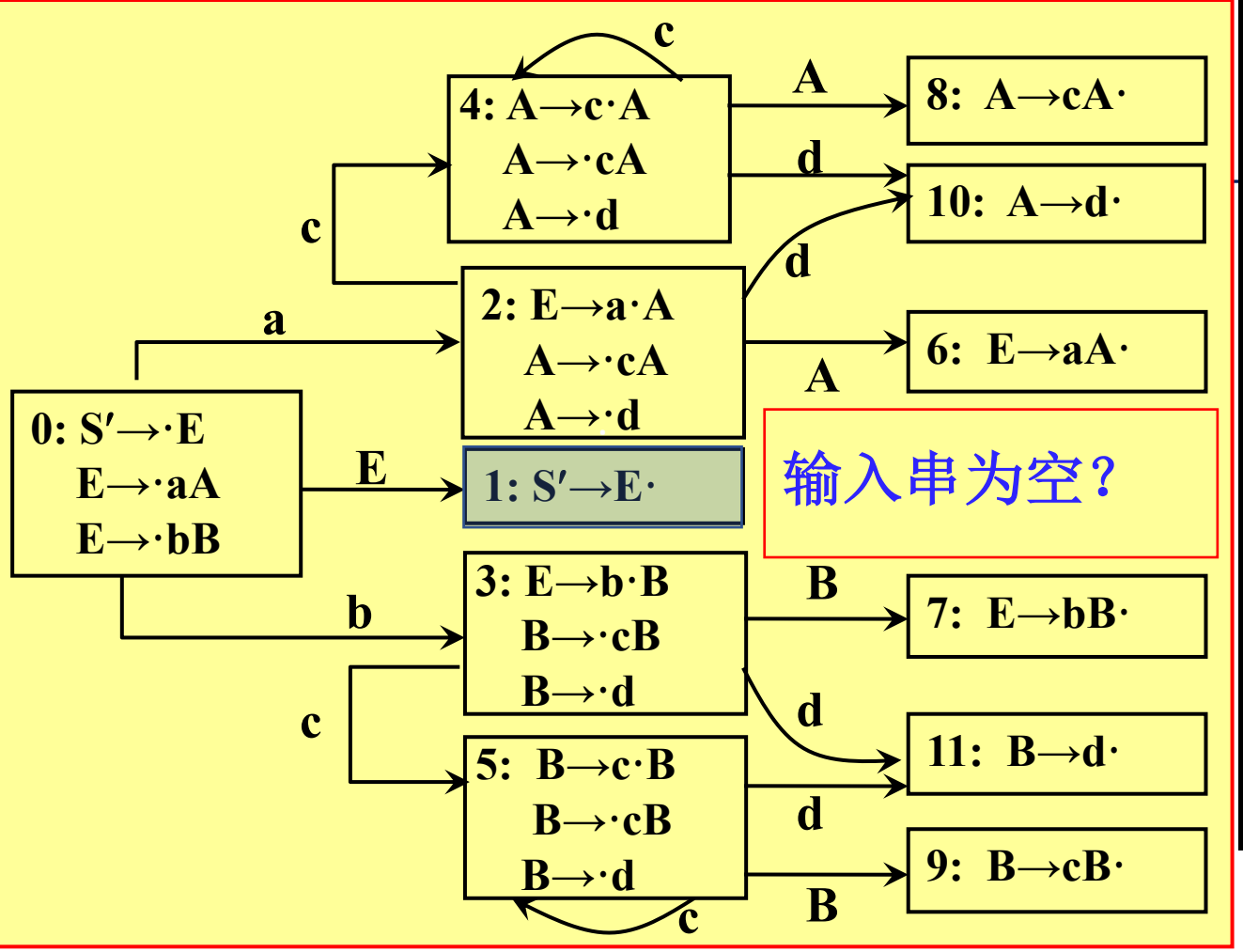
| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |
| 2 | 03 | #b | cd# |
| 3 | 035 | #bc | d# |
| 4 | 03511 | #bcd | # |
| 5 | 0359 | #bcB | # |
| 6 | 037 | #bB | # |




| 状态 | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| 馈 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |
| 2 | 03 | #b | cd# |
| 3 | 035 | #bc | d# |
| 4 | 03511 | #bcd | # |
| 5 | 0359 | #bcB | # |
| 6 | 037 | #bB | # |
| 7 | 01 | #E | # |



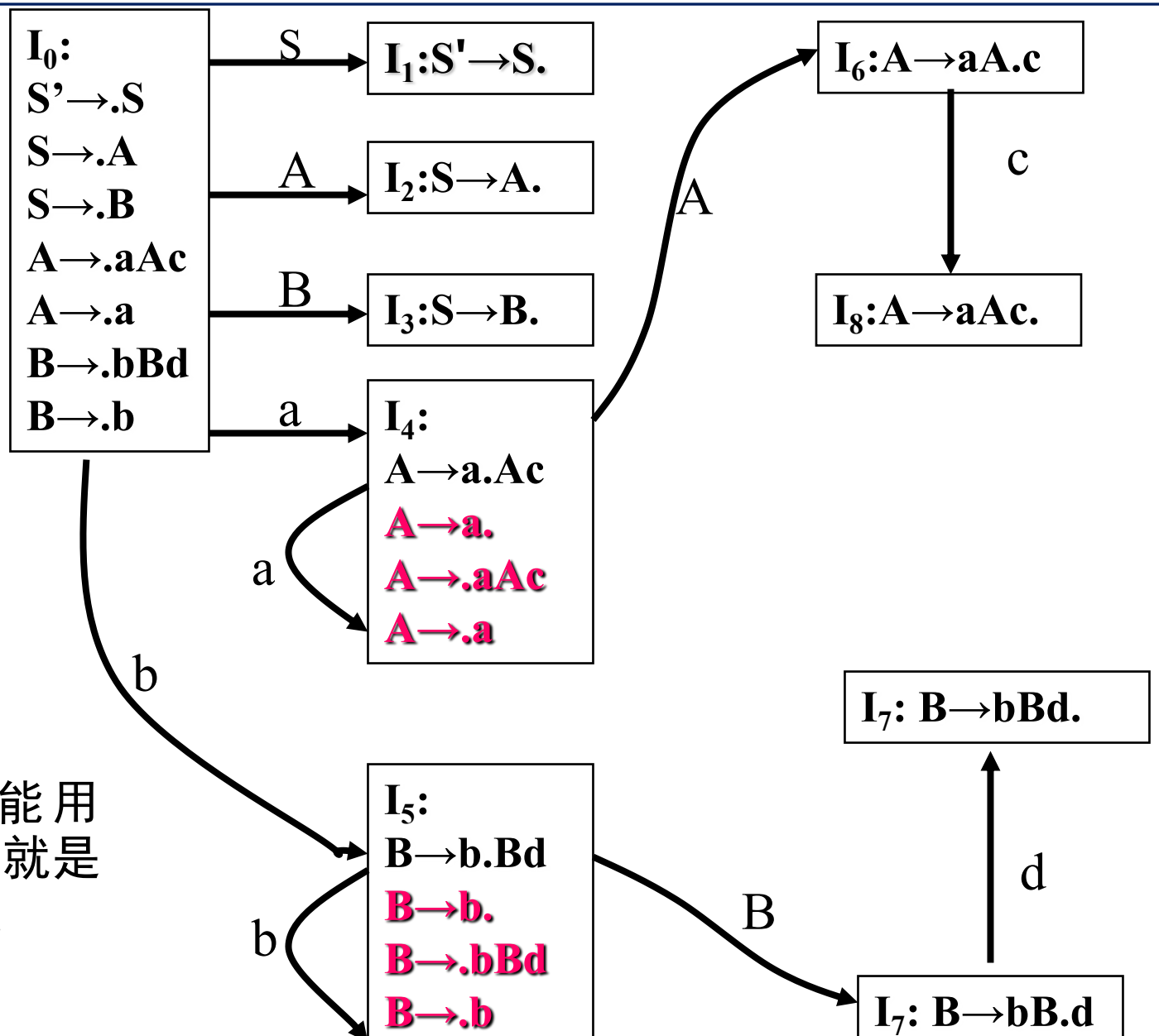

| 状态 | ACTION | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| 馈 | a | b | c | d | # | E | A | B |
| 0 | s2 | s3 | | | | 1 | | |
| 1 | | | | | acc | | | |
| 2 | | | s4 | s10 | | | 6 | |
| 3 | | | s5 | s11 | | | | 7 |
| 4 | | | s4 | s10 | | | 8 | |
| 5 | | | s5 | s11 | | | | 9 |
| 6 | r1 | r1 | r1 | r1 | r1 | | | |
| 7 | r2 | r2 | r2 | r2 | r2 | | | |
| 8 | r3 | r3 | r3 | r3 | r3 | | | |
| 9 | r5 | r5 | r5 | r5 | r5 | | | |
| 10 | r4 | r4 | r4 | r4 | r4 | | | |
| 11 | r6 | r6 | r6 | r6 | r6 | | | |

0:S′→E
1:E→aA
2:E→ bB
3:A→cA
4:A→ d
5:B→cB
6:B→ d

| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 1 | 0 | # | bcd# |
| 2 | 03 | #b | cd# |
| 3 | 035 | #bc | d# |
| 4 | 0351<u>1</u> | #bcd | # |
| 5 | 0359 | #bcB | # |
| 6 | 037 | #bB | # |
| 7 | 01 | #E | # |
| 8 | 接受 | | |

# LR(0)不是总有效的

S' →S
S →A|B
A →aAc
A →a
B →bBd
B →b




上下文无关文法不是都能用LR(0)方法进行分析的，也就是说，CFG不总是LR(0)文法.

# 项目集 I 的相容

- 如果 I 中至少含两个归约项目，则称 I 有归约—归约冲突（Reduce/Reduce Conflict）
- 如果 I 中既含归约项目，又含移进项目，则称 I 有移进—归约冲突（Shift/Reduce Conflict）
- 如果 I 既没有归约—归约冲突，又没有移进—归约冲突，则称 I 是相容的(Consistent)，否则称 I 是不相容的
- 对文法G，如果$\forall$ I$\in$C,都是相容的，则称G为LR(0)文法

# 单选题 1分

设置

若B为非终结符，则A→α·Bβ为( )项目。

A 归约

B 移进

C 接受

D 待约

提交

单选题 1分

设置

下列项目中为可归约项目的是（）。

A E′→· E

B L→ •

C L→-· L

D F→L*· F

提交

单选题 1分

设置

在LR(0)的Action表中，如果某行中存在标记为“rj”的栏，则( )。

A 该行必定填满“rj”

B 该行未必填满“rj”

C 其他行可能也有“rj”

D goto表中也可能有“rj”

提交

# SLR分析表的构造

- 考察下面的文法：

(0) S′→E
(1) E→E+T
(2) E→T
(3) T→T*F
(4) T→F
(5) F→(E)
(6) F→i

$I_0$: S′→·E
E→·E+T
E→·T
T→·T*F
T→·F
F→· (E)
F→·i

$I_1$: S′→E·
E→E·+T

$I_2$: E→T·
T→T·*F

$I_3$: T→F·

$I_4$: F→(·E)
E→·E+T
E→·T
T→·T*F
T→·F
F→· (E)
F→·i

$I_5$: F→i·

$I_6$: E→E+·T
T→·T*F
T→·F
F→·(E)
F→·i

$I_7$: T→T*·F
F→·(E)
F→·i

$I_8$: F→(E·)
E→E·+T

$I_9$: E→E+T·
T→T·*F

$I_{10}$: T→T*F·

$I_{11}$: F→(E)·

# SLR分析表的构造

- 考察下面的文法：

(0) S′→E
(1) E→E+T
(2) E→T
(3) T→T*F
(4) T→F
(5) F→(E)
(6) F→i

$I_0$: S′→·E
E→·E+T
E→·T
T→·T*F
T→·F
F→· (E)
F→·i

$I_1$: S′→E·
E→E·+T

$I_2$: E→T·
T→T·*F

$I_3$: T→F·

$I_4$: F→(·E)
E→·E+T
E→·T
T→·T*F
T→·F
F→· (E)
F→·i

$I_5$: F→i·

$I_6$: E→E+·T
T→·T*F
T→·F
F→·(E)
F→·i

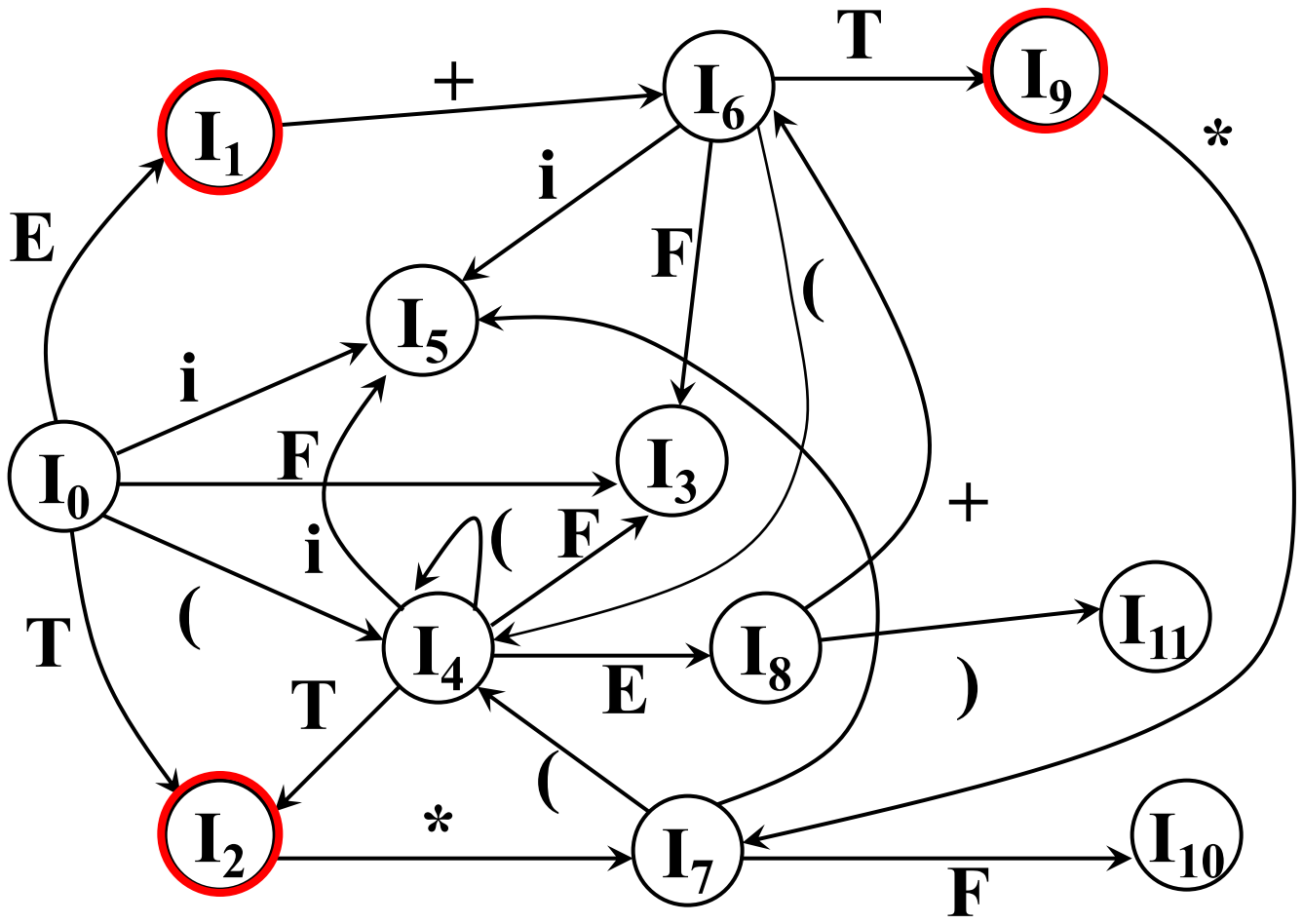
$I_7$: T→T*·F
F→·(E)
F→·i

$I_8$: F→(E·)
E→E·+T

$I_9$: E→E+T·
T→T·*F

$I_{10}$: T→T*F·

$I_{11}$: F→(E)·


I0
I1
I2
I3
I4
I5
I6
I7
I8
I9
I10
I11
E
+
T
i
F
(
)
*

$I_1$、$I_2$和$I_9$都含有“移进 - 归约”冲突。

$I_1$: $S' \rightarrow E \cdot$
$E \rightarrow E \cdot + T$

$I_2$: $E \rightarrow T \cdot$
$T \rightarrow T \cdot * F$

$I_9$: $E \rightarrow E + T \cdot$
$T \rightarrow T \cdot * F$

采取SLR（1）冲突消解

FOLLOW(S′)＝{#}

$I_1$: S′→E·
E→E·+T

其分析表如下:

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

FOLLOW(E)＝{#, ), +}

$I_2$: E→T·
T→T·*F

$I_9$: E→E+T·
T→T·*F

其分析表如下:

| 状态 | ACTION | | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|---|
| | i | + | * | ( | ) | # | E | T | F |
| 0 | s5 | | | s4 | | | 1 | 2 | 3 |
| 1 | | s6 | | | | acc | | | |
| 2 | | r2 | s7 | | r2 | r2 | | | |
| 3 | | r4 | r4 | | r4 | r4 | | | |
| 4 | s5 | | | s4 | | | 8 | 2 | 3 |
| 5 | | r6 | r6 | | r6 | r6 | | | |
| 6 | s5 | | | s4 | | | | 9 | 3 |
| 7 | s5 | | | s4 | | | | | 10 |
| 8 | | s6 | | | s11 | | | | |
| 9 | | r1 | s7 | | r1 | r1 | | | |
| 10 | | r3 | r3 | | r3 | r3 | | | |
| 11 | | r5 | r5 | | r5 | r5 | | | |

# SLR分析表的构造

- LR(0)文法太简单，没有实用价值
- 假定一个LR(0)规范族中含有如下的一个项目集(状态)

$$I=\{X\rightarrow\alpha\cdot b\beta，A\rightarrow\alpha\cdot，B\rightarrow\alpha\cdot\}。$$

- FOLLOW(A)和FOLLOW(B)的交集为∅，且不包含b，那么，当状态I面临任何输入符号*x*时，可以:

**1. 若x=b，则移进；**
**2. 若x∈FOLLOW(A)，用产生式A→α进行归约；**
**3. 若x∈FOLLOW(B)，用产生式B→α进行归约；**
**4. 此外，报错。**

- 冲突性动作的这种解决办法叫做SLR(1)解决办法。

# LR(0)分析表的ACTION和GOTO子表构造

1. 若项目A→α·aβ属于$I_k$且GO($I_k$,a)=$I_j$，a为终结符，则置ACTION[k,a]为 “sj”；

2. 若项目A→α·属于$I_k$，那么，对任何终结符a(或结束符#)，置ACTION[k,a]为 “rj”；其中，假定A→α为文法G′的第j个产生式；

3. 若项目S′→S·属于$I_k$，则置ACTION[k,#]为“acc”；

4. 若GO($I_k$,A) = $I_j$，A为非终结符，则置GOTO[k,A]=j；

5. 分析表中凡不能用规则1至4填入信息的空白格均置上“出错标志”。

# SLR(1)分析表的ACTION和GOTO子表构造

1. 若项目A→α·aβ属于$I_k$且GO($I_k$,a)=$I_j$，a为终结符，则置ACTION[k,a]为 “sj”；

2. 若项目A→α·属于$I_k$，那么，对任何终结符a，a∈FOLLOW(A)，置ACTION[k,a]为 “rj”；其中，假定A→α为文法G′的第j个产生式；

3. 若项目S′→S·属于$I_k$，则置ACTION[k,#]为“acc”；

4. 若GO($I_k$,A) = $I_j$，A为非终结符，则置GOTO[k,A]=j；

5. 分析表中凡不能用规则1至4填入信息的空白格均置上“出错标志”。

# SLR(1)文法

- 按上述方法构造出的ACTION与GOTO表如果不含多重入口，则称该文法为SLR(1)文法
- 使用SLR表的分析器叫做一个SLR分析器
- 每个SLR(1)文法都是无二义的。但也存在许多无二义文法不是SLR(1)的
- LR(0) ⊂ SLR(1) ⊂ 无二义文法

# SLR(1) 分析的特点

- 描述能力强于 LR(0)
  - SLR(1)还考虑Follow集中的符号
  - LR(0) 仅考虑产生式的首符号
- SLR(1) 文法：SLR(1)分析表无冲突的CFG
  - 一个非SLR(1) 文法
    - (0) S′→S
    - (1) S→L=R
    - (2) S→R
    - (3) L→*R
    - (4) L→i
    - (5) R→L

# SLR(1)分析的局限性

- 如果 SLR(1) 分析表仍有多重入口（移进归约冲突或归约归约冲突），则说明该文法不是 SLR(1) 文法；
- 说明仅使用 LR(0) 项目集和 FOLLOW 集还不足以分析这种文法

$I_2 = \{S \rightarrow L.=R,\ R \rightarrow L.\ \}$

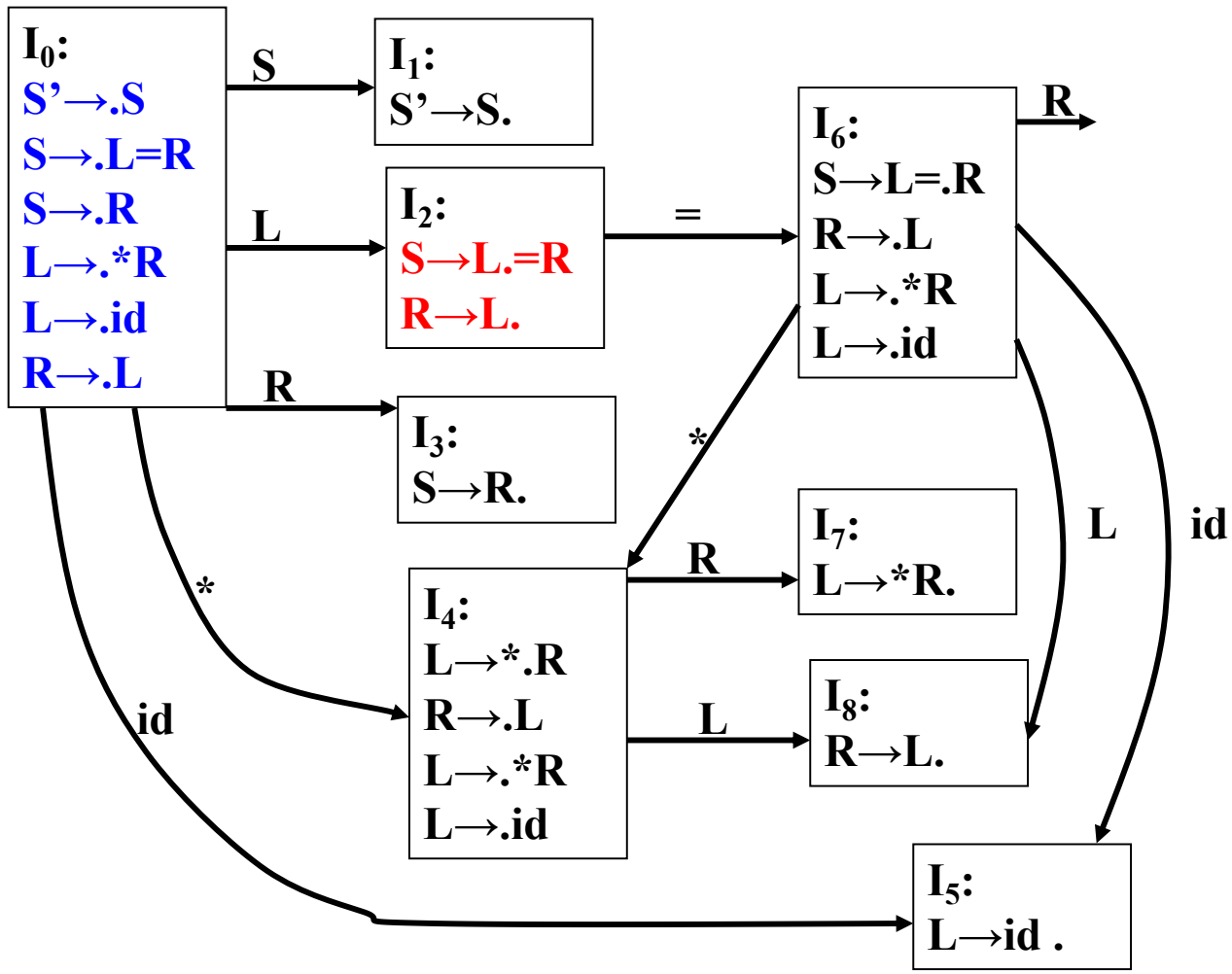
- 输入符号为 = 时，出现了移进归约冲突：

$S \rightarrow L .=R \in I_2$ and $go(I_2,=)=I_6$

$\Rightarrow$ action[2,=] = S6

$R \rightarrow L . \in I_2$ and $= \in FOLLOW(R)=\{=,\#\ \}$

$\Rightarrow$ action[2,=] = Reduce $R \rightarrow L$



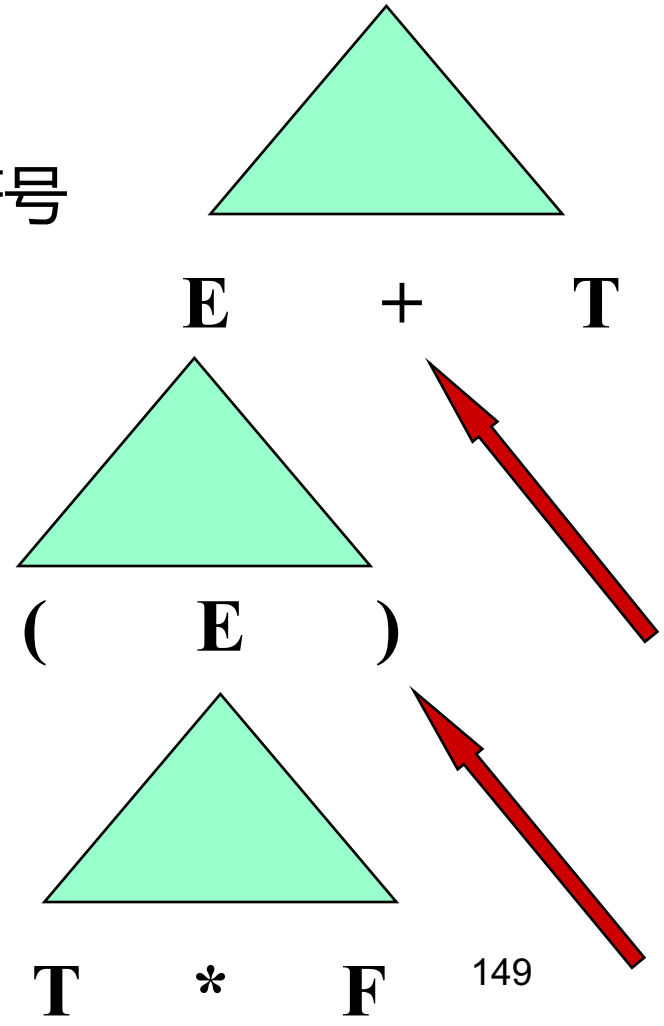

- 说明该文法不是SLR(1)文法，分析这种文法需要更多的信息。

# SLR(1)分析中存在冲突的原因

- SLR（1）只孤立地考察输入符号是否属于归约项目A→α.相关联的集合FOLLOW（A），而没有考察符号串α所在规范句型的“上下文”。
- 所以试图用某一产生式A→α归约栈顶符号串α时，不仅要向前扫描一个输入符号，还要查看栈中的符号串δα，只有当δAa的确构成文法某一规范句型的活前缀时才能用A→α归约。（规约的有效性）
- 问题：怎样确定δAa是否是文法某一规范句型的活前缀

# 5.3.4 LR(1)分析表的构造

- LR(0)不考虑后继符(搜索符)，SLR(1)仅在归约时考虑后继符(搜索符)，因此，对后继符(搜索符)所含信息量的利用有限，未考虑栈中内容。
- 希望在构造状态时就考虑后继符(搜索符)的作用：考虑对于产生式 A→α的归约，不同使用位置的 A 会要求不同的后继符号




- 不同的归约中有不同的后继符。
- 特定位置的后继符是FOLLOW集的子集

# LR(k) 项目

- 定义5.11
  - $[A\rightarrow\alpha.\beta, a_1a_2\ldots a_k]$为LR（k）项目，根据圆点所处位置的不同又分为三类：
    - 归约项目: $[A\rightarrow\alpha., a_1a_2\ldots a_k]$
    - 移进项目：$[A\rightarrow\alpha.a\beta, a_1a_2\ldots a_k]$
    - 待约项目：$[A\rightarrow\alpha.B\beta, a_1a_2\ldots a_k]$

利用LR(k)项目进行(构造)LR(k)分析(器)，当k=1时，为LR(1)项目，相应的分析叫LR(1)分析(器)

# LR(1) 项目的有效性

- 形式上
  - 称LR(1)项目[A→α.β,a]对活前缀γ=δα是有效的，如果存在规范推导
  - $S \Rightarrow^{*} \delta Aw \Rightarrow \delta\alpha B\beta w$
  - 其中a为w的首字符，如果w=ε，则a=#
- 与LR(0)文法类似，识别文法全部活前缀的DFA的每一状态也是用一个LR(1)项目集来表示，为保证分析时，每一步都在栈中得到规范句型的活前缀，应使每一个LR(1)项目集仅由若干个对相应活前缀有效的项目组成

# 识别文法全部活前缀的DFA

## LR(1) 项目集族的求法

- CLOSURE（I）：求I的闭包，目的是为了合并某些状态，节省空间
- GO（I，X）：转移函数

# 闭包的计算

(0) S′→S
(1) S→L=R
(2) S→R
(3) L→*R
(4) L→i
(5) R→L

- CLOSURE(I)的计算
  - (核心位置：A→α.Bβ, a 扩展成闭包)
- 如果[A→α.Bβ,a]对γ=δα有效

  /*即存在S⇒*δAax⇒δαBβax*/
- 假定βax⇒*by，则对任意的B→η有：
  - [B→.η，b]对γ=δα也是有效的，其中
  - b∈FIRST（βa）

**$I_0$：**
**S′→·S, #**
**S→·L=R, #**
**S→·R, #**
**L→·*R, =**
**L→·I, =**
**R→·L, #**
**L→·*R, #**
**L→·I, #**

# 闭包的计算

- CLOSURE(I)的计算
  - (核心位置：A→α.Bβ, a 扩展成闭包)
- 如果[A→α.Bβ,a]对γ=δα有效

  /*即存在S⇒*δAax⇒δαBβax*/
- 假定βax⇒*by，则对任意的B→η有：
  - [B→.η，b]对γ=δα也是有效的，其中
  - b∈FIRST（βa）

**$I_0$：**
**S′→·S, #**
**S→·L=R, #**
**S→·R, #**
**L→·*R, =/#**
**L→·I, =/#**
**R→·L, #**

# 闭包的计算和转移

闭包计算

J:=I;

repeat

J=J∪{[B→.η,b]|[A→α.Bβ,a]∈J,

b∈FIRST(βa)}

until J 不再扩大

当$\beta \Rightarrow^{+} \varepsilon$时，此时b=a叫继承的后继符，否则叫自生的后继符

转移

go(I,X) =

closure([A→αX.β,a]|[A→α.Xβ,a]∈I)

(0) S′→S

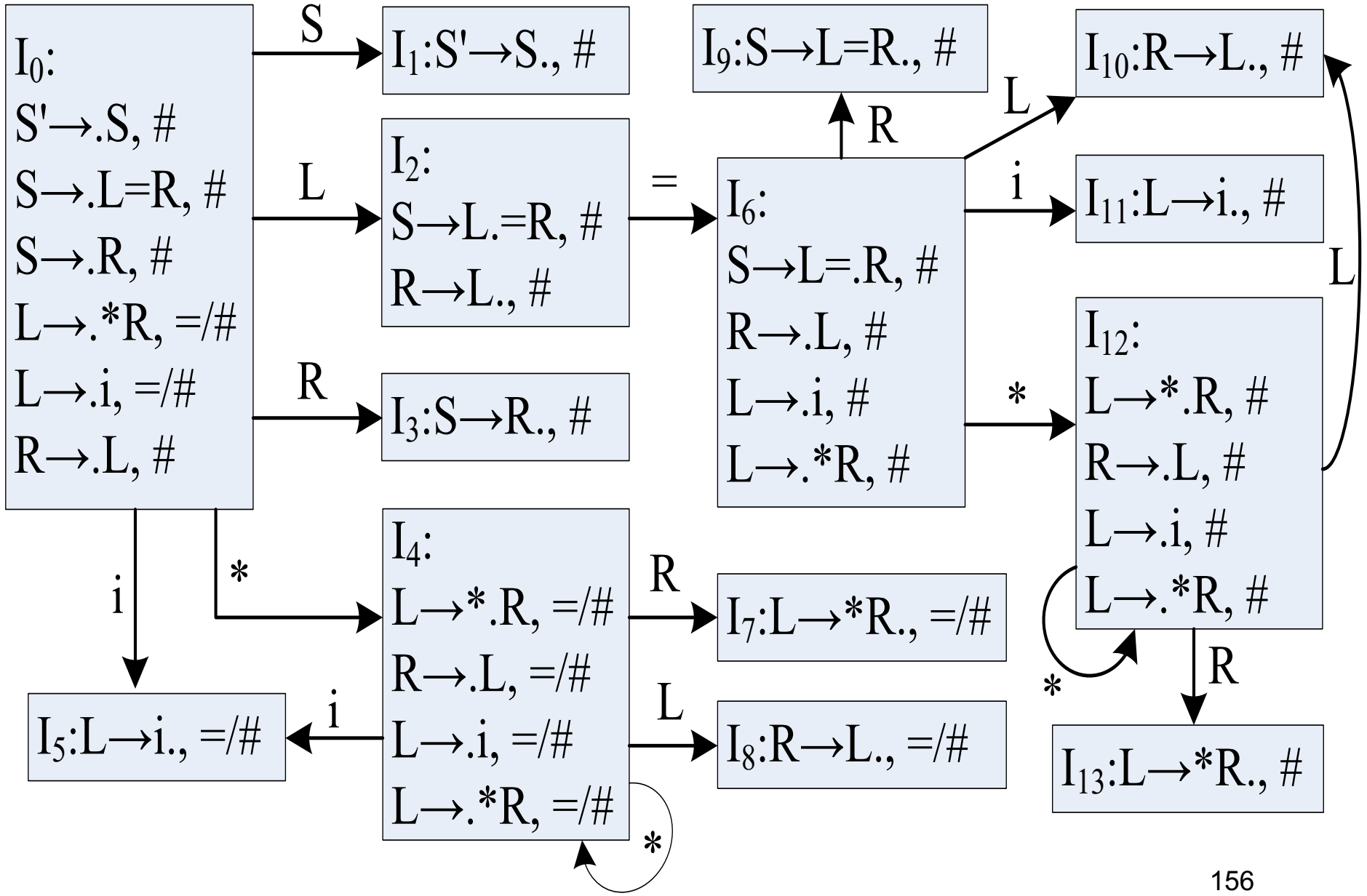
(1) S→L=R

(2) S→R

(3) L→*R

(4) L→i

(5) R→L



# 计算LR(1)项目集规范族C 即：分析器状态集合

C={$I_0$}∪{I|∃J∈C，X∈V∪T，I=go(J,X)}称为G'的LR(1)项目集规范族（算法：P181)

```
begin
C:= {closure({ S'→.S,#})};
  repeat
     for ∀I∈C，∀ X ∈ V∪T
        if go(I,X)≠Φ & go(I,X)∉C then
              C=C∪go(I,X)
  until C不变化
end.
```

# 识别活前缀的关于LR(1) 的DFA

- 识别文法G=（V，T，P，S）的拓广文法G’的所有活前缀的

$$\text{DFA } M=(C, V\cup T, go, I_0, C)$$

  - $I_0$=CLOSURE({S’ →.S, #}

- 如果CFG G的LR(1)分析表无冲突则称G为LR(1)文法

# LR(1) 分析表的构造

1．令$I_0$= CLOSURE({S'→.S})，构造C={ $I_0$, $I_1$, …, $I_n$}，即G'的LR(1)项目集规范族。

2．从Ii构造状态i，0为初始状态。

for k=0 to n do

begin

(1) if [A→α.aβ, b]∈$I_k$ & a∈T & GO($I_k$, a)=$I_j$ then action[k,a]:=Sj;

(2) if GO($I_k$, B)=$I_j$ & B∈V then goto[k,B]:=j;

(3) if [A→α., a]∈$I_k$ & A→α为G'的第j个产生式then action[k,a]:=rj;

(4) if [S'→S., #]∈$I_k$ then action[k,#]:=acc;

end

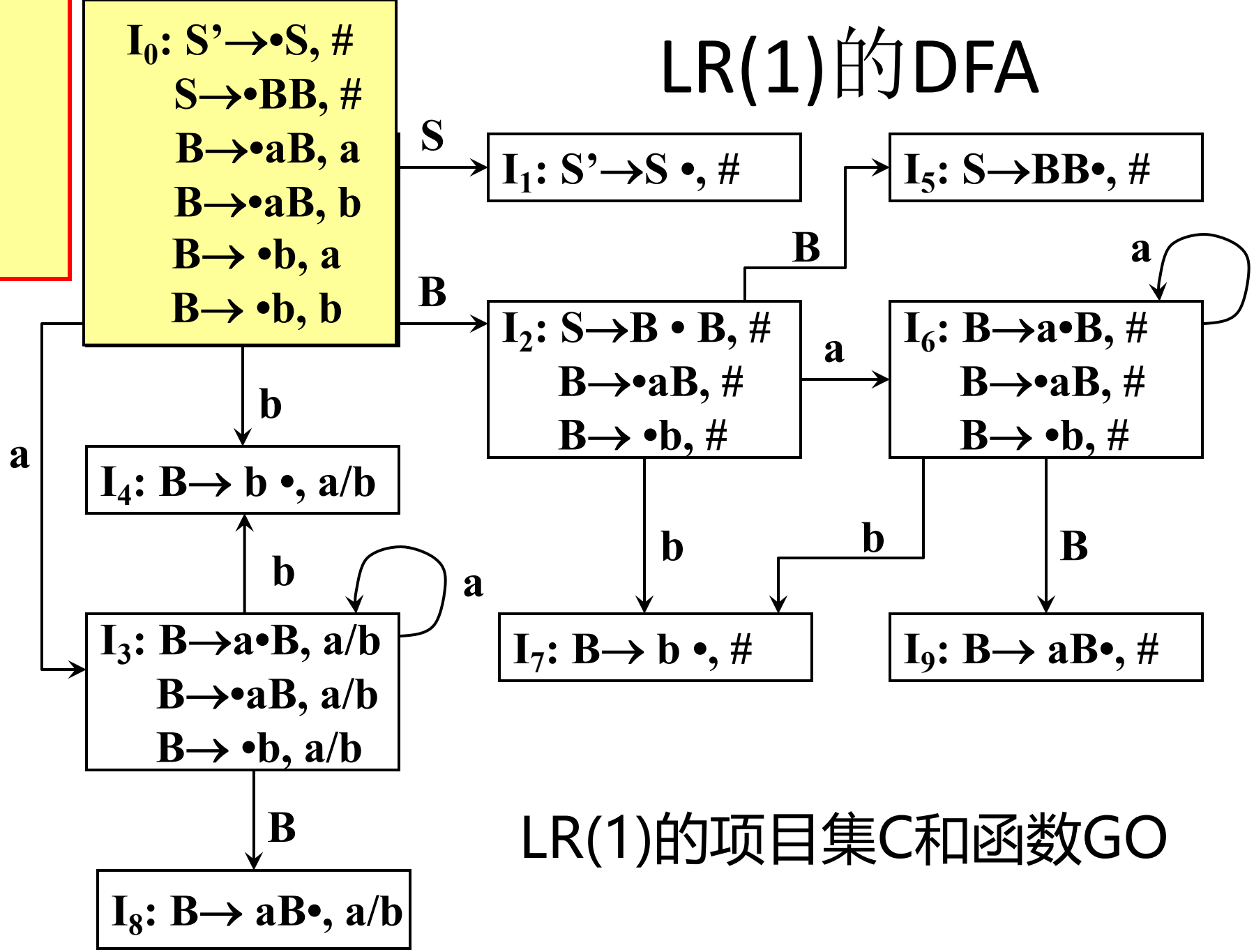
- 上述(1)到(4)步未填入信息的表项均置为error。

# LR(1) 分析表的构造

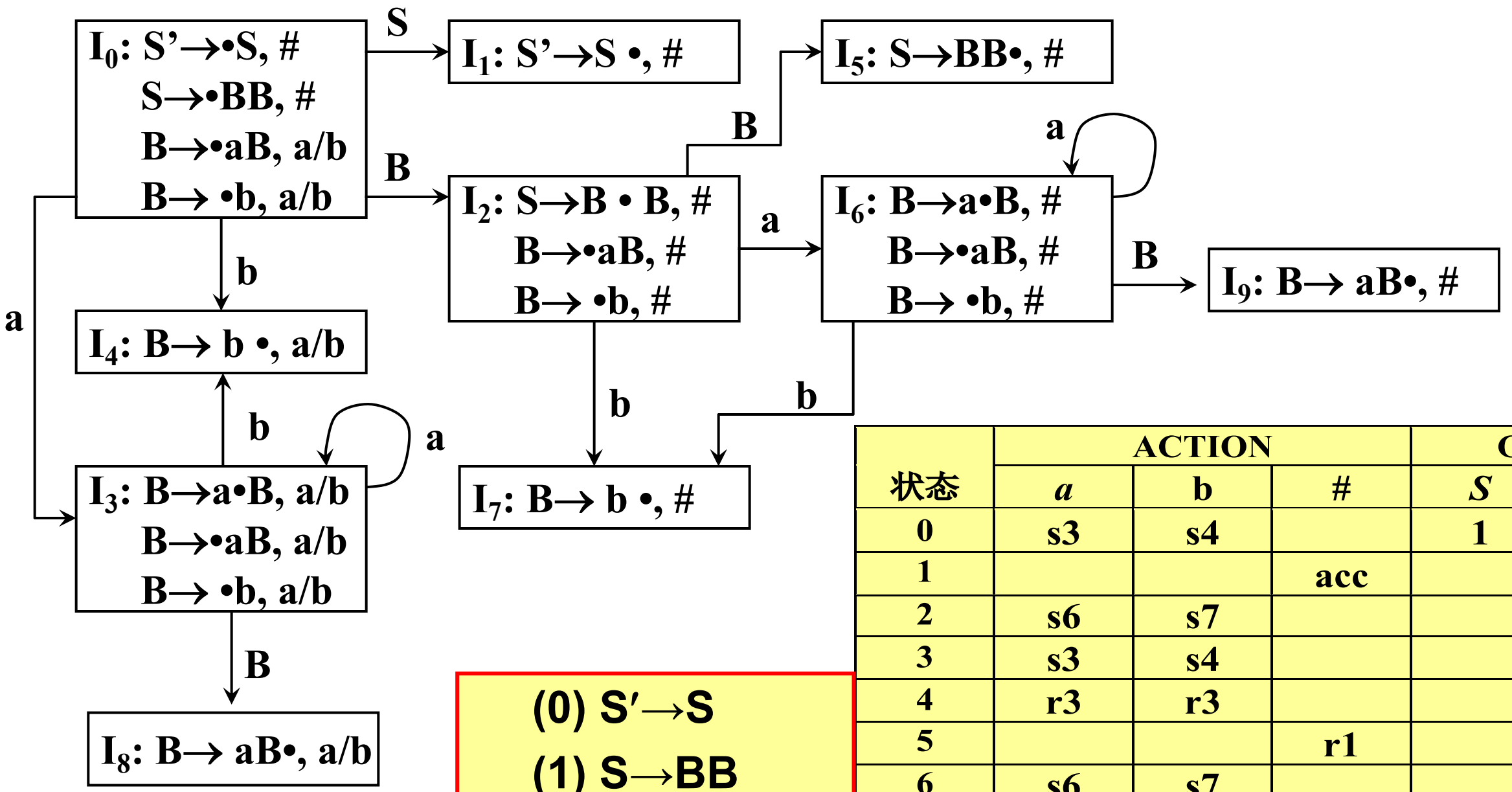
- 与LR(0)的不同点主要在归约动作的选择：
  - LR(0) 分析考虑所有终结符
  - SLR(1) 分析参考 FOLLOW 集
  - LR(1) 分析仅考虑 LR(1)项目中的后继符

(0) S′→S
(1) S→BB
(2) B→aB
(3) B→b

# LR(1)的DFA




LR(1)的项目集C和函数GO




(0) S′→S

(1) S→BB

(2) B→aB

(3) B→b

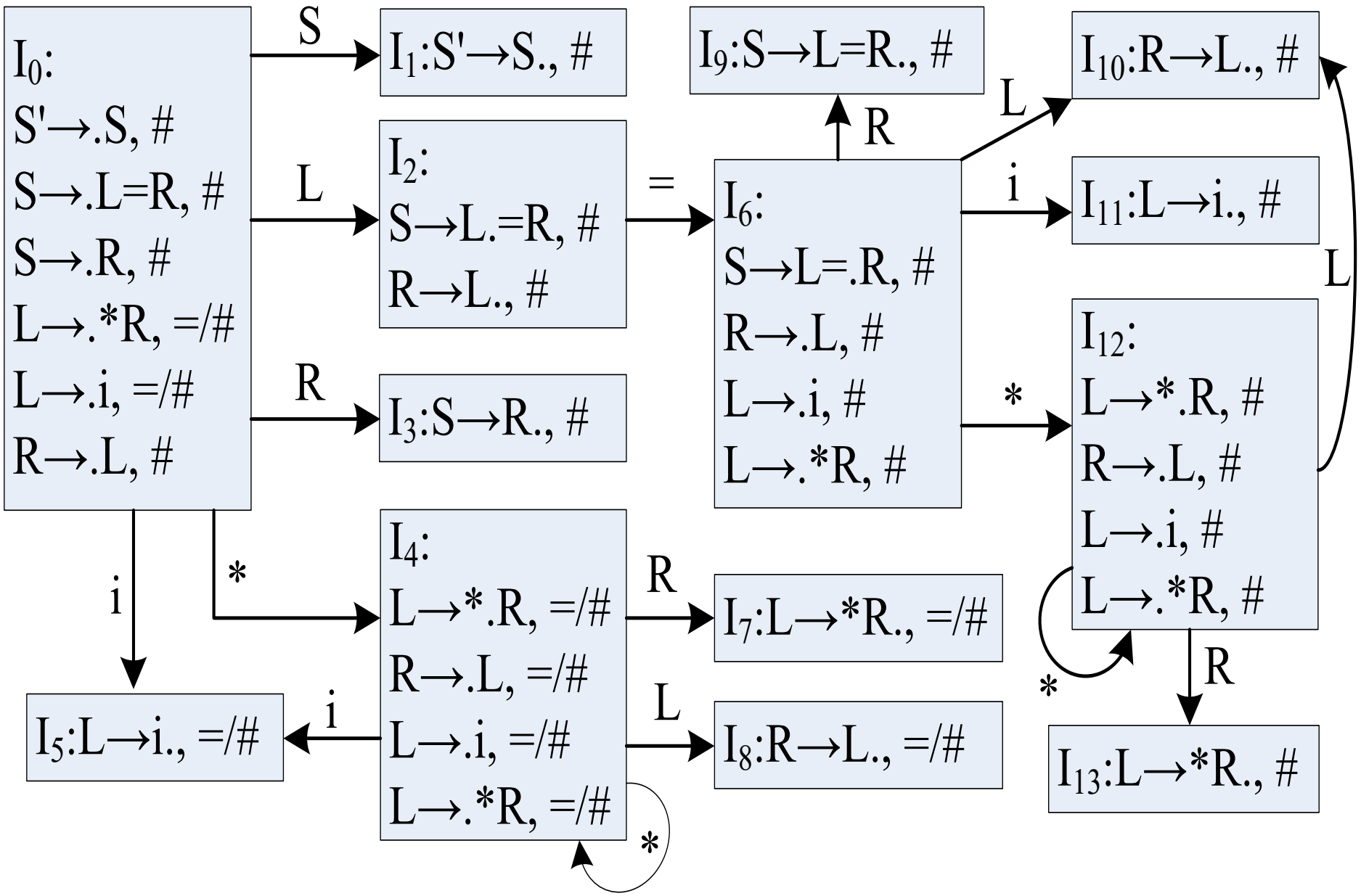
| 状态 | ACTION | | | GOTO | |
|---|---|---|---|---|---|
| | *a* | b | # | *S* | B |
| 0 | s3 | s4 | | 1 | 2 |
| 1 | | | acc | | |
| 2 | s6 | s7 | | | 5 |
| 3 | s3 | s4 | | | 8 |
| 4 | r3 | r3 | | | |
| 5 | | | r1 | | |
| 6 | s6 | s7 | | | 9 |
| 7 | | | r3 | | |
| 8 | r2 | r2 | | | |
| 9 | | | r2 | | |

# 对aabab进行分析

| 步骤 | 状态 | 符号 | 输入串 |
|---|---|---|---|
| 0 | 0 | # | aabab# |
| 1 | 03 | #*a* | abab# |
| 2 | 033 | #aa | bab# |
| 3 | 0334 | #aab | ab# |
| 4 | 0338 | #aaB | ab# |
| 5 | 038 | #aB | ab# |
| 6 | 02 | #B | ab# |
| 7 | 026 | #Ba | b# |
| 8 | 0267 | #Bab | # |
| 9 | 0269 | #BaB | # |
| 10 | 025 | #BB | # |
| 11 | 01 | #*S* | # |

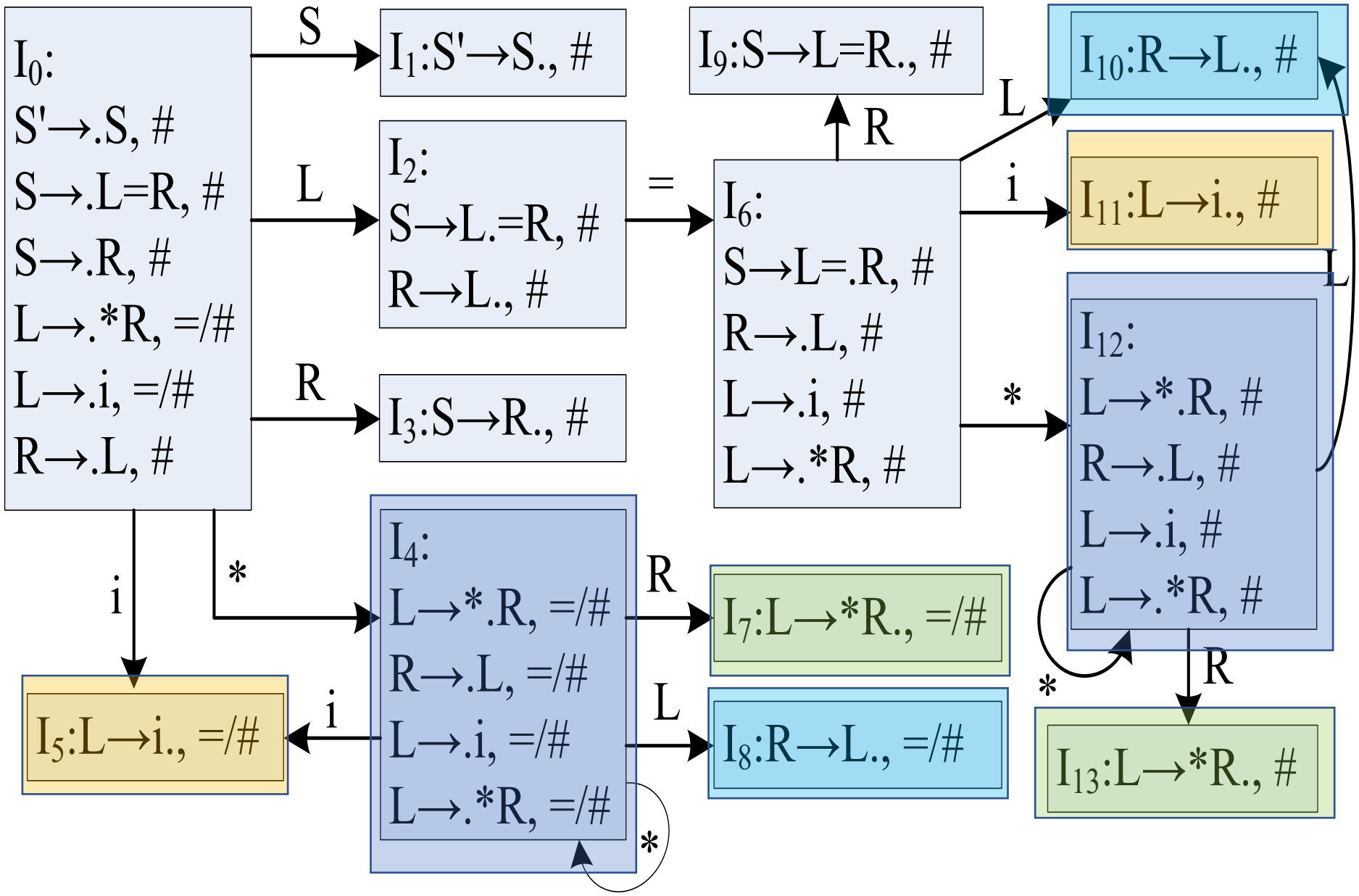
**(0) S′→S**
**(1) S→BB**
**(2) B→aB**
**(3) B→b**

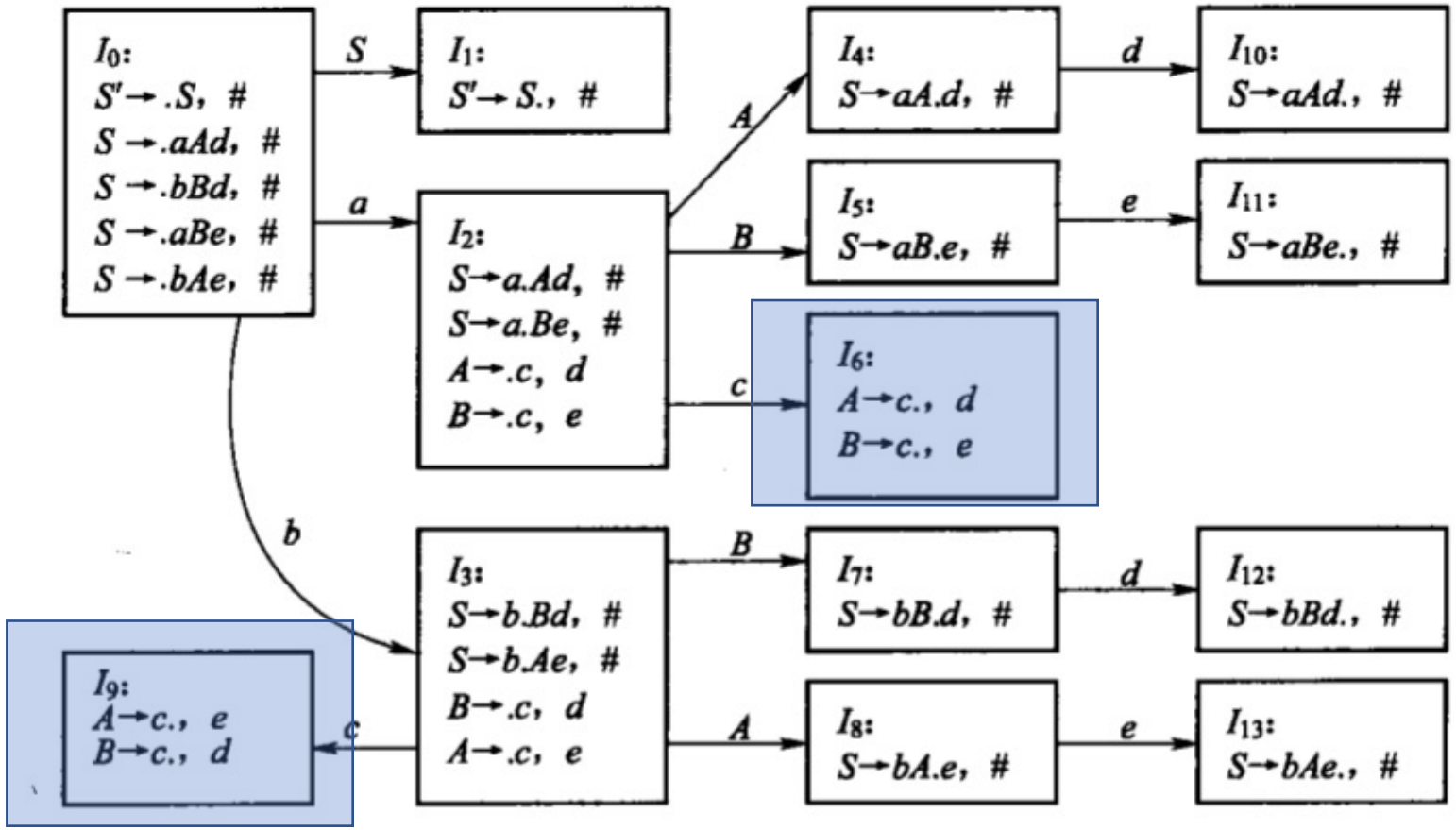
| 状态 | ACTION | | | GOTO | |
|---|---|---|---|---|---|
| | *a* | b | # | *S* | B |
| 0 | s3 | s4 | | 1 | 2 |
| 1 | | | acc | | |
| 2 | s6 | s7 | | | 5 |
| 3 | s3 | s4 | | | 8 |
| 4 | r3 | r3 | | | |
| 5 | | | r1 | | |
| 6 | s6 | s7 | | | 9 |
| 7 | | | r3 | | |
| 8 | r2 | r2 | | | |
| 9 | | | r2 | | |


I0:
S'→.S, #
S→.L=R, #
S→.R, #
L→.*R, =/#
L→.i, =/#
R→.L, #
S
L
R
i
*
I1:S'→S., #
I2:
S→L.=R, #
R→L., #
I3:S→R., #
I4:
L→*.R, =/#
R→.L, =/#
L→.i, =/#
L→.*R, =/#
I5:L→i., =/#
=
I6:
S→L=.R, #
R→.L, #
L→.i, #
L→.*R, #
I7:L→*R., =/#
I8:R→L., =/#
I9:S→L=R., #
I10:R→L., #
I11:L→i., #
I12:
L→*.R, #
R→.L, #
L→.i, #
L→.*R, #
I13:L→*R., #

# 5.3.5 LALR(1)分析表的构造

- LR(1)对应的C太大
- 问题：是否可以将某些闭包/状态合并?
  - 不同的LR(1)项目闭包可能有相同的LR(0)项目，但后继符可能不同——同心
  - 合并后可能带来归约-归约冲突
  - 合并那些不会带来冲突的同心的LR(1)闭包/状态
- ( lookahead-LR )
  - 在不带来归约归约冲突的条件下，合并状态，重构分析表


I0:
S'→.S, #
S→.L=R, #
S→.R, #
L→.*R, =/#
L→.i, =/#
R→.L, #
S
L
R
i
*
I1:S'→S., #
I2:
S→L.=R, #
R→L., #
I3:S→R., #
=
I6:
S→L=.R, #
R→.L, #
L→.i, #
L→.*R, #
I9:S→L=R., #
R
L
i
*
I10:R→L., #
I11:L→i., #
I12:
L→*.R, #
R→.L, #
L→.i, #
L→.*R, #
L
R
*
I4:
L→*.R, =/#
R→.L, =/#
L→.i, =/#
L→.*R, =/#
R
L
i
*
I5:L→i., =/#
I7:L→*R., =/#
I8:R→L., =/#
I13:L→*R., #




$I_{6/9}: A \rightarrow c.\ ,d/e$

$B \rightarrow c.\ ,d/e$

归约归约冲突

# LALR(1) 的分析能力

- 强于 SLR(1)
  - 合并的后继符仍为 FOLLOW 集的子集
- 局限性
  - 合并中不出现归约-归约冲突
  - 如果CFG G的LALR(1)分析表无冲突则称G为LALR(1)文法

# 5.3.6 二义性文法的应用

(1) $E \rightarrow E + E$　　(2) $E \rightarrow E * E$　　(3) $E \rightarrow (E)$　　(4) $E \rightarrow \mathbf{id}$

$I_1$:
E'→E.
E→E . +E
E→E.*E

$I_7$:
E→E+E.
E→E.+E
E→E.*E

$I_8$:
E→E*E.
E→E.+E
E→E.*E

- 采用二义性文法，可以减少结果分析器的状态数，并能减少对单非终结符（ E→T ）的归约。
- 在构造分析表时采用消除二义性的规则(按优先级)

# 5.3.6 二义性文法的应用

$I_1$:
E'→E.
E→E . +E
E→E.*E

$I_7$:
E→E+E.
E→E.+E
E→E.*E

$I_8$:
E→E*E.
E→E.+E
E→E.*E

表 5.13　文法(5.1)的 LR 分析表

| | action | | | | | | goto |
|---|---|---|---|---|---|---|---|
| | + | * | ( | ) | id | # | E |
| 0 | | | S2 | | S3 | | 1 |
| 1 | S4 | S5 | | | | acc | |
| 2 | | | S2 | | S3 | | 6 |
| 3 | r4 | r4 | | r4 | | r4 | |
| 4 | | | S2 | | S3 | | 7 |

# 5.3.6 二义性文法的应用

$I_1$:
E'→E.
E→E . +E
E→E.*E

$I_7$:
E→E+E.
E→E.+E
E→E.*E

$I_8$:
E→E*E.
E→E.+E
E→E.*E

| | action | | | | | | goto |
|---|---|---|---|---|---|---|---|
| | + | * | ( | ) | id | # | E |
| 5 | | | S2 | | S3 | | 8 |
| 6 | S4 | S5 | | S9 | | | |
| 7 | r1 | S5 | | r1 | | r1 | |
| 8 | r2 | r2 | | r2 | | r2 | |
| 9 | r3 | r3 | | r3 | | r3 | |

# 5.3.6 二义性文法的应用

$I_1$:
E'→E.
E→E . +E
E→E.*E

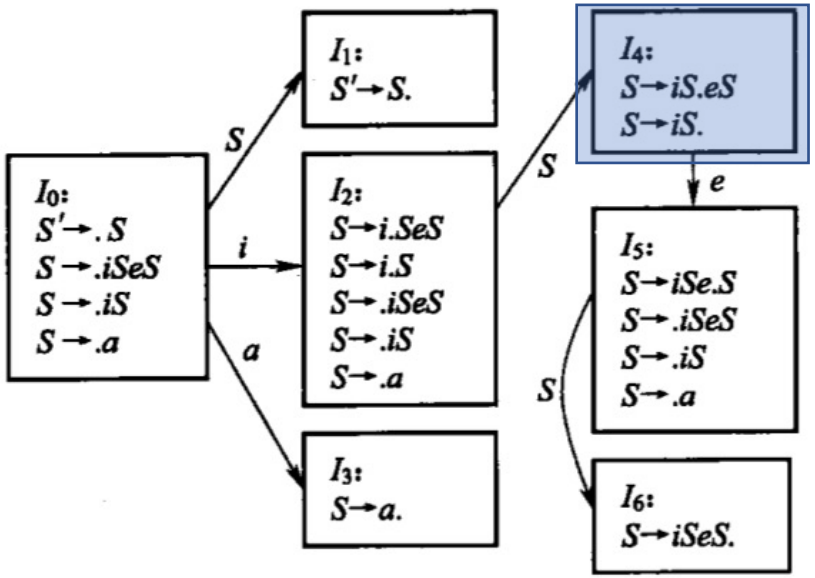
$I_7$:
E→E+E.
E→E.+E
E→E.*E

$I_8$:
E→E*E.
E→E.+E
E→E.*E

| | action | | | | | | goto |
|---|---|---|---|---|---|---|---|
| | + | * | ( | ) | id | # | E |
| 5 | | | S2 | | S3 | | 8 |
| 6 | S4 | S5 | | S9 | | | |
| 7 | r1 | S5 | | r1 | | r1 | |
| 8 | r2 | r2 | | r2 | | r2 | |
| 9 | r3 | r3 | | r3 | | r3 | |

# 5.3.6 二义性文法的应用

(1) $S \rightarrow iSeS$　　(2) $S \rightarrow iS$　　(3) $S \rightarrow a$




| | action | | | | goto |
|---|---|---|---|---|---|
| | *i* | *e* | *a* | # | *S* |
| 1 | | | | acc | |
| 2 | *S*2 | | *S*3 | | 4 |
| 3 | | *r*3 | | *r*3 | |
| 4 | | *S*5 | | *r*2 | |
| 5 | *S*2 | | *S*3 | | 6 |
| 6 | | *r*1 | | *r*1 | |

# 5.3.7 LR分析中的出错处理

- 当分析器处于某一状态$S$，且当前输入符号为$a$时，就以符号对$(S，a)$查$LR$分析表，如果分析表元素$action[S，a]$为空(或出错)，则表示检测到了一个语法错误。
- 紧急方式的错误恢复：从栈顶开始退栈，直至发现在特定语法变量$A$上具有转移的状态$S$为止，然后丢弃零个或多个输入符号，直至找到符号$a \in \mathrm{FOLLOW}(A)$为止。接着，分析器把状态$goto[S，A]$压进栈，并恢复正常分析。

# 5.4 语法分析程序的自动生成工具Yacc

- YSP(Yacc Specification)

%{变量定义：头文件和全局变量

%开始符号

词汇表：%Token $n_1$,$n_2$,…(自动定义种别码)

%Token $n_1$,$i_1$（用户指定种别码）

……

%Token $n_h$,$i_h$（用户指定种别码）

类型说明 %type

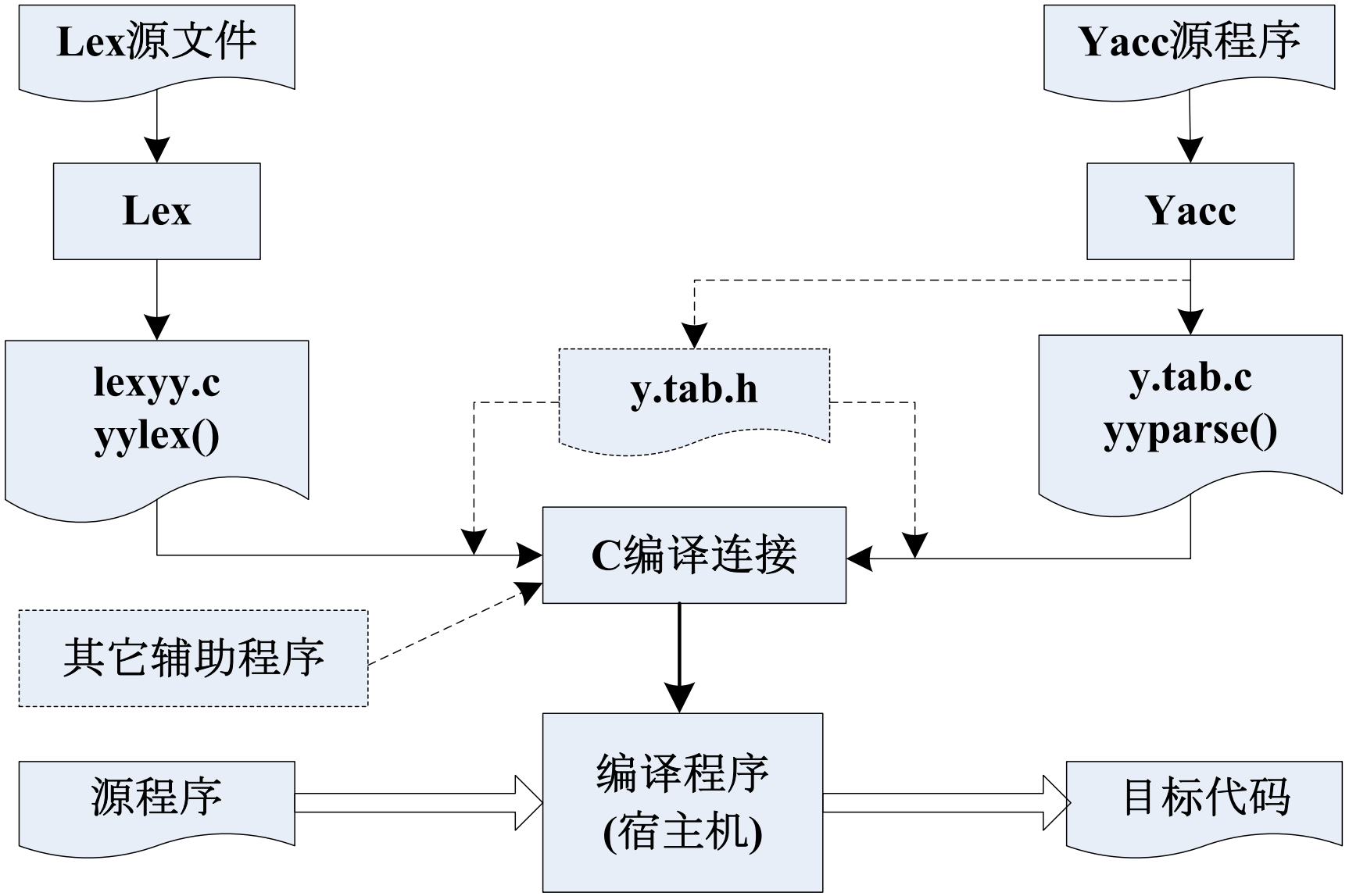
其它说明%}

%%规则部分 给出文法规则的描述

%%程序部分 扫描器和语义动作程序

- 输出：LALR(1)分析器

# 用Yacc和Lex合建编译程序



单选题 1分

设置

若状态k含有项目“A→α·”，且仅当输入符号a∈FOLLOW(A)时，才用规则“A →α”归约的语法分析方法是( )。

A LALR分析法

B R(0)分析法

C LR(1)分析法

D SLR(1)分析法

提交

单选题 1分

设置

在该项目集中含有_____和_____选择使用SLR(1)文法？

A •移进-归约冲突 归约-归约冲突

B •移进-归约冲突 移进-移进冲突

C •移进-移进冲突 归约-归约冲突

D •接受-移进冲突 接受-归约冲突

提交

# 单选题 1分

设置

构造LR分析器的关键是_____。

A •构造其语法表

B •构造其词法表

C •构造其分析表

D •构造其句柄

提交

# 本章小结

- 自底向上的语法分析从给定的输入符号串$w$出发，自底向上地为其建立一棵语法分析树。
- 移进-归约分析是最基本的分析方式，分为优先法和状态法。
- 算符优先分析法是一种有效的方法，通过定义终结符号之间的优先关系来确定移进和归约。
- $LR$分析法有着更宽的适应性。该方法通过构建识别规范句型活前缀的DFA来设计分析过程中的状态。可以将$LR$分析法分成$LR(0)$、$SLR(1)$、$LR(1)$、$LALR(1)$。
- 通过增加附加的信息可以解决一些二义性问题。
- Yacc是$LALR(1)$语法分析器的自动生成工具。